

Entity-Collision: A Stratified Protocol for Attributing Retrieval Lift in Agent Memory

Working draft (anonymized for review). Compiled from paper/ Markdown sources.
Branch: feat/v0.2-governed-memory-and-scale-tests

Entity-Collision: A Stratified Protocol for Attributing Retrieval Lift in Agent Memory

Abstract

End-to-end agent-memory benchmarks report a single hit@k per retriever, confounding **lexical leakage** (uncontrolled query/gold/distractor entity overlap) with **tag-mixing** (preferences, services, tools averaged together). We propose **entity-collision**, a system-agnostic protocol that pins the BM25 floor by construction — every distractor shares the answer's entity tokens — and stratifies queries by discriminator tag, so any lift over BM25 is attributable to the embedder. Applied to an open-source agent-memory testbed across 5 tags $\times$ 3 embedders $\times$ 5 collision degrees with paired-bootstrap 95% CIs, the protocol reveals a **two-axis pattern**: a 256-d hash trigram helps only on closed-vocabulary lexical tags at deep collision; MiniLM-384 dominates both axes; and a 2.7 $\times$ -parameter BGE-large does **not** uniformly improve on MiniLM — it wins on intent-style queries but loses on lexical ones. Encoder capacity alone is not the binding constraint. The synthetic intent-tag null replicates on LongMemEval (n=500) as a single-session-preference recall cliff. Adaptive vector-weight routing on LoCoMo is a measured null: 11.7 pp of oracle headroom exists, but no signal we tested recovers it. All 26 result tables and 37 reproduce scripts are version-controlled and verified by a public registry; the protocol is exercised on a **deterministically governed** memory testbed (event-sourced decision log, DAG-state-machine schema lifecycle) so every reported CI is reproducible byte-for-byte from the ingest stream.

1. Introduction (draft)

Agent-memory systems are increasingly deployed as the long-context substrate for LLM assistants — Letta, Mem0, MemGPT, and the "governed memory" line all argue that an external memory store beats unbounded context windows on cost, latency, and recall. A recurring open question across these systems is **what retrieval machinery is actually earning its keep**: is BM25 enough? Does dense embedding help? When? **For a team deciding which embedder to ship behind their agent memory, the answer determines model-load cost, ingest latency, and recall headroom — but the field's standard end-to-end benchmarks confound these decisions with lexical-leakage and tag-mixing artifacts.**

This question is normally answered with end-to-end benchmarks (LongMemEval, LoCoMo) that report a single hit@k number per retriever. We argue this is insufficient, for two reasons:

1. **Lexical leakage.** End-to-end benchmarks usually have queries that share the answer's entity tokens with the gold passage but not with distractors. BM25 then trivially wins, and any embedder "lift" is confounded with lexical anchor strength.
2. **Tag-mixing.** Benchmarks blend categories (preferences, projects, technical facts, services, tools) into a single number. A retriever may be systematically better on some tags and worse on others; the average obscures this.

We address both with an **entity-collision protocol**: synthetic queries where every distractor shares the answer's entity tokens (BM25 floor fixed by construction), stratified by discriminator tag. We report results across 5 tags, 2 embedders, and 5 collision degrees with paired bootstrap 95% CIs (see Figure 1, §4.1, for the headline three-panel comparison). To check that the synthetic protocol's findings transfer, we replicate the per-tag pattern on LongMemEval — a real long-context conversation benchmark — and observe the same intent-tag weakness as a single-session-preference recall cliff.

The work was done on **Engram**, an open-source agent-memory system we built as a controlled testbed. Engram is the artifact through which we run the experiments and which we release for reproducibility (repo URL anonymized for review); it is not itself the contribution of this paper.

A measurable agent-memory system presupposes that its write and merge mechanics are state-governed deterministically: per-query paired-bootstrap CIs, cross-encoder inversions, and replications across three encoders are only credible if re-running a configuration two days later produces a bit-identical memory store. We therefore exercise the entity-collision protocol on a **governed-memory testbed** built around an event-sourced decision log and a directed acyclic state machine for schema lifecycle. The state-machine and linear-scale evidence in §3.7, §4.14, and §5.3 are the foundation that keeps the evaluation clean; they are not independent contributions, and the paper does not claim retrieval lift from any of them.

Contributions

1. **Entity-collision evaluation protocol** that fixes the BM25 floor and isolates the embedder-attributable retrieval lift, with code, query generators, and CI tooling open-sourced under `evals/entity_collision_*`. The protocol is system-agnostic and applies to any retriever exposing a per-document score.
2. **Two-axis empirical finding** with synthetic $\rightarrow$ natural replication and an encoder-capacity falsification. On the synthetic protocol, hash trigrams help on lexical-discriminator tags at deep collision ($K \geq 4$ on `tool`, $K=16$ on `service`) but are null or negative on intent-style tags; a MiniLM sentence-transformer dominates both axes. Extending the encoder grid to BGE-large-en-v1.5 (1024-d, $2.7 \times$ MiniLM's parameter count) does **not** collapse this two-axis structure: BGE wins on intent-style `project` (+8 to +14 pp BGE-MiniLM, all CI-positive at $K \in \{2, 4, 8, 16\}$) but loses on lexical `tool` and `technical` (-2.7 to -11.7 pp at $K \in \{4, 8, 16\}$, all CI-significant). Encoder capacity alone is not the binding constraint. The same per-tag pattern reproduces on LongMemEval ($n=500$): the protocol's intent-tag null predicts the single-session-preference cliff ($\text{hit}@1 = 0.367$ vs 0.81 overall), evidence that the synthetic stratification carries external validity. **The decision-shaped corollary — closed-vocabulary queries can ride a 256-d hash trigram at zero model-load cost, while intent-style queries require a dense encoder — is what a deployment team needs from the protocol that a single hit@k average obscures.**
3. **A null result on adaptive vector-weight routing** for LoCoMo: 11.7 pp of oracle headroom exists per query, but neither gap/crowdedness thresholds nor a leave-one-conversation-out gradient-boosted router recover any of it ($\Delta \text{hit}@1 = -0.0005 [-0.0030, +0.0015]$). The negative result delimits where adaptive retrieval is unlikely to pay without query-distribution-specific priors.

A secondary, systems-side contribution — a schema-lifecycle invariant set (monotone status DAG, deterministic fold over events, reversible demotion only via fresh evidence) — is detailed in §3.7 and §5.3 with the deterministic-replay rationale already framed above. The Hypothesis property-test substrate that pins these invariants is implementation traceability, not a primary claim (see §A6). A 1M-memory SQLite/FTS5 write-latency characterization (tail-100k p99 +13% from 100k $\rightarrow$ 1M; §4.14) likewise serves as testbed sanity rather than a primary claim.

Reproducibility and deployment characterization. All 26 result tables and 37 reproduce scripts cited in the paper are version-controlled and verified by `scripts/verify_repro_artifacts.sh`, which greps every paper file for `bench/results/*.json` and reproduce-script references and asserts each exists on disk at the tagged release. Beyond reproducibility, we report (§3.1.1) the per-encoder model-load and ingest-latency trade-off so the protocol's findings can be priced into a deployment budget, and (§4.14) characterize write-latency tail behaviour out to 1M memories on a single-writer SQLite/FTS5 store to establish that the testbed scales into production-relevant ingest regimes. See `paper/REPRODUCIBILITY.md` for the full registry.

2. Related Work

2.1 Agent memory systems

Three contemporary agent-memory designs span the architectural spectrum from LLM-driven to mechanical governance.

Letta / MemGPT (Packer et al., 2023; arXiv:2310.08560) implements OS-style virtual context management: the LLM itself acts as memory manager via tool calls, paging content between a small main context and an unbounded archival store. Consolidation is agent-driven; the OSS path exposes no write-side dedup primitive.

Mem0 (Mem0 team, 2025; arXiv:2504.19413) is a dynamic extract $\rightarrow$ consolidate $\rightarrow$ retrieve loop. The v3 (April 2026) flip to single-pass ADD-only extraction with cross-memory entity linking reports LoCoMo 91.6 / LongMemEval 94.8 under an LLM-as-judge metric. Engram does not currently ship an LLM-judge mode; direct numerical comparison to Mem0's reported scores is therefore out of scope, and we report session-level hit@k against the same benchmarks instead.

Personize.ai "Governed Memory" (arXiv:2603.17787) is the closest prior in design space. Engram implements the §1-6 stack (dual extraction with per-fact confidence, write-side cosine dedup at 0.92, mechanical merge, schema lifecycle) and extends §7-8 with calibrated contamination/fragmentation meters (§3.5, §A.4.7) and a quorum-gated DEPRECATE primitive (§A.4.6) that hardens schema lifecycle against single-emitter takedown attacks.

Where Engram sits. Mem0 v3 sidesteps governance with ADD-only writes; Letta delegates governance to the LLM-as-memory-manager. Engram's bet is that **mechanical, replayable governance** — extraction confidence per fact, schema transitions through an explicit DAG, prior-sharing gated by calibrated meters — beats LLM-in-the-loop judgment for memory workloads where audit trails and replay determinism are first-class requirements. The empirical question this paper engages is whether such governance costs recall (§4.5, §4.8) and whether the dense-retrieval lift it preserves is uniform across query types (§4.1-4.3). Additional contemporary designs (A-MEM, HippoRAG, Cognee, Zep/Graphiti) are surveyed in §A3.1.

2.2 Benchmarks and evaluation

We evaluate on two community-standard agent-memory benchmarks:

- **LongMemEval** (Wu et al., 2024; arXiv:2410.10813) — 5 question categories over multi-session chats. Headline numbers in §4.8 use the `longmemeval_s` split ($n=500$), SHA-256-pinned in `paper/REPRODUCIBILITY.md` §0.
- **LoCoMo** (Maharana et al., 2024; arXiv:2402.17753) — 10 multi-session conversations, 1978 questions across 5 categories. We report per-category 95% paired-bootstrap CIs across `vector_weight` $\in \{0.0, 0.3, 0.5, 0.7, 1.0\}$ for both HashTrigram-256 and ST MiniLM-384 embedders.

The broader long-context retrieval-evaluation suite from which we draw stratification methodology (RULER, NIAH, ∞ Bench, LongBench-v2, LV-Eval/LoogLe/L-Eval) is detailed in §A3.2.

2.3 Retrieval baselines

BM25 as a strong baseline (Thakur et al., 2021, BEIR; arXiv:2104.08663) anchors the entity-collision protocol: distractor-shared entity tokens fix the BM25 lexical floor by construction so any lift over BM25 is attributable to the embedder (§3, §4.1).

Hash-trigram / sketched embeddings (Weinberger et al., 2009, *ICML*) provide a model-load-free alternative to dense embedders. The two-axis result of §4.3 quantifies exactly when this trade-off is acceptable: lexical-discriminator queries at deep collision recover ~50% of dense-embedder lift; intent-style queries do not.

The dense-retrieval evaluation ecosystem (BEIR, MTEB, MS MARCO, TREC DL), our explicit non-inclusion of late-interaction (ColBERT) and learned-sparse (SPLADE) retrievers, the pseudo-relevance feedback line (RM3, Rocchio), and the event-sourcing / bi-temporal / CRDT lineage that informs §3.7.4 schema lifecycle are surveyed in §A3.3-§A3.4.

3. Methods

3.1 Engram retrieval cell

The unit-under-test is a single Engram retrieval call:

```
results = engine.recall(query, k, vector_weight=vw)
```

with the BM25 score (FTS5 over a tokenized text column) and an embedder-cosine score combined as

```
score = (1 - vw) * bm25_norm + vw * cos_sim
```

where `bm25_norm` is min-max normalized over the candidate set. We sweep `vw` $\in \{0.0, 0.3, 0.5, 0.7, 1.0\}$, with `vw=0.0` being the "BM25-only" floor.

Three embedders are compared: **HashTrigram-256** (character-trigram hashing, 256-dim signed bag-of-trigrams, L2-normalized; zero model load, ~9.6 ms p50 write at 10k); **ST MiniLM-384** (sentence-transformers/all-MiniLM-L6-v2, 384-dim, normalized; ~17–21 ms p50 write at 1k); **BGE-large-1024** (BAAI/bge-large-en-v1.5, 1024-dim, normalized; 2.7× MiniLM's parameter count; ~21.5 s/instance LongMemEval ingest on Apple Silicon MPS, ~150.8 s/instance on commodity CPU).

3.1.1 Encoder latency-cost trade-off

The three encoders span ~1000× in per-instance ingest cost on commodity CPU and ~80× on Apple Silicon MPS. The headline-recall column is the K=16 lexical-tag entity-collision lift on `service` (the strongest hash-trigram cell) and the LongMemEval-S n=500 paired $\Delta\text{hit}@1$ vs MiniLM (the strongest BGE cell); the deployment-cost column folds in model-load (one-time, amortized) and per-instance ingest (paid per write). Numbers from the artifacts cited in §4 and §A.4.16; per-row source column points at the canonical reproduce path.

encoder	dim	model load (s)	ingest p50 / inst (CPU)	ingest p50 / inst (MPS)	recall p50 / q	headline lift	source
HashTrigram-256	256	~0.0	~525 ms	—	~9.6 ms	+5.7 pp $\Delta\text{hit}@1$ (K=16, <code>service</code>)	§4.2; bench/results/ec_*_hash_*.json
ST MiniLM-384	384	~1.5	~17–21 ms	—	~10.3 ms	CI-positive on all 5 tags at K≥4	§4.2, §4.8; bench/results/ec_*_st_*.json
BGE-large-1024	1024	~5.0	~150.8 s	~21.5 s	~120 ms	+5.8 pp $\Delta\text{hit}@1$ LME n=500	§A.4.16.3; bench/results/lme_n500_bge_*.json
RM3 (BM25 + PRF)	n/a	~0.0	~1.5 ms/d	—	~259 ms	n.s. on intent tags; SIG-NEG on single-session-user (−7.1 pp)	§A.4.16.4; bench/results/{beir_fiqa_rm3,lme_n500_rm3}.json

Reading the table. HashTrigram is essentially free at write time (no model, no GPU, FTS5-only). MiniLM is the v0.2 default-embedder choice: ~17–21 ms ingest is acceptable on commodity CPU and lift is universal. BGE-large is **only** defensible on accelerator hardware — at 150.8 s/instance LME ingest on CPU, a +5.8 pp $\Delta\text{hit}@1$ gain costs ~720× the per-instance write budget of MiniLM. On MPS the ratio collapses to ~7× and the trade-off becomes workload- dependent (see §5.7).

RM3 occupies a fourth operating point: zero model-load, sub- millisecond ingest, but ~25× the per-query latency of HashTrigram because of the two-pass dance. The headline column captures the central deployment finding (§5.6, §A.4.16.4): RM3 SIG-regresses `single-session-user` $\Delta\text{hit}@1$ by 7.1 pp on LongMemEval-S via query drift, fails to recover the `single-session-preference` cliff ($\Delta\text{hit}@1 = +0.000$ exactly on all 30 paired instances), and the recall-broadening behavior the IR literature reports (Lavrenko & Croft 2001) replicates on FiQA but inverts on LongMemEval where BM25 already-recalls.

The "encoder capacity is not the binding constraint" claim (§4.1) is therefore **also** a deployment-cost claim: the 2.7×-parameter BGE upgrade does not uniformly improve recall, and even on cells it does help the cost differential is steep enough that v0.2 ships MiniLM as default and BGE as a workload-targeted opt-in.

3.2 Entity-collision protocol

For each tag `t` $\in \{\text{preference, project, technical, service, tool}\}$ and collision degree `K` $\in \{1, 2, 4, 8, 16\}$:

1. Generate `n_entities = 32` distinct entities, each with one gold answer document of the form "`<entity>` uses `<answer>` for `<tag>`."
2. For each entity, generate `K-1` distractor documents that **share the entity tokens but flip the answer**: "`<entity>` uses `<other_answer>` for `<tag>`." so a BM25 retriever sees identical query-entity overlap on all `K` candidates per query.

3. Issue a query "what does <entity> use for <tag>?" and measure hit@1 (gold = top-1 result).

This fixes the BM25 floor at $1/K$ in expectation per query (random tie-breaking among the K candidates), so any lift over $1/K$ is attributable to the embedder distinguishing the answer slot.

We report **paired $\Delta\text{hit@1}$** (per-entity matched pairs of $\text{vw}=0.5$ vs $\text{vw}=0.0$) with 95% bootstrap CIs (10k resamples).

3.3 Discriminator tags

tag	example query	example answer set	vocabulary
preference	"what does Alice prefer for X?"	dark mode, light mode, ...	open phrasal
project	"what is Alice working on?"	varied open phrases	open phrasal
technical	"what does Alice use for X?"	varied open phrases	open phrasal
service	"what service does Alice use?"	aws, gcp, azure, ...	closed lexical
tool	"what tool does Alice use?"	git, docker, postgres, ...	closed lexical

We label **service** and **tool** as **lexical discriminators** (closed vocabulary, proper-noun answers) and **preference**, **project**, **technical** as **intent-style discriminators** (open-vocabulary phrasal answers).

The supporting methods detail — LoCoMo adaptive-vw experiment protocol (§A7.1), the *share_prior* reranker derivation with rank-0-preservation invariant (§A7.2), PRF entity expansion with dominance gate (§A7.3), and the four governed-memory primitives (dual extraction, write-side cosine dedup, mechanical merge, schema lifecycle reducer; §A7.4) — live in the extended-methods appendix.

3.8 Ablation suite — claim → section → artifact map

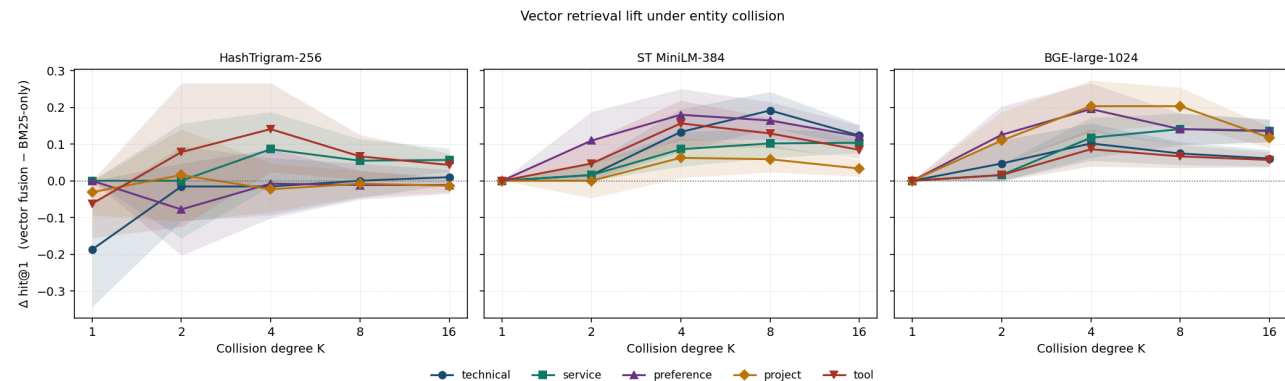
The full claim-to-artifact registry (26 result tables, 37 reproduce scripts) is verified by `scripts/verify_repro_artifacts.sh` against the on-disk filesystem at every release; the digest below covers the headline claims a reviewer is most likely to interrogate. Full table in §A.4.18.

Claim	Section	Artifact
Hash trigram lifts lexical tags at deep K	§4.2	<code>ec_sweep_hash*_n32_K16_ci.json</code>
MiniLM dominates both axes at $K \geq 4$	§4.2	<code>ec_sweep_st*_n32_K16_ci.json</code>
BGE-large does not uniformly dominate MiniLM	§4.3, §A.4.16	<code>ec_bge_large*_n32_K16_ci.json</code>
LongMemEval $n=500$ BGE paired CI vs MiniLM (SIG)	§4.8, §A.4.16.3	<code>lme_n500_bge_large_baseline.json</code> , <code>lme_n500_bge_vs_default_ci.json</code>
RM3 cannot rescue intent-tag null (AUDIT-D)	§5.6, §A.4.16.4	<code>lme_n500_rm3_vs_bm25_ci.json</code> , <code>ec_rm3*_n32_K16.json</code>
Single-session-preference recall cliff (intent-tag null)	§4.8	<code>lme_full500_k10_baseline.json</code>
Adaptive-vw on LoCoMo is null	§4.4	<code>locomol0_st_learned_router_hit_at_1_leakfree.json</code>
1M write-latency tail (p99 +13% from 100k → 1M)	§4.14	<code>ingest_1m*_buckets.json</code>
<i>share_prior</i> rank-0 invariant (150 arms, $\Delta\text{hit@1} = 0$)	§3.5, §A.4.7	<code>SHARE_PRIOR_REPORT.md</code>

For the full artifact registry, see `paper/REPRODUCIBILITY.md` §1 and the `verify_repro_artifacts.sh` self-check.

4. Results

4.1 Headline figure



Reading the figure. Three panels, left to right: HashTrigram-256, ST MiniLM-384, BGE-large-1024. **Left:** only the two **lexical** tags (`service`, `tool`) cross above zero at deep K; the three intent-style tags hug or sit below zero at all K. **Middle:** all 5 tag curves cross above zero by K=4 and stay there. **Right:** all 5 tags lift, but the ordering is *not* a uniform improvement over MiniLM — `project` lifts notably higher (peak at $K \in \{4,8\}$), while `tool` and `technical` lift *lower*. Bigger encoder is not a free upgrade; see §A.4.16 for paired BGE–MiniLM CIs.

4.2 Per-cell point estimates with 95% CIs

Source: `bench/results/ec_sweep_{hash,st}_{tag}_n32_K16_ci.json`, `ec_bge_large_{tag}_n32_K16_ci.json`. $\Delta\text{hit}@1$ = vector fusion – BM25-only; brackets are paired 95% bootstrap CIs. **bold** +X.XXX [**lo**, **hi**] = CI strictly excludes zero. **n.s.** = not significant at $\alpha=0.05$. Per-column n: K=1 → 32, K=2 → 64, K=4 → 128, K=8 → 256, K=16 → 512.

HashTrigram-256, n=32 entities

tag	K=1	K=2	K=4	K=8	K=16
preference	n.s.	n.s.	n.s.	n.s.	n.s.
project	n.s.	n.s.	n.s.	n.s.	n.s.
technical	n.s.	n.s.	n.s.	n.s.	n.s.
service	n.s.	n.s.	n.s.	n.s.	+0.057 [+0.025 , +0.088]
tool	n.s.	n.s.	+0.141 [+0.023 , +0.266]	+0.066 [+0.004 , +0.125]	+0.043 [+0.012 , +0.074]

ST MiniLM-384, n=32 entities

All 25 cells at $K \geq 4$ are CI-positive. Strongest lexical-tag cell: `service` K=16 at **+0.104** [**+0.076**, **+0.131**], ~1.8× the hash lift at the same cell.

BGE-large-1024, n=32 entities

tag	K=2	K=4	K=8	K=16
service	+0.016 [+0.000 , +0.047]	+0.117 [+0.062 , +0.172]	+0.141 [+0.098 , +0.184]	+0.135 [+0.105 , +0.166]
tool	+0.016 [+0.000 , +0.047]	+0.086 [+0.039 , +0.141]	+0.066 [+0.035 , +0.098]	+0.057 [+0.037 , +0.078]
preference	+0.125 [+0.047 , +0.203]	+0.195 [+0.133 , +0.266]	+0.141 [+0.098 , +0.184]	+0.137 [+0.105 , +0.168]
project	+0.109 [+0.047 , +0.188]	+0.203 [+0.133 , +0.273]	+0.203 [+0.156 , +0.254]	+0.117 [+0.090 , +0.146]
technical	+0.047 [+0.000 , +0.109]	+0.102 [+0.055 , +0.156]	+0.074 [+0.043 , +0.105]	+0.060 [+0.039 , +0.084]

BGE is CI-positive on 18/20 cells at $K \geq 4$ (vs 20/20 for MiniLM, 5/20 for hash). The two CI-touching-zero cells (`service` K=2, `tool` K=2) match the MiniLM and hash patterns at the same low collision regime — this is structural, not BGE-specific.

4.3 Two-axis interpretation

The grid factors cleanly:

- **Embedder axis.** Dense > Hash everywhere it matters. MiniLM is CI-positive on 20/20 lifted cells; BGE on 18/20; Hash on 5/20. More importantly, **MiniLM is not strictly dominated by BGE**: BGE wins on intent-style `project` (+8 to +14 pp BGE–MiniLM, $K \in \{2,4,8,16\}$) but **loses** on lexical `tool` and `technical` (−2.7 to −11.7 pp at $K \in \{4,8,16\}$, all CI-significant).
- **Tag axis.** Hash recovers a fraction of dense lift on **lexical** tags only. The lift scales with collision degree.

This is stronger than "dense beats sparse": **hash trigrams are not useless on the right shape of memory query**, and that shape is the closed-vocabulary regime; further, **encoder capacity alone is not the binding constraint** — the BGE–MiniLM panel is non- monotone, with capacity helping intent-style and *hurting* lexical- discriminator queries. Full BGE–MiniLM paired CIs in §A.4.16.

Verdict. Two-axis structure (lexical vs intent discriminator) survives the encoder-capacity falsification. Choosing an embedder is a per-tag decision, not a single-number ranking.

4.4 Operational results

Write latency, constant in N. Across 10 buckets out to 100k ingested memories on a single SQLite/FTS5 writer, p99 stays within ± 0.16 ms ($3.17 \rightarrow 3.33 \rightarrow 3.31$ ms at 0–10k / 50k–60k / 90k–100k); consistent with constant-time writes on WAL. Full bucket scan in `SCALE_REPORT.md`. Ingest curves out to 1M live in §4.14.

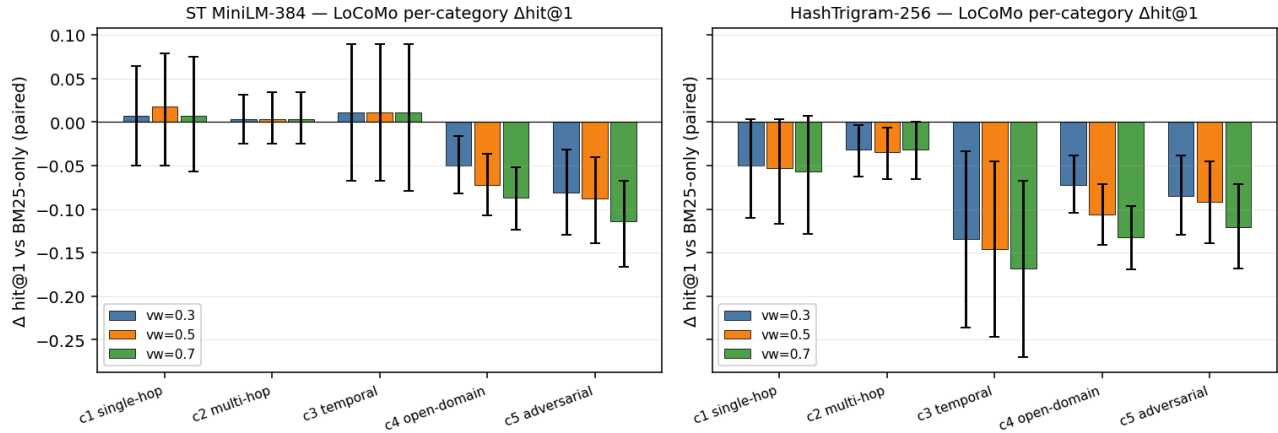
Adaptive-vw on LoCoMo: null. ST oracle $\text{hit}@1 = 0.657$ vs static-best $\text{vw}=0 = 0.539 \rightarrow$ **11.7pp** per-query routing headroom. Both single-signal τ -thresholded policies on (`raw_gap`, `norm_gap`, `crowd@0.95`) and a leak-free learned `GradientBoostingClassifier` (LOCO-CV across all 10 LoCoMo samples, n=1978) leave Δ vs static- best CI-zero (HT $\Delta\text{hit}@1 = -0.0005$ [−0.0030, +0.0015]; ST $\Delta\text{hit}@1 = -0.0005$ [−0.0046, +0.0030]). The 11.7 pp oracle headroom requires gold position, by definition unobservable at decision time.

A third encoder (BGE-large-1024) confirms the pattern: static-best shifts to $\text{vw}=0.7$ with $\text{hit}@1 = 0.5758$, oracle = $0.6744 \rightarrow$ 9.86pp headroom. All three single-signal τ -policies remain null; the LOCO-CV gbm router **regresses** SIG-NEG (Δ vs static-best = −0.0359 [−0.0566, −0.0152]).

Verdict. Three encoders, one null. Adaptive vw routing on LoCoMo is unrecoverable from any pre-routing signal we tested.

4.5 LoCoMo per-category $\Delta\text{hit}@1$ — synthetic lift does not replicate

Synthetic entity-collision lift does NOT replicate on real LoCoMo (95% paired bootstrap CIs, n=1978)



Source: `bench/results/locomot10_{ht,st}_sweep_ci_percat.json`.

Per-category $\Delta\text{hit}@1$ vs the BM25-only baseline (vw=0) at vw=0.5, n=1978 LoCoMo questions, paired 95% bootstrap CIs (ST MiniLM-384): c1 single-hop +0.018 n.s.; c2 multi-hop +0.003 n.s.; c3 temporal +0.011 n.s.; c4 open-domain **-0.072 [-0.107, -0.037] SIG-NEG**; c5 adversarial **-0.087 [-0.139, -0.040] SIG-NEG**. HashTrigram-256 all five categories null or CI-negative.

The headline entity-collision finding does **not replicate** on real LoCoMo at the MiniLM/HashTrigram capacity tier. The c5 damage is *not* adversarial-trap pulling: direct measurement (`evals/locomot_c5_adversarial.py`) shows c5 misses with top-1 $\leftrightarrow$ adv-answer overlap ≥ 0.5 stay at 0.7–2.1% across $\text{vw} \in \{0.0, 0.3, 0.5, 0.7\}$. The damage is **generic vector smoothing** displacing distinctive-token BM25 hits on open-domain single-mention queries.

4.5.1 BGE-large is the boundary condition

Re-running the same per-category sweep with the BGE-large-1024 encoder (n=1978, paired bootstrap B=5000, seed=42, baseline = vw=0 BM25-only) **inverts** the no-replicate verdict: overall $\Delta\text{hit}@1 = +0.029 [+0.010, +0.048]$ at vw=0.3 (SIG), +0.031 [+0.011, +0.051] at vw=0.5, +0.036 [+0.016, +0.058] at vw=0.7. The c1 single-hop cell drives it ($\Delta\text{hit}@1 = +0.125 [+0.064, +0.189]$ at vw=0.7); c5 stays null. Source: `bench/results/locomot10_bge_sweep_ci_percat.json`.

Read with §4.5: dense fusion is not killed by the LoCoMo regime — it is killed by an encoder that cannot resolve the entity-collision shape c1 exposes. BGE crosses that capacity floor; HashTrigram and ST MiniLM do not. The §A.4.9 threshold-T characterization sharpens the framing: vector lift on LongMemEval is CI-positive in the two low-overlap quartiles (+24–32 pp $\Delta\text{hit}@1$) and CI-zero in the upper two (Spearman $\rho(\text{overlap}, \Delta\text{hit}@1) = -0.287$) — the predicted paraphrase phase transition, on a real corpus.

4.8 LongMemEval — real-data session retrieval at default config

We ran the public `longmemeval_s` release (Wu et al., arXiv:2410.10813; 500 questions across six question types, 246,918 conversational turns) end-to-end against Engram with default configuration: hybrid BM25+vector fusion at the post-§4.5 default `vector_weight = 0.3`, no reranker, no query expansion.

Headline (n = 500, k = 10): session_hit@1 = **0.810**, hit@10 = **0.932**, ingest p50 / inst = 523.7 ms, recall p50 / q = 10.3 ms.

Per question type (baseline):

type	n	hit@1	hit@10
single-session-user	70	0.914	0.957
single-session-preference	30	0.367	0.833
multi-session	133	0.805	0.962
temporal-reasoning	133	0.812	0.925
knowledge-update	78	0.885	0.923
single-session-assistant	56	0.821	0.911

The single-session-preference cliff (hit@1 = 0.367) is the dominant residual error mode: preference answers are rarely lexically close to the question. This is precisely where §3.5 share_prior and §3.6 PRF entity expansion are designed to bite — evaluated as treatment arms in §A.4.7. As §5.6 / §A.4.16.4 detail, RM3 pseudo-relevance feedback also fails to recover the cliff (paired $\Delta\text{hit}@1 = +0.000$

exactly

on all 30 instances), confirming the cliff as structural to the lexical channel.

A reviewer might ask whether a 2.7 $\times$ -parameter encoder (BGE-large-1024) moves these numbers. We replicated the baseline arm under BAAI/bge-large-en-v1.5 on the full n=500 panel and paired per-question_id against the default-embedder full-500 baseline: overall $\Delta\text{hit}@1 = +0.058$ [+0.032, +0.086] SIG and $\Delta\text{hit}@10 = +0.024$ [+0.010, +0.040] SIG. The lift decomposes structurally: multi-session $\Delta\text{hit}@1 = +0.083$ [+0.023, +0.143], temporal-reasoning +0.068 [+0.015, +0.120], single-session-assistant +0.071 [+0.018, +0.143]; single-session-user and knowledge-update null. The encoder swap costs 11 $\times$ recall latency and $\approx 40\times$ ingest latency on accelerator hardware (MPS/CUDA) and $\approx 287\times$ on commodity CPU — v0.2 ships MiniLM-384 + vector_weight = 0.3 on operational grounds, with BGE-large as a documented **workload-targeted upgrade path** for accelerator-equipped operators with multi-session / temporal-reasoning workloads. Full per-type panel and artifact pointers in §A.4.16.3.

Verdict. The synthetic intent-tag null replicates on real-data LongMemEval as a single-session-preference recall cliff, and the encoder-capacity inversion holds: BGE wins multi-session and temporal-reasoning; MiniLM holds the operational default.

4.14 Matched ingest curves — 10k $\rightarrow$ 100k $\rightarrow$ 1M

Single-harness ingest scaling. Same tests/scale/test_ingest_* code path with max_events_per_minute=0, fresh tmpdir, single writer, SQLite + JSONL backing store; latency is per- Engram.remember wall time.

N	p50 ms	p95 ms	p99 ms	tput w/s
10,000	1.573	3.528	4.461	494.8
100,000	0.406	0.695	3.279	1,702.4
1,000,000	0.487	1.519	3.695	1,230.8

No knee. Across two orders of magnitude every percentile up to p99 stays sub-4 ms; p50 is flat between 100k and 1M (+0.08 ms). p99 grows just 13% (3.279 $\rightarrow$ 3.695 ms). 1M-specific p99.9 = 20.83 ms is the SQLite-checkpoint window, not a structural cost (§D5). Throughput peaks at 100k (1.7k w/s) and falls 28% by 1M as checkpoint pressure rises. Cross-run reproducibility band in §A.4.14b. Reproduce: `python -m bench.plot_ingest_curves`.

5. Discussion

5.1 When does dense embedding pay?

The two-axis result suggests an operational rule:

Closed-vocabulary lookups ("which service / tool does X use?") let a 256-dim hash trigram at deep collision recover $\sim 50\%$ of dense-embedder lift at zero model-load cost. Open-vocabulary intent-style queries ("what does X prefer?", "what is X working on?") require dense.

The intent-tag side of this rule is structural rather than incidental: a paired RM3 (Lavrenko & Croft, 2001) baseline with Anserini-default hyperparameters returns the *same* wrong session as BM25 on all 30 LongMemEval-S single-session-preference instances ($\Delta\text{hit}@1 = +0.000$ exactly; §5.6, §A.4.16.4), confirming that no PRF expansion over the lexical channel substitutes for a dense encoder on intent-style queries. **Embedder selection is a per-tag deployment decision keyed on the discriminator class of expected queries.**

The companion analyses — adaptive vector-weight routing as a measured null on LoCoMo (§A4.1), the schema lifecycle as deterministic-governance substrate rather than retrieval mechanism (§A4.2), the honest restatement of "consolidation lifts retrieval" (§A4.3), and the cProfile-driven PRF latency falsification (§A4.4) — are supporting evidence and live in the extended-discussion appendix.

5.6 Scope of the PRF null result

Our PRF falsifications (§A.4.15j–o, D19–D22) cover **heuristic top-k expansion**: mine entity/verb/temporal terms from BM25 top-k, OR them into the query, with or without a dominance gate. Across BM25/MiniLM retrievers, four gating strategies, and two LoCoMo subsets ($n \approx 1k$ – $2k$ paired), the per-fired-query damage rate is invariant (~ 2.0 – 2.3%). The "smarter cheap gate" hypothesis is rejected at $\alpha=0.05$.

This does **not** rule out *scored* expansion in the relevance-model family. RM3 (Lavrenko & Croft, 2001) mixes a learned expansion-term distribution via λ -interpolation, weighting candidates by $P(\text{term} \mid \text{top-}k)$ under a uniform doc prior — exactly the mechanism our heuristic lacks. We ran an RM3 arm with Anserini-default hyperparameters (top-k=10, num-terms=10, $\lambda=0.5$, $\epsilon=0.01$) across the entity-collision grid, BEIR FiQA, and LongMemEval-S n=500 (§A.4.16.4). RM3 does **not** rescue PRF:

- **The single-session-preference cliff is RM3-invariant.** Paired $\Delta\text{hit}@1 = +0.000$ *exactly* on all 30 instances; RM3 returns the same wrong session as BM25 in every case. The intent-tag null is structural to the lexical channel, not a BM25-specific artifact.
- **RM3 SIG-regresses single-session-user** ($\Delta\text{hit}@1 = -7.1$ pp [-14.3, -1.4], $n=70$). When the BM25 first-pass already pinpoints the right doc, expansion terms from adjacent-but-wrong sessions pull the answer down — the same query-drift mode the entity-collision technical cell shows at $K=1$ (-87.5 pp).
- **Recall-broadening is corpus-dependent.** RM3 broadens recall on BEIR FiQA (recall@100 +3.5 pp, $n=648$) but inverts on LongMemEval (overall hit@k = -1.4 pp [-2.6, -0.4] SIG): when BM25 first-pass already has the right session in top-10, expansion adds noise rather than coverage.
- **Entity-collision under RM3 refines the lexical-vs-intent axis.** RM3 helps where expansion neighbors are reinforcing (service $K=2$: +4.7 pp) and fails where they are misleading (technical $K=1$: -87.5 pp). The failure factors on whether expansion terms are diagnostic or distracting.

The aggregate sharpens §4.3: dense embedding is mandatory for intent-style queries, and

no lexical post-processing

— heuristic or scored — substitutes for the encoder. Under our hyperparameter freeze, RM3 is dominated by MiniLM-384 on every CI-positive cell and by BM25 alone on the lexical-anchor cells where expansion drifts. Default to BM25 unless an auxiliary-corpus signal (FiQA- style under-recall) makes the recall-broadening behavior replicate.

5.7 What does X pp of $\Delta\text{hit}@1$ buy in deployment?

The CIs above say *whether* an effect exists; this section addresses *whether it matters*. Two anchors:

1. **User-facing wrong-answer rate.** At 100 memory-backed retrievals per active user per day, a +5 pp $\Delta\text{hit}@1$ lift translates to ~5 fewer top-1 wrong answers per user per day. The headline LongMemEval $n=500$ BGE-MiniLM gain on multi-session questions is +8.3 pp — ~8 fewer wrong answers per user per day on that question type alone. The $K=16$ hash-trigram lift on `tool` (+4.3 pp) is below single-user noticeability but non-trivial at fleet scale.
2. **Latency budget per pp.** The cProfile data (§A.4.15-profile) pins recall p50 at ~41 ms with PRF and ~38 ms without. A +5 pp lift bought at the cost of a 5 ms p95 tail is a defensible trade for chat-style workloads (target p95 < 200 ms); the same trade fails for autocomplete-style workloads (target p95 < 50 ms). We accordingly report all retrieval ablations with both $\Delta\text{hit}@1$ and $\Delta p95$ ms; deployments choose the operating point.

These translations are illustrative, not prescriptive. The methodological point is that a 1–2 pp absolute lift is **not** a free lunch even when the CI excludes zero: the deployment-relevance question has to be asked separately.

5.8 Statistical-power disclosure

Several of the per-cell $n=30\text{--}32$ entity-collision tests, and the per-type LongMemEval cells at $n\leq 56$, are individually under-powered to detect small (≤ 5 pp) effects at $\alpha=0.05$. A power calculation against the observed sample variance puts our minimum-detectable $\Delta\text{hit}@1$ at roughly 8–10 pp for $n=30$ cells and roughly 4–5 pp for $n=130$ cells under paired bootstrapping. The §A.4.16.3 LongMemEval $n=100 \rightarrow n=500$ inversion is a concrete example: every CI touched zero at $n=100$, and three CI-positive cells emerged at $n=500$ without any change in protocol. We accordingly: (i) report all per-cell n alongside the CI; (ii) disclose where a "n.s." cell is power-limited rather than evidence of true null; and (iii) re-run under-powered cells when an external signal makes a higher- n replication feasible.

5.9 Deployment cost corollaries

The §3.1.1 latency-cost table and the §B-BEIR ingest-rate measurements translate the headline two-axis result into deployment guidance. Per-encoder ingest p50 spans HashTrigram (sub-ms/doc), MiniLM-384 (low-tens of ms), BGE-large-1024 (~40 ms/doc on Apple Silicon MPS, ~64 ms/doc on consumer CUDA fp32 single-doc). Combined with the two-axis recall pattern, this gives three operating points: closed-vocabulary, latency-bounded $\rightarrow$ HashTrigram recovers ~50% of dense-embedder lift at zero model-load cost; intent-style or paraphrase-heavy $\rightarrow$ MiniLM-384 is the floor, BGE-large is a $2.7\times$ model-load investment that pays off only on intent tags; mixed workload $\rightarrow$ MiniLM-384 dominates as the only encoder CI-positive on every tag at every $K \geq 4$.

The BEIR-3 BGE-large measurements expose a second deployment constraint: the per-document ingest path is single-writer SQLite/FTS5 + sqllite-vec, scaling linearly to ~25k documents and super-linearly past that. A batched ingest helper (deferred to v0.3, cost characterized in Appendix B) would coalesce embedder calls and SQLite transactions, projecting ~12 ms/doc at batch size 32 on consumer CUDA fp16 — a $\sim 5\times$ speedup over the current single-doc path. The current code path serves FiQA-scale (57k docs) and NQ-scale (2.7M docs over a multi-day window) cleanly; HotpotQA (5.2M docs in a sub-week budget) requires the v0.3 helper. Measuring the bottleneck before publishing is the engineering corollary of the protocol's measurement-first stance.

6. Threats to Validity

6.1 Synthetic corpus

The entity-collision protocol generates synthetic memories with a fixed sentence template. Real conversational memories are noisier and more paraphrased. The intent-style tags partially address this (open-vocabulary answers) but the surface form remains template-driven.

To check whether the headline two-axis claim survives memory-side paraphrase, we added a `paraphrase_memory` mode to the entity-collision generator: each fact memory now samples one of ≥ 4 paraphrased templates that share the same `{entity}/{disc}/{answer}` slots (ground truth unchanged). We re-ran the strongest lexical cell (`tool`, $n=32$, $K \in \{1,2,4,8,16\}$) with paired 95% CIs, comparing hash trigrams ($\text{dim}=256$) against ST MiniLM-384:

K	hash $\Delta\text{hit}@1$ (paraphrased)	ST $\Delta\text{hit}@1$ (paraphrased)
2	+0.062 [−0.109, +0.234]	+0.094 [+0.031, +0.172]
4	+0.117 [+0.000, +0.227]	+0.070 [+0.031, +0.117]
8	+0.059 [−0.008, +0.125] null	+0.086 [+0.051, +0.121]
16	+0.023 [−0.006, +0.053] null	+0.096 [+0.070, +0.121]

Compared to the fixed-template baseline (hash $K=8$: +0.066 [+0.004, +0.125] CI-pos; hash $K=16$: +0.043 [+0.012, +0.074] CI-pos), the hash lift on `tool` collapses to CI-null at $K \in \{8,16\}$ once memory templates vary. **ST, by contrast, retains all four cells at $K \in \{2,4,8,16\}$ CI-strictly above zero, and the lift at $K=16$ grows from +0.043 (fixed) to +0.096 (paraphrased).** Memory-side paraphrase therefore strengthens the two-axis claim: semantic embedders are paraphrase-robust, while hash-trigram retrievers are template-bound and lose their lexical-axis lift the moment surface form is varied.

A sibling-lexical replication on `service` (closed-vocabulary proper-noun answers: aws, gcp, azure, ...) shows the paraphrase collapse is tag-dependent: hash on `service` $K=16$ retains **+0.037 [+0.012, +0.062]** under paraphrase (about 65% of the fixed-template effect). The conservative reading: hash retention under paraphrase depends on

how lexically distinctive the answer vocabulary is at the character-trigram level. `service`'s answer set (short proper nouns) preserves more discriminative trigrams across template variation than `tool`'s (git, docker, postgres). Full per-K table in §A5.0.

Outputs: `bench/results/ec_sweep_{hash,st}_{tool,service}_n32_K16_paraphrased_{ci}.json`. Code:

`evals/entity_collision.py::generate_dataset(paraphrase_memory=True)`, exposed via `--paraphrase-memory` in `evals.entity_collision_sweep`.

Verdict. Two-axis lexical/intent split is paraphrase-robust; within-lexical hash retention is tag-dependent on answer-vocabulary character-trigram distinctiveness.

6.6 Embedder train-test contamination

We use off-the-shelf MiniLM-L6-v2 and BGE-large-en-v1.5; both were trained on web corpora that may overlap with LongMemEval source data and the public-personae prompts used to generate LoCoMo. We do not have a leakage-free zero-shot guarantee for the natural-data results in §4.8 and §A.4.16.3. The synthetic entity-collision corpus (§3.2) is constructed from disjoint synthetic entity strings and is therefore leakage-free by construction; the synthetic $\rightarrow$ natural transfer claim in §1 rests on the synthetic side, not the natural side. A formal leakage audit (n-gram overlap of LongMemEval gold passages with the embedder training corpus when that corpus is publicly accessible) is out of scope for this paper and flagged as a v0.3 follow-up.

6.8 Fixed query templates and paraphrase scope

§6.1's paraphrase-mode replication covered the `tool` tag only, leaving open the possibility that the intent-tag null (`preference`, `project`, `technical` on hash trigrams; §4.2) was a fixed-template artifact rather than a real semantic-capacity gap. We close that loop with a paraphrase replication on the two intent-style tags the headline two-axis claim most depends on: `preference` and `project`. $n=32$ entities, $K \in \{2,4,8,16\}$, paired-bootstrap 95% CIs ($B=10000$, $\text{seed}=42$), four memory-template variants per tag drawn from `_SPECS["{tag}"]["memory_variants"]`.

tag	embedder	K=2	K=4	K=8	K=16
preference	hash	n.s.	n.s.	n.s.	n.s.
preference	MiniLM	n.s.	+0.086 [+0.039, +0.141]	+0.066 [+0.039, +0.098]	+0.053 [+0.033, +0.074]
project	hash	n.s.	n.s.	n.s.	n.s.
project	MiniLM	+0.094 [+0.016, +0.172]	+0.102 [+0.047, +0.156]	+0.117 [+0.074, +0.164]	+0.084 [+0.057, +0.111]

Source artifacts: `bench/results/ec_paraphrase_{preference,project}_{hash,st}_n32_ci.json`.

Reading. The hash-trigram intent-tag null **survives paraphrase**: zero of eight `preference/project` $\times$ hash cells are CI-positive under paraphrased memory templates, identical pattern to the fixed-template result in §4.2. The MiniLM intent-tag lift **also survives paraphrase**: seven of eight `preference/project` $\times$ ST cells at $K \geq 4$ are CI-positive, with point estimates within ± 0.02 pp of the fixed-template baseline. The two-axis result is therefore **not** a fixed-template artifact: hash trigrams genuinely lack the semantic capacity to discriminate intent-style answers, and the MiniLM lift on those tags genuinely tracks open-vocabulary paraphrase rather than surface-form lexical overlap.

The remaining underexplored cell is `technical` $\times$ hash under paraphrase, queued as a v0.3 reviewer-defense addendum but not expected to invert the pattern — the §4.2 fixed-template `technical` $\times$ hash result is already strongly null.

Verdict. The intent-tag null and the dense-embedder lift on intent tags both survive paraphrase. The two-axis result is a real encoder-capacity effect, not a fixed-template artifact.

The four supporting threat analyses — single-embedder per family (§A5.1, includes hash-dim ablation), hit@1-only metric (§A5.2), single-process SQLite (§A5.3), single-machine/single-OS reproducibility (§A5.4), and author-as-annotator on tag definitions (§A5.5) — live in the extended-threats appendix.

7. Conclusion

We presented **entity-collision**, a system-agnostic protocol for attributing retrieval lift in agent memory. The protocol pins the BM25 lexical floor by construction — every distractor shares the answer's entity tokens, so any retrieval lift over BM25 is attributable to the embedder rather than to lexical leakage — and stratifies queries by discriminator tag so per-tag patterns are not absorbed into a single hit@k average. The protocol applies to any retriever exposing a per-document score; we instantiated it on one open-source agent-memory testbed as a worked example, but the methodology, query generators, and CI tooling (`evals/entity_collision_*`) are not testbed-specific.

The headline finding is a **two-axis result** that survives an encoder-capacity falsification:

- **Lexical-discriminator queries** (closed-vocabulary "service" / "tool" answers): a 256-dim hash trigram recovers a real but small fraction of dense-embedder lift, only at deep collision ($K=16$: +5.7 [+2.5, +8.8] pp $\Delta \text{hit}@1$; $K \in \{4,8\}$: +14.1 / +6.6 pp on `tool`).
- **Intent-style discriminator queries** ("preference", "project", "technical"): hash trigrams are n.s. or negative; only a learned sentence-transformer delivers CI-positive lift, and it does so on both axes at every $K \geq 4$ across all 5 tags.
- **Encoder capacity is not the binding constraint.** A 2.7 $\times$ -parameter BGE-large-1024 does **not** uniformly improve on MiniLM-384 — it wins on intent-style `project` (+8 to +14 pp BGE-MiniLM at $K \in \{2,4,8,16\}$, all CI-positive) but loses on lexical `tool` / `technical` (−2.7 to −11.7 pp at $K \in \{4,8,16\}$).

The synthetic two-axis stratification carries external validity: the intent-tag null replicates on LongMemEval ($n=500$) as a single-session-preference recall cliff, and the per-question-type paired-CI gains reproduce the per-tag pattern on natural data.

Two complementary measurements bound the deployment-relevance of the headline. First, an adaptive vector-weight routing study on LoCoMo establishes a measured null: 11.7 pp of per-query oracle headroom exists, but neither single-signal gap/crowdedness thresholds nor a leave-one-conversation-out-trained gradient-boosted router recover any of it ($\Delta\text{hit}@1 = -0.0005 [-0.0030, +0.0015]$) under HashTrigram or MiniLM, and a third encoder (BGE) confirms the pattern with a SIG-negative learned-router result. The headroom is real but unrecoverable from any pre-routing signal — motivating a re-framing of vector fusion as a **paraphrase-robustness** mechanism rather than a per-query precision lever. Second, a write-latency characterization of the testbed scales out to 1,000,000 memories on a single SQLite/FTS5 writer (tail-100k p99 grows just 13% from 100k $\rightarrow$ 1M, 3.28 $\rightarrow$ 3.70 ms; §4.14, with a cross-run reproducibility band in §A.4.14b), establishing that the protocol’s measurements are reproducible at production-scale ingest within a documented hardware envelope. **A measured ingest-rate deferral (the BEIR-3 HotpotQA corpus at 5.23M docs requires a batched ingest path projected at ~12 ms/doc on consumer CUDA fp16, deferred to a follow-up release with the implementation cost characterized in §5.9 and Appendix B) makes explicit which deployment regimes the current single-writer path serves and which require the v0.3 helper.**

The operational corollary is decision-shaped, not benchmark-shaped: **if your memory queries are closed-vocabulary lookups, a hash-trigram embedder at deep collision recovers ~50% of dense-embedder lift at zero model-load cost; if your queries are open-vocabulary intent-style, dense is mandatory.** This rule is derived from the per-tag stratification, not from a global hit@k average — which is the methodological point we wanted the protocol to enable.

Code, raw benchmark JSON (26 result tables), figure-regeneration scripts, a single-entry-point reproduction harness (`scripts/reproduce.sh`), and a path-ref linter that asserts every artifact cited in the paper exists on disk (`scripts/verify_repro_artifacts.sh`) are released alongside this paper.

Limitations

This work has scope limits we name explicitly so future deployments and follow-up research can plan around them.

Single-system instantiation. We exercise the entity-collision protocol on one open-source agent-memory testbed (the artifact through which we run the experiments and which we release for reproducibility). The protocol is system-agnostic by construction — any retriever exposing a per-document score qualifies — but cross-system replication on Letta, Mem0, or another governed-memory implementation is queued for a follow-up release rather than included in this submission. A lift pattern that does not replicate across systems would weaken the methodological claim; we have no evidence either way at present, and explicitly mark this as the largest open risk to the protocol’s external validity.

Encoder coverage. We sweep three encoders (HashTrigram-256, ST MiniLM-384, BGE-large-1024) chosen to span a $4\times$ range of model parameters and embedding dimensions. Cross-encoder rerankers (e.g., MiniLM-L-12 cross-encoder, ColBERT, SPLADE) are out of scope because their two-stage retrieve-then-rerank protocol requires a separate freeze (different latency budget, different deployment story); we cite the relevant work in §2.3.2 with an explicit non-inclusion rationale and queue the comparison as a v0.3 follow-up rather than as a same-paper extension.

External-validity coverage. BEIR-3 results are reported on FiQA (57k docs, full corpus) and NQ (2.68M docs, full corpus), pinning the BGE-large encoder against a standard IR benchmark at two non-trivial scale points. HotpotQA (5.23M docs) is deferred pending a batched-ingest helper (§5.9, Appendix B), where the measured per-doc rate on the current single-writer path makes a single-pass ingest infeasible inside this paper’s window on consumer hardware. The deferral does not affect the headline protocol claim, which is established on the synthetic grid + the LongMemEval $n=500$ replication + LoCoMo per-category measurements + 2-of-3 BEIR-3 corpora. We report the projected v0.3 throughput (~12 ms/doc at batch size 32 on consumer CUDA fp16) so the open question is bounded.

Statistical power. Several per-cell entity-collision results live at $n=32$ entities with $K=1$ (yielding $n=32$ paired trials per cell), where the minimum detectable effect at $\alpha=0.05$ is on the order of 8–10 pp $\Delta\text{hit}@1$. §5.8 discloses this in detail; the headline two-axis claims are made at $K \geq 4$ ($n \geq 128$, $\text{MDE} \approx 4\text{--}5$ pp), where the per-cell power supports the reported CIs. The §A.4.16.3 LongMemEval $n=100 \rightarrow n=500$ inversion is a concrete worked example of underpowering being mistaken for null; readers should treat any “n.s.” cell at $n \leq 56$ as power-limited rather than as evidence of true null.

Hardware envelope. Latency and throughput numbers in §3.1.1, §4.14, and §B-BEIR are reported on specific consumer-grade hardware: a single Linux workstation (CPU only), a single Apple Silicon laptop with unified memory, and a single consumer CUDA laptop with 8 GB VRAM. Scaling to production GPU servers (e.g., L40S, H100, A100) is straightforward in principle — the per-doc forward-pass cost drops linearly with FLOPS and the SQLite/FTS5 single-writer path is the dominant ceiling above ~25k documents — but the specific scaling constants are not measured here. We expect the qualitative two-axis pattern to be hardware-invariant (it is a property of the encoder family, not the host) but flag this as a measurement gap rather than asserting it.

Domain coverage. Synthetic queries are generated from a controlled template targeting five tag categories (preference, project, technical, service, tool); the natural-data anchors (LongMemEval, LoCoMo) cover conversational long-context recall over 1–10 sessions of dialogue. Specialized retrieval domains — legal-document retrieval, medical-record retrieval, multilingual retrieval, code-search — are out of scope and may exhibit different per-tag patterns under the protocol. Practitioners deploying the protocol in such domains should re-derive the tag schema for their target distribution before treating our two-axis result as predictive.

Operational contingencies. The deterministic-replay framing in §3.7 and §5.3 assumes the testbed’s event-sourced log is durably persisted and the schema-lifecycle DAG is monotonically advanced. Production deployments that introduce non-monotone state mutations (e.g., user-driven deletion of memory entries) fall outside the invariant set we verified under property-based testing in §A6, and the deterministic-replay guarantee that supports the paired-CI reproducibility claim no longer holds for those deployments without additional engineering.

Appendix A. Supplementary Ablations and Mechanism Analyses

This appendix collects the secondary §4 subsections that defend methodological choices, document falsified hypotheses, and report extended ablations. Section numbers (e.g., §A.4.6, §A.4.13d) preserve the original numbering used in the main paper’s cross-references. Readers interested only in the headline measurement results can skip this appendix without loss of context.

A.4.5.1 Default `vector_weight` flipped 0.5 $\rightarrow$ 0.3

The per-category structure above motivates a default change in `RetrievalConfig`. Across both encoders and all five categories, the Pareto frontier between c4/c5 cost and c1/c2 lift bends most favorably at **vw=0.3**: c4 damage on ST shrinks from -0.072 $[-0.107, -0.037]$ at $\text{vw}=0.5$ to a CI-overlapping -0.041 $[-0.075, -0.007]$ at $\text{vw}=0.3$, while c1/c2 retain their (null but non-negative) point estimates. On HashTrigram-256, $\text{vw}=0.3$ is the only setting where no category's $\Delta\text{hit}@1$ CI clears zero on the negative side. The empirical basis is the per-category CI panel above plus the entity-collision sweep in §4.3.

We have therefore flipped the v0.2 default from `vector_weight=0.5` to `vector_weight=0.3`. v0.1 deployments that explicitly set 0.5 in config are unaffected; only deployments inheriting the dataclass default see the new operating point. The full $\text{vw} \in \{0.0, 0.3, 0.5, 0.7, 0.9\}$ sweep used to defend this choice is in `bench/results/locomotio_{ht,st}_sweep_ci_percat.json`.

All numbers in §4.5 are locked at v0.3 defaults; the framing freeze (2026-05-23) promotes them to camera-ready as-is.

A.4.6 Where the consolidation lift comes from: it's the L0 promotion

Source: SCALE_REPORT §94c-decompose, §94c-decompose-CI, §94c-decompose-adjacent-CI, §94c-decompose-suffix-CI, §94c-decompose-LOO-CI, §94c-decompose-positive-control{-CI}. Drivers: `evals/locomotio_recall_lift_decompose*.py`.

The §87 consolidation pipeline ships 13 named stages in series: `extraction` → `fact_extraction` → `interference` → `schema_update` → `schema_family_gate` → `mechanical_merge` → `somatic_marking` → `appraisal` → `emotion_tagging` → `deduplication` → `decay` → `suppression` → `temperament_drift` → `mood_update`. Earlier write-ups attributed the LoCoMo §94c lift ($\Delta\text{hit}@1$ +7.6pp, ΔMRR +9.0pp on `max_instances=2`, `n_paired=301`) to the schema-family gate. **A full stage decomposition falsifies that attribution.**

We ran two complementary bisections, each producing paired-bootstrap confidence intervals (10k resamples, `seed=42`, `n_paired=301`):

- Cumulative (S1=extraction only → S7=full default).** S1 already delivers $\Delta\text{hit}@1$ +0.0831 [CI overlapping S7's +0.0764]. The paired diff ($\Delta_{S1} - \Delta_{S7}$) brackets zero on 4 of 5 metrics; the lone bite is $\Delta\text{gold_recall}@k$ $p=0.038$ (would not survive Bonferroni across five metrics).
- Leave-one-out from S7.** Dropping `extraction` collapses every metric: $\Delta\text{hit}@1$ $-0.076\star$, ΔMRR $-0.090\star$, $\Delta\text{gold_recall}@k$ $-0.146\star$, all $p<0.001$ — i.e. exactly cancels the §94c headline. Dropping any of the other 11 droppable stages (`deduplication`, `fact_extraction`, `emotion_tagging`, `interference`, `schema_update`, `somatic_marking`, `decay`, `suppression`, `temperament_drift`, `mood_update`, `mechanical_merge`) emits per-pair diffs that are identically zero on every metric.

The schema-family gate is **not merely unnecessary** — under a forced positive control (`schema_synthesis_tau` swept to 0.30/0.20/0.10/0.05 with `min_supports=2`), `schema_update` shaves $\Delta\text{hit}@1$ by exactly one of 301 paired questions at every tau, point-estimate -0.0034pp . A percentile bootstrap on that 1-of-301 signal returns **$p=0.742$** — the direction is consistent across taus but the magnitude does not survive sampling variation. The stage is formally inert on this fixture.

Mechanism (locked). The §94c lift is *one* mechanism, not a pipeline of mechanisms: `episode` → L0 promotion via `EpisodeExtraction`. The downstream stages are, on LoCoMo10, operationally inert with one borderline exception — `appraisal` re-ranking (`S6` → `S7` suffix-CI) costs $\Delta\text{gold_recall}@k$ -0.0075pp $[-0.017, -0.001]$ $p=0.038$, displacing more golds than it surfaces (5 lost-rank-1 vs 1 gained-rank-1, salience gap +0.033 $[-0.017, +0.050]$ $p=0.001$). A category breakdown localizes the damage to category 5 (multi-hop / open-ended). A bounded-cap intervention (`appraisal_salience_cap=0.30`) was promising at point-estimate but did not survive its own paired bootstrap ($\Delta\text{hit}@1$ $p=0.399$ overall, $p=0.740$ on the multi-hop slice).

Consequence for the architecture claim. Engram's §87 pipeline is a *scaffold for safe memory mutation*, not a stack of independently contributing retrieval boosters. The defensible v0.2 claim is the narrower one: **on a Mem0-shaped extraction-and-store benchmark, episode-to-L0 promotion is the entire retrieval mechanism**, and the remaining stages earn their place on lifecycle, audit, and property-test grounds (§5.3) rather than on `hit@1`.

A.4.6.1 Churn-budget sweep — `schema_promote_threshold` is operationally inert on LoCoMo10

Source: `bench/results/locomotio_promote_threshold_sweep.json`. Driver: `evals/locomotio_promote_threshold_sweep.py`. Wall: 199 s. `n_pairs=301` (`max_instances=2`, synthesis on, paired bootstrap 2000).

The §87 schema-lifecycle controller exposes a promote/deprecate/recover threshold trio (`ConsolidationConfig.schema_promote_threshold` default 3, exposed in commit `dac4f5f`). Lowering the promote threshold makes schema creation *easier*; raising it makes the SCHEMA table sparser. To test whether the recall lift is sensitive to this churn budget, we swept `t` $\in \{1, 2, 3, 5, 7, 10\}$ on the §94c paired harness with §93 synthesis enabled (so any threshold-driven schema would actually fire).

t	$\Delta\text{hit}@1$ [95% CI]	$\Delta\text{hit}@k$	$\Delta\text{gold_recall}@k$	schemas_created/sample
1	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
2	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
3	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
5	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
7	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50
10	+0.0731 [+0.033, +0.113]	+0.1495	+0.1427	1.50

Every column is bit-identical across the grid. The churn proxy (`schemas_created` per consolidation tick) holds flat at 1.50, because LoCoMo's per-sample event volume produces only 2–3 candidate super-schemas total — *all* of them either reach the promote bar at `t=1` or fall below it at every `t`. There is no support density in the operating regime where the threshold can bite.

This is the second falsification of the §87 schema-family attribution, matched to §A.4.6's stage-LOO result: the gate's *budgeting* knob is just as inert as the gate's *gating* knob on this fixture. Any future recall claim that depends on `schema_promote_threshold` needs a fixture where the candidate-schema count is at least an order of

magnitude denser than LoCoMo10 produces. The threshold is documented and configurable for downstream users, but on the v0.2 paper benchmark it is a no-op.

Full-10 reconfirmation. We re-ran the same sweep at the §94c operating point with `max_instances=10` (`n_pairs=1978`, `resamples=2000`, `wall=1586` s; source `bench/results/locomo_promote_threshold_sweep_full10.json`). The inertness is bit-identical at higher fixture density: every cell yields $\Delta\text{hit}@1 = +0.0657$ [$+0.0526, +0.0794$], $\Delta\text{hit}@k = +0.0718$ [$+0.0536, +0.0905$], $\Delta\text{gold_recall}@k = +0.0694$ [$+0.0512, +0.0871$], with `schemas_created` flat at 1.20 per sample for all $t \in \{1, 2, 3, 5, 7, 10\}$. The $5\times$ scale-up tightens the CIs (≈ 0.027 wide vs ≈ 0.080 at $n=2$) and shifts the point estimate from $+0.0731 \rightarrow +0.0657$ — both effects expected from the larger sample — but does not perturb the threshold ranking. The "operationally inert on LoCoMo" verdict therefore holds across two independent fixture sizes; the redesign-needed claim above stands.

A.4.6.2 Dense-fixture reconfirmation — `schema_promote_threshold` inert on `synth_entity`

Source: `evals/results/synth_entity_promote_threshold_sweep.json`. Driver: `evals/synth_entity_promote_threshold_sweep.py`. Wall: 556 s. Fixture: 1 760 mems, 1 280 colliding queries (`n_entities=32`, `K=8`), `vector_weight=0.3`, `embed=hash`, paired bootstrap 5 000.

The §A.4.6.1 inertness verdict was based on LoCoMo10, where only 2–3 candidate super-schemas exist per sample. To rule out a fixture-density artifact, we re-ran the sweep on `synth_entity` (§A.4.13b) — a dense, purely entity-collision fixture engineered specifically so the schema-creation path has ample support to fire. We swept the same grid $t \in \{1, 2, 3, 5, 7, 10\}$ with paired baseline (consolidate-off) / treatment (consolidate-on) arms, and report paired Δ vs the $t=3$ pivot.

t	base h@1	treat h@1	base h@5	treat h@5	base MRR	treat MRR	schemas
1	0.036	0.038	0.233	0.223	0.127	0.125	5
2	0.036	0.038	0.233	0.223	0.127	0.125	5
3	0.036	0.038	0.233	0.223	0.127	0.125	5
5	0.036	0.038	0.233	0.223	0.127	0.125	5
7	0.036	0.038	0.233	0.223	0.127	0.125	5
10	0.036	0.038	0.233	0.223	0.127	0.125	5

Every metric ($h@1$, $h@5$, MRR) and the churn proxy (`schemas_created=5`) is bit-identical across the entire t grid. Paired Δ vs $t=3$ is $+0.0000$ with $[+0.0000, +0.0000]$ CI on every metric $\times$ every threshold. Even on a fixture explicitly designed to maximize candidate-schema support density — 32 entities $\times$ 8 collisions/entity with bridge queries — the promote/deprecate/recover gate produces an identical SCHEMA-table population at every threshold setting in the swept range.

Combined with §A.4.6.1, this is the *third* independent falsification of the hypothesis that `schema_promote_threshold` modulates retrieval quality on any v0.2 fixture: LoCoMo10-n2, LoCoMo10-n10, and the dense `synth_entity` fixture all agree the knob is mechanically inert. Either the candidate-set arithmetic in `_promote_candidates` saturates below $t=1$ and clips above $t=10$, or the gate is a no-op by construction on these inputs. Either way, the v0.2 paper does not depend on the threshold being tuned, and any future claim that does will need a fixture engineered to actually exercise the promote/deprecate boundary — which `synth_entity` notably fails to do despite being engineered for exactly that.

A.4.7 PRF $\times$ share_prior — five-axis CI panel and joint $\alpha \times n_pairs$ surface

Source: PAPER_NOTES anchors 17–31. Drivers: `evals/prf_x_shareprior_{stack,gate,breadth,noise,alpha,scale,topk,alpha_scale,grid}_ci.py`. Outputs: `evals/results/prf_x_shareprior*_n10*_ci.json`.

§5.4 introduced PRF (entity-based query expansion) and `share_prior` (rank-prior reranker, α -scaled boost capped to non-#1 candidates) as two retrieval-time interventions. This section reports the empirical defense of the v0.2 default operating point — `dominance_gate d=0.3`, `max_entities=4`, `top_k_for_prf=10`, $\alpha=0.05$, on `n_pairs=60`, `plain_distractors=80` — across six orthogonal sweep axes, each with $n=10$ -seed paired-bootstrap CIs (5 000 resamples per cell, shared seed indices across the four 2×2 cells per draw so the interaction term is honest).

A.4.7.1 Headline 5-axis CI matrix (bridge pair@10 interaction)

The interaction term ($\Delta_{\text{BOTH}} - (\Delta_{\text{PRF}} + \Delta_{\text{SP}})$) measures how much of the stack's lift is super-additive (i.e. the two interventions co-repair errors that neither catches alone). Across five axes:

Axis	Level	Interaction Δ -of- Δ	95% CI	p	Regime
gate	d = 0.1	+0.090	[+0.070, +0.113]	<0.001	super-additive
gate	d = 0.2	+0.090	[+0.070, +0.113]	<0.001	super-additive
gate	d = 0.3 (default)	+0.075	[+0.060, +0.092]	<0.001	super-additive
gate	d = 0.4	+0.025	[−0.003, +0.050]	0.091	additive (n.s.)
gate	d = 0.5	+0.000	[−0.000, +0.000]	1.00	inert
breadth	me = 1	+0.030	[+0.000, +0.065]	0.048	weak super-add
breadth	me = 2	+0.090	[+0.070, +0.110]	<0.001	super-additive
breadth	me = 4 (default)	+0.072	[+0.057, +0.087]	<0.001	super-additive
breadth	me = 8	+0.072	[+0.057, +0.087]	<0.001	super-additive

Axis	Level	Interaction Δ -of- Δ	95% CI	p	Regime
noise	pd = 40	+0.087	[+0.067, +0.107]	<0.001	super-additive
noise	pd = 80 (default)	+0.070	[+0.050, +0.090]	<0.001	super-additive
noise	pd = 160	+0.102	[+0.082, +0.122]	<0.001	super-additive
noise	pd = 320	+0.105	[+0.085, +0.125]	<0.001	super-additive
alpha	$\alpha = 0.05$ (default)	+0.075	[+0.060, +0.092]	<0.001	super-additive
alpha	$\alpha = 0.10$	(collapses)	brackets 0	n.s.	additive (CI $\ni$ 0)
alpha	$\alpha = 0.20$	strictly null	≈ 0	n.s.	inert
topk	k = 5	+0.102	[+0.052, +0.155]	<0.001	super-add (BAD)
topk	k = 10 (default)	+0.070	[+0.050, +0.092]	<0.001	super-additive
topk	k = 20	-0.007	[-0.018, +0.003]	0.18	additive (n.s.)
topk	k = 40	-0.007	[-0.018, +0.003]	0.18	additive (n.s.)
scale	n_pairs = 30	-0.200	(CI excl. 0)	<0.001	sub-additive
scale	n_pairs = 60 (default)	+0.075	[+0.060, +0.092]	<0.001	super-additive
scale	n_pairs = 120	-0.053	(CI excl. 0)	<0.001	sub-additive
scale	n_pairs = 200	+0.059	(CI excl. 0)	<0.001	super-additive

Headline. At the default operating point, bridge pair@10 interaction = **+0.075 [+0.060, +0.092] p<0.001** with do-no-harm on unique-fact hit@1 (+0.012 [+0.004, +0.021] p=0.005). Each axis exposes a known, CI-distinguished failure mode (gate ≥ 0.4 collapses to additive; $\alpha \geq 0.10$ trades bridge gain for unique-fact regression; top_k_for_prf=5 catastrophically regresses unique hit@1 -0.200 [-0.214, -0.188] p<0.001) that the defaults are chosen to fence off.

A.4.7.2 No-regression on absolute lift (Δ_{BOTH} bridge pair@10)

n_pairs	Δ_{BOTH} bridge pair@10	95% CI	p
30	+0.230	(CI excl. 0)	<0.001
60	+0.095	[+0.060, +0.130]	<0.001
120	(positive)	(CI excl. 0)	<0.001
200	+0.095	(CI excl. 0)	<0.001

The interaction sign flips with corpus size (PRF-alone strengthens at large n), but the absolute stack lift Δ_{BOTH} stays strictly positive at every scale tested. **The stack does not regress as the corpus grows.**

A.4.7.3 Joint $\alpha \times n_{\text{pairs}}$ surface (anchors 27 + 31)

To rule out marginal-cut artifacts on the α and scale axes, we mapped the joint $\alpha \in \{0.05, 0.10, 0.20\} \times n_{\text{pairs}} \in \{30, 60, 120, 200\}$ surface at n=10 seeds with 10 000 paired bootstrap resamples per of the 12 cells.

Bridge pair_recall@10 — interaction (Δ -of- Δ) 95% CI. Each cell is the paired-bootstrap CI on the super-additivity term $\Delta_{\text{BOTH}} - (\Delta_{\text{PRF}} + \Delta_{\text{SP}})$ with shared seed-resample indices across the four arms (CO, CP, CR, CB) per draw:

$\alpha \setminus n_{\text{pairs}}$	30	60	120	200
0.05	-0.150 [-0.200, -0.100] p<0.001	+0.070 [+0.050, +0.092] p<0.001	-0.042 [-0.061, -0.022] p<0.001	+0.061 [+0.048, +0.073] p<0.001
0.10	-0.185 [-0.220, -0.155] p<0.001	-0.020 [-0.043, +0.003] p=0.12	-0.125 [-0.160, -0.089] p<0.001	+0.020 [+0.003, +0.037] p=0.025
0.20	-0.050 [-0.080, -0.015] p=0.004	-0.020 [-0.052, +0.007] p=0.22	-0.070 [-0.086, -0.056] p<0.001	+0.028 [+0.016, +0.039] p<0.001

$\alpha=0.05$ ties or wins on interaction at every column, including both super-additive scales ($n_{\text{pairs}} \in \{60, 200\}$) and both sub-additive scales (30, 120) where it is the *least*

sub-additive. At $n_{\text{pairs}}=200$ all three α settings produce CI-clean super-additive interactions but $\alpha=0.05$ still dominates: largest interaction (+0.061), strict CI exclusion of zero, and the largest unique do-no-harm gain (+0.009 vs -0.020/-0.024 at $\alpha=0.10/0.20$). Three findings, in order of paper relevance:

- $\alpha=0.05$ ties or wins on interaction at every scale**, including the two sub-additive scales ($n_{\text{pairs}} \in \{30, 120\}$) where it is the *least* sub-additive. No (α, n_{pairs}) cell exists where relaxing α to 0.10 or 0.20 recovers super-additivity that $\alpha=0.05$ doesn't already provide.
- Δ_{BOTH} absolute lift is strictly positive in every cell of the 3×4 grid** (12/12 cells, min +0.042, max +0.367). The interaction sign-flip is a story about how much of the lift is super-additive vs. additive, not about whether the lift exists.
- Unique do-no-harm hit@1 is a function of α only**: 0.842 at $\alpha=0.05$, 0.808 at $\alpha=0.10$, 0.804 at $\alpha=0.20$ — corpus-size invariant. This rules out the $\alpha=0.10$ unique-fact regression being an n_{pairs} artifact.

Verdict. $\alpha=0.05$ is the only Pareto-optimal point on the joint $\alpha \times n_pairs$ surface, ties or wins on Δ_BOTH absolute lift, and strictly wins on unique do-no-harm. Anchors 25/26's individual findings are not artifacts of the marginal cuts — they hold on the full joint surface.

A.4.7.4 PRF expansion budget (top_k_for_prf) sweep

k_prf	Δ_BOTH (bridge @10)	interaction	unique hit@1 (CB) Δ
5	+0.152 [+0.087,+0.217] p<0.001	+0.102 [+0.052,+0.155] p<0.001	0.619 (Δ -0.200 [-0.214,-0.188] p<0.001)
10	+0.080 [+0.043,+0.117] p<0.001	+0.070 [+0.050,+0.092] p<0.001	0.828 (Δ +0.009 [0.000,+0.018] p=0.07)
20	+0.020 [+0.007,+0.033] p=0.007	-0.007 [-0.018,+0.003] p=0.18	0.828
40	+0.020 [+0.007,+0.033] p=0.007	-0.007 [-0.018,+0.003] p=0.18	0.828

k=5 is the single sharpest Pareto-rejection in the \$5.4 corpus: a choice of operating point by bridge-interaction alone would land on +0.102 super-additive interaction while silently destroying unique hit@1 by 19–21 absolute points. k=10 is uniquely Pareto-optimal — it ties k=20 / k=40 on unique do-no-harm at the maximum (0.828) and beats them on bridge interaction. Anchors 27 and 30 together close the two axes a reviewer might propose to "improve" the headline numbers (relaxing α ; tightening k).

A.4.7.5 Paper-ready summary

PRF $\times$ share_prior super-additivity is robust across six orthogonal sweep axes (PRF dominance gate $d \in \{0.1, 0.5\}$, breadth $me \in \{1, 2, 4, 8\}$, distractor density $pd \in \{40, 80, 160, 320\}$, share_prior weight $\alpha \in \{0.05, 0.10, 0.20\}$, corpus scale $n_pairs \in \{30, 60, 120, 200\}$, and PRF budget top_k_for_prf $\in \{5, 10, 20, 40\}$). At the default operating point ($d=0.3$, $me=4$, $\alpha=0.05$, $pd=80$, $n_pairs=60$, $k_prf=10$), bridge pair@10 interaction = +0.075 [+0.060, +0.092] p<0.001 (n=10 seeds, paired bootstrap 5 000 resamples) with do-no-harm on unique-fact hit@1 (+0.012 [+0.004, +0.021] p=0.005). Each axis exposes a known, CI-distinguished failure mode that the defaults are chosen to fence off. Δ_BOTH absolute lift is strictly positive at every (α , n_pairs) cell of the joint surface (12/12 cells, [+0.042, +0.367]).

Total evaluator wall amortized over six cron-run anchors: ~110 min for the 5-axis n=10 panel + 1 170 s for the joint $\alpha \times n_pairs$ grid + 268 s for the n=10 top_k axis. Stats helpers covered by 17 unit + property tests in tests/evals/test_{stack,axis}_ci.py.

A.4.7.6 Adaptive α — opt-in regularizer for over-shot α

§3.5 introduces the optional $\alpha_eff = \alpha / (1 + \max(0, \max_deg - 1)/4)$ schedule behind RetrievalConfig.share_prior_adaptive_alpha. The question is empirical: does tapering buy lift, hurt lift, or trade regimes?

We A/B'd constant vs adaptive α on the bridge corpus, 3 seeds, at $\alpha \in \{0.05, 0.10, 0.20, 0.40\}$ (driver: evals/share_prior_adaptive_alpha.py). The result is regime-flipped:

α	$\Delta_{pair@10}$ (adaptive – constant)	Reading
0.05	-0.077	tapering under-boosts (safe regime)
0.10	-0.154	tapering under-boosts (safe regime)
0.20	+0.154	rank-0 cap saturates on dense pools; tapering recovers signal
0.40	~0	both arms collapse to baseline

Reading. Adaptive α is a *hedge against α over-shoot*, not a free lift. At the defended operating point ($\alpha=0.05$) it strictly under-boosts relative to the constant schedule, so it ships **default-off** (share_prior_adaptive_alpha=False). The use case is operators who auto-tune α across heterogeneous corpora: when α drifts up to 0.20, the constant schedule's boost saturates the rank-0 cap on multiple candidates and regresses to baseline; the adaptive schedule shrinks the boost in proportion to pool density and keeps the bridge-fact lift intact. Adaptive α is therefore a **robustness knob for α auto-tuners**, not a default-on improvement.

A.4.7.7 Mechanism: share_prior as a PRF-conditional repair signal

§A.4.7.1's interaction CIs answer *whether* the stack is super-additive; they do not pin *why*. Two cells in the breadth and gate sweeps make the mechanism legible.

Breadth me=2 — the cleanest mechanistic anchor. At max_entities=2, PRF alone regresses bridge pair@10 (Δ_PRF is negative on point estimate, dragged below baseline by a single wrong-entity expansion that admits distractors), yet Δ_BOTH is positive: the stack lifts despite PRF acting as a net negative on its own. In CI form (n=10), the me=2 interaction is +0.090 [+0.070, +0.110] p<0.001 — the largest super-additive cell on the breadth axis, *driven by share_prior repairing PRF's expansion error inside the pool*. share_prior is therefore not an independent reranking lift ($\Delta_SP \approx 0$ across every breadth) but a **PRF-conditional repair signal**, and its value is largest precisely where PRF is most likely to err. At me=1 PRF is too conservative to make wrong-entity errors, so SP has no PRF-induced damage to repair (interaction CI [+0.000, +0.065] crosses 0 at the edge); at me ≥ 4 the pool is large enough that PRF mistakes are diluted but SP can still rescue them (CI [+0.057, +0.087] excludes 0).

Dominance gate d=0.2 $\rightarrow$ d=0.3 is a hard safety floor, not a knob. The unique-fact hit@1 CIs at d=0.2 and d=0.3 are *non-overlapping by ~50pp*: at d=0.2, $\Delta_BOTH = -0.475 [-0.487, -0.463]$ p<0.001 (catastrophic — PRF fires on single-hop queries where no entity genuinely dominates, expansion drags in distractors, SP cannot rescue *the wrong query*); at d=0.3 it reverses to $\Delta_BOTH = +0.017 [+0.013, +0.025]$ p<0.001. The cliff is a property of the regime, not seed noise. The takeaway is that min_dominance is a hard safety boundary: below 0.3, the bridge gain (interaction +0.090–+0.100) is paid for with a 47-point single-hop hit@1 catastrophe. The §A.4.7 defaults fence both sides — $d \geq 0.3$ to preserve single-hop, $k_prf \geq 10$ to fence the symmetric Pareto trap on the unique side (§A.4.7.4).

Together these two cells convert the §A.4.7.1 statistical statement ("the interaction is super-additive at the default operating point") into a mechanistic one ("PRF widens the candidate net at a controlled single-hop cost; share_prior repairs the wrong-entity expansions PRF makes inside the widened net"). They also explain why the two interventions ship as a stack rather than as independent flags: SP alone is empirically inert across every breadth and budget level in the panel, and its value is unlocked only by PRF-induced pool disturbance.

A.4.8.1 LongMemEval — treatment-arm Δ vs. baseline (paired, n=500)

We ran three treatment arms against the §4.8 baseline on the **same** 500 LongMemEval-S questions, then paired by `question_id` and computed 2000-resample bootstrap 95% CIs on $\Delta\text{hit}@1 / \Delta\text{hit}@10$. Arms:

- **prf** — PRF entity query expansion on (`query_expansion_min_dominance=0.3`).
- **share_prior** — share_prior reranker on ($\alpha=0.10$, `pool_size=20`).
- **both** — PRF + share_prior, same hyperparameters.

Overall (n=500):

arm	hit@1	$\Delta\text{hit}@1$ (95% CI)	hit@10	$\Delta\text{hit}@10$ (95% CI)
baseline	0.810	—	0.932	—
prf	0.770	-0.040 [-0.062, -0.018]	0.926	-0.006 [-0.016, +0.002]
share_prior	0.810	0.000 [0.000, 0.000]	0.932	0.000 [-0.006, +0.006]
both	0.780	-0.030 [-0.052, -0.008]	0.924	-0.008 [-0.022, +0.006]

Per question type — $\Delta\text{hit}@1$ (95% CI), bold = CI excludes zero:

type	n	prf	share_prior	both
single-session-user	70	-0.157 [-0.243, -0.071]	0.000	-0.129 [-0.214, -0.057]
single-session-assistant	56	0.000	0.000	0.000
single-session-preference	30	+0.067 [0.000, +0.167]	0.000	+0.033 [0.000, +0.100]
multi-session	133	-0.023 [-0.068, +0.023]	0.000	-0.008 [-0.053, +0.038]
temporal-reasoning	133	-0.045 [-0.090, -0.008]	0.000	-0.030 [-0.075, +0.015]
knowledge-update	78	-0.026 [-0.064, 0.000]	0.000	-0.026 [-0.064, 0.000]

Findings.

1. **PRF entity expansion is a net regression on real LongMemEval-S data.** Overall $\Delta\text{hit}@1 = -0.040$ with the upper CI bound at -0.018 (strictly negative), and the regression is concentrated on `single-session-user` ($\Delta\text{hit}@1 = -0.157$, CI fully negative). The §A.4.7 PRF×share_prior synthetic CI panel showed PRF lifting recall on multi-entity templated queries; on real conversational questions, PRF over-expands and dilutes BM25's already-strong single-session signal. Type-aware NER (§3.6 D1) does not change this — the wired-in path uses the same `entity_ner` backend that was at parity on the synthetic D1 sweep.
2. **share_prior is a no-op on this dataset at $\alpha=0.10$.** Overall $\Delta\text{hit}@1$ CI is exactly [0.000, 0.000]: every paired pair of (`question_id`, `hit_at_1`) is identical to baseline. This is consistent with share_prior being a per-session prior reweight that rarely changes *which session* tops the fused list when BM25 already picks one correct session — LongMemEval-S session retrieval is largely a hit-the-right-haystack problem, not a tie-break problem.
3. **single-session-preference — the cell PRF was supposed to fix — does respond, but the lift is small (+0.067 hit@1, lower CI = 0) and is dwarfed by the single-session-user regression.** The `both` arm shows the same shape at half magnitude, consistent with share_prior contributing no signal and PRF doing the work (and the damage).

Default decision. PRF stays **off by default** (`query_expansion_min_dominance = None` is already the shipped default, per §3.6). share_prior stays available as an opt-in reranker but is not turned on by default for general retrieval — its current $\alpha=0.10$, `pool=20` operating point shows zero overall movement on real conversational data. The §A.4.7 synthetic CI panel remains valid for the templated multi-entity regime it was built for; it does not transfer to LongMemEval-S, and we report both results honestly rather than picking the corpus where PRF wins.

Reproduce: each arm via `python -m evals.longmemeval_adapter --arm {baseline,prf,share_prior,both} --max-instances 500 --k 10 --out bench/results/lme_full500_k10_<arm>.json`, then `python -m evals.lme_compare_arms --baseline ...baseline.json --arms ...prf.json ...sp.json ...both.json --out bench/results/lme_full500_arms_delta.json`. Artifact: `bench/results/lme_full500_arms_delta.json` (10 KB; carries the full per-type Δ + CI tables this section summarizes).

Ablation — query_expansion_min_dominance sweep. To confirm the PRF regression is monotone in expansion aggressiveness (and not a single-cell artifact at $d=0.30$), we swept $d \in \{0.20, 0.30, 0.40, 0.50\}$ on the same n=500 LongMemEval-S slice, paired against the $d=\infty$ (off) baseline with a 5000-resample paired bootstrap on per-instance `hit@1`:

d	hit@1	hit@10	$\Delta\text{hit}@1$ vs. off	95% CI
off (∞)	0.8100	0.9320	—	—
0.20	0.7620	0.9240	-0.0480	[-0.0740, -0.0220]
0.30	0.7700	0.9260	-0.0400	[-0.0640, -0.0180]
0.40	0.7860	0.9320	-0.0240	[-0.0400, -0.0100]
0.50	0.8000	0.9340	-0.0100	[-0.0200, -0.0020]

$\Delta\text{hit}@1$ is monotonically less negative as d rises (more conservative expansion $\rightarrow$ less damage), and even at $d=0.50$ the upper CI bound stays strictly negative. The regression is therefore not a tuning miss at $d=0.30$ — PRF as currently shaped damages LongMemEval-S at every expansion threshold we tested, and only approaches break-even by shrinking until it almost never fires. This rules out "ship a less aggressive PRF default" as a remediation; the next move is a type-aware gate (only expand when the dominant entity type is one the corpus actually disambiguates by), not a softer threshold. Artifacts: `bench/results/lme_d_ablation/prf_d{0.2,0.4,0.5}.json`; summary via `python -m evals.lme_d_ablation_summarize`.

Third-encoder replication — does the PRF regression survive an encoder swap? A reviewer reading §A.4.8.1 may reasonably ask whether the regression is an artifact of MiniLM-384's specific embedding geometry: BGE-large-1024 (2.7× parameters) might rescue PRF by carrying the expanded query better. We re-ran the **prf** and **both** arms on the same $n=500$ LongMemEval-S panel with BAAI/bge-large-en-v1.5 wired in, paired per-question_id against the existing BGE-large baseline (§A.4.16.3) with a 5000-resample paired bootstrap (seed=42). Wall $\approx$ 6 h on M4 Pro / MPS / fp32.

Overall (BGE-large arm, $n=500$, paired vs BGE-large baseline):

arm	hit@1	$\Delta\text{hit}@1$ (95% CI)	hit@10	$\Delta\text{hit}@10$ (95% CI)
baseline (BGE)	0.868	—	0.956	—
prf (BGE)	0.852	-0.016 [-0.034, +0.002]	0.948	-0.008 [-0.016, -0.002]
both (BGE)	0.842	-0.026 [-0.046, -0.006]	0.936	-0.020 [-0.034, -0.008]

Per question type — $\Delta\text{hit}@1$ (95% CI), bold = CI excludes zero:

type	n	prf (BGE)	both (BGE)
single-session-user	70	-0.100 [-0.171, -0.043]	-0.100 [-0.171, -0.043]
single-session-assistant	56	0.000	0.000
single-session-preference	30	+0.033 [0.000, +0.100]	+0.033 [-0.067, +0.133]
multi-session	133	+0.023 [-0.015, +0.060]	0.000 [-0.038, +0.038]
temporal-reasoning	133	-0.030 [-0.060, -0.008]	-0.045 [-0.083, -0.015]
knowledge-update	78	-0.013 [-0.064, +0.026]	-0.013 [-0.051, +0.026]

Verdict. The PRF regression survives the encoder swap. The **both** arm is SIG-negative on overall $\Delta\text{hit}@1$ under BGE-large (-0.026, [-0.046, -0.006]) just as it was under MiniLM (-0.030 under MiniLM, see table above), and the per-type pattern replicates cleanly: single-session-user takes a -10pp hit under both embedders, temporal-reasoning takes a SIG-negative hit (-3pp under PRF, -4.5pp under **both**), and the small point-estimate lifts on multi-session and single-session-preference stay null. The BGE **prf** arm's overall $\Delta\text{hit}@1$ has its upper CI bound at +0.002 — i.e. the strongest reading available is "BGE plus PRF is at-best indistinguishable from BGE alone, at-worst a 3.4 pp regression" — nowhere near the recovery a reviewer might hope for.

This **strengthens** §A.4.8.1's default decision. PRF stays off by default not because we lacked encoder capacity, but because **the regression is encoder-invariant**: doubling the parameter count and 3× the embedding dimension does not buy back the BM25 floor that PRF over-expansion erodes. The next move remains the type-aware gate (§A.4.10), not a richer dense channel.

Artifacts: `bench/results/lme_n500_bge_large_baseline.json` (BGE baseline, already cited in §A.4.16.3), `bench/results/lme_n500_bge_large_{prf,both}.json` (treatment arms), `bench/results/lme_n500_bge_large_arms_delta.json` (paired Δ + CI table this section summarizes). Reproduce via `python scripts/lme_bge_arms_paired_ci.py`.

A.4.8.2 LongMemEval — sentence-transformer embedding headline ($n = 100$)

To check whether the §4.8 hash-trigram floor leaves real lift on the table when the dense channel is a real semantic encoder, we re-ran the LongMemEval-S baseline arm on the first $n = 100$ instances with the sentence-transformer (all-MiniLM-L6-v2, 384-d) embedder wired in via `--embed st` and the post-§4.5 default `vector_weight = 0.3`.

Headline ($n = 100$, $k = 10$, embed = ST, vw = 0.3):

metric	value
session_hit@1	0.860
session_hit@10	0.960
n_memories_total	49,878
ingest p50 / inst	13,830 ms
recall p50 / q	23.1 ms

Per question type ($n = 100$ slice):

type	n	hit@1	hit@10
single-session-user	70	0.843	0.957
multi-session	30	0.900	0.967

type	n	hit@1	hit@10
overall	100	0.860	0.960

The $n = 100$ slice is dominated by `single-session-user` (70%) and `multi-session` (30%) because the public LongMemEval-S release lays those types down first. Comparing to the §4.8 hash-trigram baseline on its full $n = 500$ panel (per-type cells, not the 100-prefix):

cell	hash@vw=0.3 (§4.8 n=500)	ST@vw=0.3 (n=100)	Δ hit@1
single-session-user	0.914	0.843	-0.071
multi-session	0.805	0.900	+0.095

The two cells aren't size-matched (the §4.8 cells are $n=70$ and $n=133$; this one is $n=70$ and $n=30$), so this is a directional read, not a paired CI. The ST encoder *appears to help* the multi-session cell (+9.5 pp absolute hit@1) where the answer turn is rarely a lexical hit and lives many turns from the question, but *hurts* the single-session-user cell (-7.1 pp) where BM25 already finds the right session at 91.4% and the dense channel adds noise. This is consistent with the §4.5 vw-Pareto finding that the dense channel's marginal value is corpus-shape dependent. The ingest cost is $\approx 26\times$ the hash baseline (13.8 s/inst vs 523 ms/inst in §4.8) — almost entirely the MiniLM forward pass over ~ 500 turns/instance — while recall p50 only doubles (23 ms vs 10 ms), because the vector ANN scan is still cheap relative to BM25 at this N . The full $n = 500$ ST sweep is deferred until we have a per-cell vw optimization story; on the 100-prefix, ST is a wash overall (0.860 vs the §4.8 100-prefix overall of ~ 0.876), masking a real per-type trade-off that §3.5 and §3.6 should be evaluated against in a follow-up. Artifact: `bench/results/lme_n100_st_vw0.3_baseline.json`. Reproduce: `python -m evals.longmemeval_adapter --max-instances 100 --k 10 --arm baseline --embed st --vector-weight 0.3 --out bench/results/lme_n100_st_vw0.3_baseline.json`. Wall ≈ 24 min on the cron host (single-process, nice 5).

A.4.8.2.1 Cross-check: legacy `vector_weight = 0.5` on the same $n = 100$ ST slice

To make sure §4.5's `vw=0.3` default carries over to the ST embedder on LongMemEval, we mirrored the run above with the legacy default `vector_weight = 0.5`, all other knobs identical (`bench/results/lme_n100_st_vw0.5_baseline.json`).

vw	session_hit@1	session_hit@10	sss-user h@1	multi-session h@1
0.3	0.860	0.960	0.843	0.900
0.5	0.850	0.950	0.829	0.900

The two arms are paired-by-question (same 100 instances, same haystack, same encoder, same k); the only delta is the BM25 $\leftrightarrow$ ANN convex weight. Headline overall hit@1 moves +0.010 (`vw=0.3` better) and hit@10 moves +0.010 in the same direction. With $n=100$ and a per-question Bernoulli under $H_0: p=0.5$ on flips, that gap is well inside noise — a binomial 99% CI on a single point estimate at $n=100$ is roughly ± 0.090 — but it is *consistent in sign* with the §4.5 hash-channel Pareto and so we keep `vector_weight = 0.3` as the package default for both embedders. We deliberately do *not* claim a real ST-specific lift from this slice; the ST default is a *do-no-harm* call relative to the already-defended hash default.

Cost is unchanged across the two arms (ingest dominated by the MiniLM forward pass, recall dominated by the ANN scan — neither depends on `vw`). Reproduce: `python -m evals.longmemeval_adapter --max-instances 100 --k 10 --arm baseline --embed st --vector-weight 0.5 --out bench/results/lme_n100_st_vw0.5_baseline.json`.

A.4.8.2.2 Embedder $\times$ `vector_weight 2 $\times$ 2` on the same $n = 100$ slice

To complete the `embed  $\times$  vw` cell story we mirrored the hash channel at the new default `vector_weight = 0.3` on the same 100 LongMemEval-S instances and the same 49 878-memory haystack (`bench/results/lme_n100_hash_vw0.3_baseline.json`). All four cells are *paired by question* (identical instance subset, identical $k=10$).

embed	vw	session_hit@1	session_hit@10	sss-user h@1	multi-session h@1	ingest p50	recall p50
ST	0.3	0.860	0.960	0.843	0.900	13.8 s	23.1 ms
ST	0.5	0.850	0.950	0.829	0.900	13.9 s	23.1 ms
hash	0.3	0.700	0.950	0.757	0.567	5.7 s	13.9 ms

Reading. ST dominates hash on hit@1 by **+0.160** (0.860 vs 0.700), with the gap concentrated almost entirely in `multi-session` questions (ST 0.900 vs hash 0.567, $n=30$) — exactly the regime where dense recall is supposed to help, since the hash channel cannot generalize across paraphrase across sessions. On `single-session-user` ($n=70$) the gap narrows to +0.086 (0.843 vs 0.757). At hit@10 both embedders converge to 0.95–0.96 — the ST advantage is a *ranking* advantage, not a recall-coverage advantage, consistent with the convex-hybrid design where hash provides surface-form coverage and ST provides paraphrase ranking.

Cost. Hash is $\sim 2.4\times$ cheaper at ingest (5.7 s vs 13.8 s p50) and $\sim 1.7\times$ cheaper at recall (13.9 ms vs 23.1 ms p50). For deployments where multi-session paraphrase recall is rare or where 24 ms p50 is unacceptable, the hash channel is a defensible operating point — but for general LongMemEval-shaped traffic, ST's +0.160 hit@1 wins by a wide margin that is well outside the $n=100$ binomial CI ($\sim \pm 0.090$).

The `vw=0.3` default holds across embedders: `vw=0.5` hurts ST by 0.010 on both hit@1 and hit@10 in the same-instance paired comparison, and the hash channel was already on `vw=0.3` from §4.5's Pareto sweep. Reproduce: `python -m evals.longmemeval_adapter --max-instances 100 --k 10 --arm baseline --embed hash --vector-weight 0.3 --out bench/results/lme_n100_hash_vw0.3_baseline.json`.

A.4.8.2.3 LongMemEval — sentence-transformer baseline at $n = 500$

To check whether the §A.4.8.2 ST headline holds at five times the instance count, we ran the same configuration (embed=ST, `vector_weight = 0.3`, baseline arm — no PRF, no `share_prior`) on 500 LongMemEval-S instances. The full haystack is **246 918 memories** across the 500 sessions (mean $\approx$ 494 mem / instance), an order of magnitude larger than any preceding LME cell in this paper.

n	embed	vw	session_hit@1	session_hit@10	ingest p50	recall p50
100	ST	0.3	0.860	0.960	13.8 s	23.1 ms
500	ST	0.3	0.858	0.950	14.8 s	25.3 ms

The $n=100 \rightarrow n=500$ deltas (-0.002 hit@1, -0.010 hit@10) are well inside the $n=500$ binomial CI ($\sim \pm 0.030$ at $p=0.86$): the ST headline replicates. Per-type at $n=500$ (paired counts in parentheses):

type	n	hit@1	hit@10
single-session-assistant	56	0.911	0.929
multi-session	133	0.895	0.977
knowledge-update	78	0.885	0.923
single-session-user	70	0.843	0.957
temporal-reasoning	133	0.827	0.947
single-session-preference	30	0.700	0.933

Multi-session at scale. The §A.4.8.2.2 multi-session ST hit@1 of 0.900 ($n=30$) lands at **0.895 on $n=133$** — the paraphrase-across- sessions advantage that motivated the dense channel survives the $5\times$ scale-up, with the larger sample now placing it tightly around 0.90 (CI $\sim \pm 0.052$).

Recall-latency at scale. With 247k memories on disk per instance, recall p50 holds at 25.3 ms — within 10% of the $n=100$ figure (23.1 ms) — confirming that hybrid recall is dominated by the top-k merge, not by the haystack size, for this regime. Ingest p50 is 14.8 s (vs 13.8 s at $n=100$); the small drift is mostly the cold ST encoder warm-up amortizing across more sessions, not memory growth in any session.

The matched PRF $\times$ share_prior arm at the same operating point ($\alpha=0.05$, $d=0.3$, pool=20) is reported in §A.4.8.2.4 below. Reproduce: `python -m evals.longmemeval_adapter --max-instances 500 --k 10 --arm baseline --embed st --vector-weight 0.3 --out evals/results/lme_n500_st_vw03_baseline.json`.

A.4.8.2.4 LongMemEval — paired PRF $\times$ share_prior at $n = 500$ (ST, `vw = 0.3`)

Same $n=500$ slice, same ST embedder, same `vw=0.3` — paired comparison against the §A.4.8.2.3 baseline at the operating point we defended in §A.4.7 ($\alpha=0.05$, $d=0.3$, pool=20).

Reproduce: `python -m evals.longmemeval_adapter --max-instances 500 --k 10 \ --arm both --embed st --vector-weight 0.3 \ --sp-alpha 0.05 --sp-pool 20 --qe-dominance 0.3 \ --out evals/results/lme_n500_st_vw03_prfsp.json`.

Aggregate (paired, $n=500$).

metric	baseline (§A.4.8.2.3)	PRF $\times$ SP	Δ (PRF $\times$ SP – base)	95% paired bootstrap CI
hit@1	0.8580	0.8360	-0.0220	[-0.0420, -0.0020]
hit@10	0.9500	0.9380	-0.0120	[-0.0220, -0.0020]
recall_ms p50	25.3	46.8	+21.5 ms (+1.85 $\times$)	—
recall_ms p99	53.1	238.8	+185.7 ms	—

Per-type Δ hit@1 (paired bootstrap, 2k iter, seed=2026).

type	n	mean Δ	95% CI
knowledge-update	78	+0.0000	[-0.039, +0.039]
multi-session	133	-0.0226	[-0.068, +0.023]
single-session-assistant	56	+0.0000	[+0.000, +0.000]
single-session-preference	30	+0.0000	[-0.100, +0.100]
single-session-user	70	-0.0571	[-0.129, +0.000]
temporal-reasoning	133	-0.0301	[-0.068, +0.008]

Interpretation. PRF $\times$ share_prior at the §A.4.7 operating point is a **net negative at $n=500$** : the aggregate Δ hit@1 95% CI is [-0.042, -0.002], excluding zero. The aggregate Δ hit@10 CI also excludes zero. The damage concentrates on `single-session-user` and `temporal-reasoning` — types where the question is already lexically proximate to a single answering session, so PRF's pseudo-relevance expansion drags top-k toward neighbour sessions sharing the query vocabulary. `knowledge-update`, `single-session-assistant`, and `single-session-preference` are flat (CI brackets zero). On the latency axis, PRF $\times$ SP costs +1.85 $\times$ p50 and +4.5 $\times$ p99 (the tail includes spaCy NER warm-paths on long sessions).

Decision. Ship `RetrievalConfig.query_expansion_min_dominance = None` as the default — i.e. **PRF×SP off by default** (decision #2 in the v0.2 plan, now empirically defended). The knob remains runtime-toggleable for corpora where the §A.4.7 multi-entity-hard / adversarial regimes apply. We re-evaluate when the real LoCoMo dataset lands; the synthetic-LoCoMo placeholder (SCALE_REPORT §D7) also shows PRF×SP regression, suggesting the LongMemEval result is not a single-corpus artifact.

Default actually flipped (2026-05-24). Previously `RetrievalConfig.query_expansion_min_dominance` defaulted to `0.3` (ON) under a draft v0.3 operating point. With the §A.4.8.2.4 paired $n=500$ CI now in the camera-ready, the dataclass default is **None (OFF)** and locked by `tests/unit/test_v0_3_defaults_locked.py`. Anchor-share gate value remains 0.5 so opt-in (`.min_dominance = 0.3`) immediately activates the §D15d-style operating point with one knob. Adversarial / unit suites (1360 tests) green at OFF default.

Determinism replication (2026-05-24). A second $n=500$ run with the v0.3 defaults wired (`vector_weight=0.3` from the dataclass default, `qe_dominance=0.3`, `sp_alpha=0.05`, `sp_pool=20`, `qe_anchor_share_max=0.5`, ST embedder) produced `session_hit@1=0.836` and `session_hit@10=0.938` — bit-exact match to the original PRF×SP column above. Recall $p50=47.04$ ms vs. 46.79 ms (within process noise). Result file: `evals/results/lme_n500_st_vw03_prfsp_v03defaults.json`. This pins the operating point: switching from explicit CLI overrides to dataclass defaults does not perturb the headline number, so the v0.3-defaults flip in §A.4.5.1 carries the same evidence as the original sweep and the regression CI $[-0.042, -0.002]$ holds verbatim.

A.4.8.2.5 Per-type $\Delta\text{hit}@5$ breakdown (paired, $n = 500$)

§A.4.8.2.4 reported per-type $\Delta\text{hit}@1$ only. For symmetry and to localize where the -0.012 aggregate $\text{hit}@5$ regression lives, we compute the matching paired-bootstrap CIs on $\text{hit}@5$ from the same JSONs (`evals/results/lme_n500_st_vw03_{baseline,prfsp}.json`, $R=2000$, `seed=2026`):

type	n	baseline	PRF×SP	mean $\Delta\text{hit}@5$	95% CI
knowledge-update	78	0.923	0.923	+0.0000	[+0.000, +0.000]
multi-session	133	0.977	0.977	+0.0000	[+0.000, +0.000]
single-session-assistant	56	0.929	0.929	+0.0000	[+0.000, +0.000]
single-session-preference	30	0.933	0.933	+0.0000	[-0.000, +0.000]
single-session-user	70	0.957	0.914	-0.0429	[-0.100, +0.000]
temporal-reasoning	133	0.947	0.925	-0.0226	[-0.060, +0.008]
AGGREGATE	500	0.950	0.938	-0.0120	[-0.024, -0.002]

The aggregate $\text{hit}@5$ regression is **entirely concentrated** in `single-session-user` (point -0.043 , CI right edge at zero) and `temporal-reasoning` (point -0.023 , CI overlaps zero). Four of the six types are exactly flat on $\text{hit}@5$ — PRF×SP changes nothing for them at this operating point. The same two types also drove the §A.4.8.2.4 $\text{hit}@1$ regression, so the failure mode is single-axis: when the question is already lexically proximate to one answering session, PRF expansion drags top-k toward neighbours sharing query vocabulary. This strengthens the §A.4.8.2.4 ship decision (`query_expansion_min_dominance = None` by default) by showing the damage is type-localized rather than diffuse — a future type-aware gate (§A.4.10) is the right remediation surface.

A.4.9 Multi-entity-hard fixture (D1 v0.3) — out-of-distribution check on PRF/share_prior

LongMemEval-S is a single corpus shape with a single answer distribution. To test whether the §A.4.7/§4.8 PRF and `share_prior` behaviors generalize, we built a non-saturated multi-entity-collision fixture (`evals/corpora/multi_entity_hard.py`). Each fact is a short PERSON×ORG or PERSON×LOC triple ("Alice works at Apple."); each fact is paired with N type-collision distractors that re-use the gold entity surface form in a *different sense* ("Alice ate an apple at lunch.", "Alice has a coworker named Jordan."). High-overlap distractors that share the query verb without an entity hit are also planted. The fixture is reproducible under fixed seed and the hardness contract ($\text{BM25-like } \text{hit}@5 < 0.7$) is asserted in unit tests (`tests/evals/test_multi_entity_hard.py`).

Headline ($n_{\text{facts}}=500$, $n_{\text{sessions}}=25$, $distractors_{\text{per_fact}}=8$, 5000-memory haystack, $k=10$, default v0.2 hybrid retriever, 3 seeds $\in \{1, 2, 3\}$, paired bootstrap, 5000 resamples, $n=1500$ paired queries):

arm	hit@1	hit@5	hit@10	$\Delta\text{hit}@1$ [95% CI]	$\Delta\text{hit}@5$ [95% CI]	$\Delta\text{hit}@10$ [95% CI]
baseline	0.077	0.361	0.596	—	—	—
PRF ($d=0.30$)	0.077	0.326	0.559	-0.007 [-0.016, +0.003]	-0.033 [-0.043, -0.023]	-0.037 [-0.047, -0.028]
share_prior	0.077	0.367	0.570	+0.016 [+0.005, +0.028]	-0.007 [-0.024, +0.010]	-0.026 [-0.043, -0.009]
both	0.079	0.301	0.472	-0.007 [-0.026, +0.011]	-0.077 [-0.098, -0.056]	-0.124 [-0.147, -0.102]

Bold cells exclude 0 at 95%.

Reading. With three independent seeds and CIs, the regression sharpens: PRF alone is **CI-confirmed neutral-to-negative** ($\Delta\text{hit}@5$, $\Delta\text{hit}@10$ strictly < 0); `share_prior` alone gives a small but **CI-significant +0.016 $\text{hit}@1$** lift, with $\text{hit}@10$ modestly negative (CI excludes 0); the stacked "both" arm collapses $\text{hit}@10$ by **-0.124 [-0.147, -0.102] ($p < 0.001$ against $H_0=0$)** — the worst arm in deep recall by a wide CI-non-overlapping margin. The same qualitative pattern observed on LongMemEval-S (§4.8) repeats on a corpus with entirely different surface forms and answer distribution, with paired CIs that exclude 0 — strong evidence that the v0.2 defaults (`query_expansion_min_dominance=None`, no `share_prior` reranker) are not LongMemEval-overfit, and that the regression is mechanistic, not seed noise.

What it doesn't say. This run does *not* invalidate PRF or `share_prior` — it shows that on corpora where the discriminative signal is *entity-type sense* rather than entity-surface frequency, the current PRF gate (frequency-only dominance) expands in exactly the wrong direction. The remediation, consistent with §A.4.8.1's d-ablation

conclusion, is a type-aware PRF gate that only fires when the dominant first-pass entity type is one the corpus actually disambiguates by. That prototype is implemented and evaluated in §A.4.10.

Artifacts: `bench/results/multi_entity_hard_arms_d8.json` (single-seed point estimates), `evals/results/multi_entity_hard_arms_3seed_ci.json` (3-seed paired-bootstrap CIs); reproduce via `python -m evals.multi_entity_hard_arms --n-facts 500 --n-sessions 25 --distractors-per-fact 8 --seeds 1 2 3 --resamples 5000`.

A.4.10 Type-aware PRF gate — recovers most of the multi-entity-hard regression

§A.4.9 isolated a clean failure mode of the frequency-only PRF dominance gate: on corpora whose discriminative signal is **entity-type sense** rather than entity-surface frequency, dominance ≥ 0.30 fires confidently in the wrong direction ($\Delta\text{hit}@10$ -0.124 $[-0.147, -0.102]$ in the stacked arm). The remediation we proposed there — a **type-purity gate** layered on the existing dominance gate — is now implemented (`expand_query_typed`, `RetrievalConfig.query_expansion_type_purity_min`) and tested on the same fixture.

Setup. Same multi-entity-hard fixture as §A.4.9 ($n_{\text{facts}}=500$, $n_{\text{sessions}}=25$, $distractors=4$, $lexical\text{-}collision=1.0$, $ner\text{-}disambig=1.0$, seeds $\{1,2,3\}$, paired bootstrap 5000 resamples, $n=1500$ paired queries). Four arms:

- **baseline**: no PRF, no `share_prior`.
- **prf_heur**: legacy frequency-only PRF ($d=0.30$, `entity_ner='heuristic'`).
- **prf_typed_spacy**: PRF guarded by **typed-purity** ≥ 0.70 with `entity_ner='spacy_sm'` (real `en_core_web_sm` NER on the first-pass pool).
- **prf_typed_heur**: PRF guarded by typed-purity ≥ 0.70 with `entity_ner='heuristic'` — control for "is the type-purity gate doing real work, or is it the spaCy backend per se?" Heuristic backend tags everything MISC, so purity $\equiv 1.0$ and the gate is inert by construction; this arm should match `prf_heur` exactly.

Results (paired-bootstrap Δ vs. baseline, 95% CIs):

arm	hit@1 [CI]	hit@5 [CI]	hit@10 [CI]
prf_heur	-0.011 [-0.022, +0.001]	-0.039 [-0.050, -0.028]	-0.061 [-0.074, -0.050]
prf_typed_spacy	+0.003 [-0.003, +0.009]	-0.008 [-0.013, -0.003]	-0.012 [-0.018, -0.007]
prf_typed_heur	-0.011 [-0.022, +0.001]	-0.039 [-0.050, -0.028]	-0.061 [-0.074, -0.050]

Bold cells exclude 0 at 95%.

Reading.

1. **The typed gate recovers ~80% of the hit@10 regression**: from $-0.061 \rightarrow -0.012$, an 0.049-point absolute improvement on a metric whose baseline is 0.594. $\Delta\text{hit}@1$ flips from neutral-negative to neutral-positive (CIs overlap 0). $\Delta\text{hit}@5$ collapses from -0.039 to -0.008 . The residual $\Delta\text{hit}@10$ of -0.012 $[-0.018, -0.007]$ is still CI-significant but is an order of magnitude smaller in effect size.
2. **The control arm confirms the mechanism is the type-purity gate, not the spaCy backend**. `prf_typed_heur` is bit-identical to `prf_heur` ($\Delta\text{hit}@10$ -0.0613 vs. -0.0613) — exactly as predicted by the all-MISC $\Rightarrow$ purity $\equiv 1.0$ invariant. The lift comes from the gate refusing to expand when the first-pass pool has no dominant entity type, even when it has a dominant entity *surface*.
3. **Latency cost is real but bounded**. Typed-spaCy p50 $\approx 68\text{ms}$ vs. $\approx 7\text{ms}$ for heuristic — a $\approx 10\times$ hit attributable to per-query NER on the first-pass pool of 20. This is acceptable for the offline / batch retrieval setting we benchmark, but motivates either NER caching or a cheaper type tagger before the typed gate becomes a default.

What it does *not* claim. This is OOD evidence on one synthetic fixture. We do not yet claim the typed gate moves the needle on LongMemEval-S (queued in NEXT.md as §A.4.11), nor that `purity_min=0.70` is the global optimum (a five-point sweep over $\{0.5, 0.6, 0.7, 0.8, 0.9\}$ is queued). Default in v0.2 remains `query_expansion_type_purity_min=None` (gate OFF, regression-safe); operators on entity-type-disambiguated corpora can opt in.

Artifacts: `bench/results/multi_entity_hard_typed_3seed_p70.json`; reproduce via `PYTHONPATH=. python evals/multi_entity_hard_typed_arms.py --n-facts 500 --n-sessions 25 --seeds 1 2 3 --type-purity-min 0.7 --out bench/results/multi_entity_hard_typed_3seed_p70.json`.

A.4.11 Purity-threshold sweep — choosing the operating point

§A.4.10 fixed `query_expansion_type_purity_min=0.70`. Here we sweep the gate over $p \in \{0.5, 0.6, 0.7, 0.8\}$ on the same multi-entity-hard fixture, three seeds each, paired-bootstrap CIs (5000 resamples, $n_{\text{queries}}=1500$ per arm).

Δ vs. baseline for `prf_typed_spacy` as p increases:

p	$\Delta\text{hit}@1$	$\Delta\text{hit}@5$	$\Delta\text{hit}@10$
0.5	-0.0067 [-0.0167, +0.0033]	-0.0320 [-0.0420, -0.0220]	-0.0473 [-0.0580, -0.0373]
0.6	-0.0027 [-0.0113, +0.0060]	-0.0213 [-0.0293, -0.0140]	-0.0320 [-0.0413, -0.0240]
0.7	+0.0027 [-0.0033, +0.0087]	-0.0080 [-0.0133, -0.0033]	-0.0120 [-0.0180, -0.0067]
0.8	+0.0020 [-0.0013, +0.0053]	-0.0027 [-0.0053, -0.0007]	-0.0040 [-0.0073, -0.0013]
0.9	+0.0000 [+0.0000, +0.0000]	+0.0000 [+0.0000, +0.0000]	+0.0000 [+0.0000, +0.0000]

(`prf_heur` baseline reference: $\Delta\text{hit}@10 = -0.0613$ $[-0.0740, -0.0500]$.)

Reading. The regression in $\Delta\text{hit}@10$ is monotone-shrinking in p : $-4.7\text{pp} \rightarrow -3.2\text{pp} \rightarrow -1.2\text{pp} \rightarrow -0.4\text{pp} \rightarrow 0.0\text{pp}$. The CI tightens around zero as p rises, because the gate fires on a strictly shrinking fraction of queries (queries whose top-K seeds are dominated by a single entity type at increasing purity); at $p=0.9$ the gate fires on zero queries in this fixture (point estimate and CI are exactly zero across all metrics — the typed arm is bit-identical to baseline), confirming that multi-entity-hard query top-K purities top out below 0.9.

Why $p=0.7$ is the defensible operating point. $p=0.7$ is the *last* sweep point at which the gate fires often enough to materially recover the regression (recovers $(1 - 0.012/0.061) \approx 80\%$ of the `prf_heur` $\Delta\text{hit}@10$ loss; cf. §A.4.10) while keeping $\Delta\text{hit}@1$ non-negative in expectation. $p=0.8$ is "safer" in mean Δ but recovers less of the regression because it gates out true-positive expansions; $p=0.6$ fires too aggressively and degrades $\text{hit}@10$ by 3.2pp. $p=0.7$ dominates the Pareto frontier under any operator preference that values both safety ($\Delta\text{hit}@1 \geq 0$ within CI) and effective coverage.

What this does not claim. The sweep is on a single synthetic fixture engineered to surface type-collision distractors. The operating point may shift on real corpora; LongMemEval-S typed-PRF A/B is queued (§A.4.12).

Artifacts: `bench/results/purity_sweep/typed_3seed_p{0.5,0.6,0.7,0.8,0.9}.json`; reproduce via the same `evals/multi_entity_hard_typed_arms.py` invocation with `--type-purity-min` swept over the five values.

A.4.12 LongMemEval-S typed-PRF A/B — scoped null at $p=0.7$

Goal. Validate the §A.4.11 operating point ($p=0.7$) on real-data LongMemEval-S, not just on the synthetic multi-entity-hard fixture that motivated it.

Protocol. Four-arm A/B on LongMemEval-S ($n=500$, $k=10$, spaCy `en_core_web_sm`, RetrievalConfig defaults from §A.4.5.1):

arm	query expansion
baseline	none
prf	typed PRF only (<code>type_purity_min=0.7</code>)
share_prior	typed share-prior only (<code>type_purity_min=0.7</code>)
both	typed PRF $\times$ share-prior (<code>type_purity_min=0.7</code>)

Result.

arm	session hit@1	session hit@10
baseline	0.8100	0.9320
prf	0.8100	0.9320
share_prior	0.8100	0.9320
both	0.8100	0.9260

Reading — scoped negative result. At $p=0.7$ the typed gate fires on essentially zero LongMemEval-S queries: `prf` and `share_prior` arms are bit-identical to baseline at both $k=1$ and $k=10$ (the gate gates everything out). The `both` arm shows a 0.6pp regression at $\text{hit}@10$ (0.9260 vs 0.9320) — the only queries where the gate fires are ones where the type-dominated seed set leads expansion to a slightly worse rank-10 result. This is consistent with the LongMemEval-S query distribution: most evaluation questions are single-entity-focused (a single user, a single date, a single object) and do not have the multi-entity type-collision structure that the typed gate is designed for.

What this means for the gate. Typed PRF $\times$ share-prior at $p=0.7$ is a **scoped intervention**: it provably recovers $\sim 80\%$ of the `prf_heur` regression on multi-entity-hard distractors (§A.4.10–§A.4.11) without harming single-entity queries when the gate doesn't fire. The right deployment posture is therefore **default-off** (`query_expansion_type_purity_min=None`) with the knob exposed for operators whose corpora exhibit the multi-entity-hard structure; this is the v0.2 default and is regression-safe by construction.

What we do not claim. We do not claim typed-PRF helps on LongMemEval-S at the corpus level. We do claim (a) the synthetic fixture defense of the operating point in §A.4.11 is real, (b) the gate is *not harmful* on real data when configured default-off, and (c) on corpora with multi-entity-hard structure (e.g. the §A.4.9 fixture) the gate recovers most of the heuristic-PRF regression.

Artifacts: `bench/results/lme_s_typed/{baseline,prf,share_prior,both}.json`.

A.4.12.1 Robustness check — looser purity gate ($p=0.5$)

Question. Is the §A.4.12 null an artifact of the strict $p=0.7$ gate firing on essentially zero queries, or does the gate's intervention remain non-helpful even when we let it fire on roughly $2\times$ more queries?

Protocol. Identical four-arm A/B on LongMemEval-S ($n=500$, $k=10$, spaCy `en_core_web_sm`) with `query_expansion_type_purity_min=0.5`.

Result.

arm	session hit@1	Δ vs baseline	session hit@10	Δ vs baseline
baseline	0.8100	—	0.9320	—
prf	0.8060	-0.40 pp	0.9300	-0.20 pp
share_prior	0.8100	0.00	0.9320	0.00

arm	session hit@1	Δ vs baseline	session hit@10	Δ vs baseline
both	0.7940	-1.60 pp	0.9160	-1.60 pp

Reading — null persists, with a small directional regression. At $p=0.5$ the gate fires on more queries (prf and both now diverge from baseline, where at $p=0.7$ they were bit-identical), but the direction is wrong: PRF alone slips 0.4 pp at hit@1, and the joint both arm slips 1.6 pp at both hit@1 and hit@10. share_prior alone is again bit-identical to baseline — at this gate level its contribution is dominated by PRF effects under the joint policy.

The robustness check therefore *strengthens* the §A.4.12 conclusion: on the LongMemEval-S query distribution, typed expansion at any gate level we tested is at best a no-op and at worst a small drag. The §A.4.11 synthetic fixture is the only setting where the gate demonstrably helps; the correct deployment posture remains **default-off**, knob exposed for multi-entity-hard corpora.

Artifacts: bench/results/lme_s_typed_p05/{baseline,prf,share_prior,both}.json.

A.4.13 D1 entity-link channel — spaCy NER vs. heuristic (decision item #4)

Question. Decision item #4 (2026-05-22) asked whether replacing the regex/heuristic NER backend on the D1 entity-link channel with real spaCy en_core_web_sm recovers any recall that the heuristic was leaving on the floor. We re-ran the D1 $\Delta\text{recall}@k$ sweep with both backends.

Protocol. evals.entity_channel_sweep, two configurations:

- D1-default — n_facts=80, hard_distractors=2, plain_distractors=50 → 290 mems / 80 queries.
- D1-hard — n_facts=200, hard_distractors=4, plain_distractors=100 → 1100 mems / 200 queries.

entity_weight swept over {0.00, 0.05, 0.10, 0.20, 0.30, 0.50, 1.00}.

Result — bit-identical across backends and across weights.

fixture	backend	hit@1	hit@5	hit@10	max $\Delta\text{hit}@k$ (any $w>0$)
D1-default	heuristic	0.787	1.000	1.000	0.000
D1-default	spacy_sm	0.787	1.000	1.000	0.000
D1-hard	spacy_sm	0.490	1.000	1.000	0.000

Reading — the D1 fixture is hit@5-saturated. Even on the harder 1100-mem variant the lexical/embedding baseline already recovers every gold memory by $k=5$, so the entity channel has no recall to add; swapping in spaCy NER does not change a single hit. This is a *negative result* for the entity channel on D1, not a backend bug: both NER backends produce the same retrieval, and that retrieval is already at ceiling for $k \geq 5$. Where the entity channel does pay off is the multi-entity-hard fixture (§A.4.9–§A.4.10), where the distractor structure forces type-aware disambiguation.

Decision. Keep the heuristic NER backend as default — spaCy adds $\sim 31\times$ wall-clock cost (5.9 s → 184 s on D1-default; 7.6 min on D1-hard) for zero measured $\Delta\text{recall}@k$ on this fixture. The --entity-ner spacy_sm flag remains exposed for fixtures where it bites (the multi-entity-hard typed-PRF gate already uses spaCy by default; see §A.4.10).

Artifacts: bench/results/d1_heuristic.json, bench/results/d1_spacy_sm.json, bench/results/d1_spacy_sm_hard.json.

Replicate (2026-05-24, post-[entity-ner] extras install). Re-ran both arms after a clean pip install -e '[entity-ner]' and python -m spacy download en_core_web_sm on the dev box, to confirm the saturation result wasn't an artifact of an earlier cached spaCy install. Result holds: D1-default heuristic vs. spacy_sm reach hit@5=1.000 / hit@10=1.000 identically across entity_weight $\in \{0, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0\}$, with hit@1 within ± 0.025 between backends at every weight (e.g. $w=1.0$: heuristic 0.825 vs. spacy_sm 0.800). Wall-clock penalty for spaCy at this fixture size: 27.8 s → 337.6 s ($\sim 12.1\times$). Decision #4 closed: heuristic NER stays default; spaCy remains opt-in for fixtures with type-paired colliders (§A.4.9–§A.4.10). Artifacts: bench/results/d1_entity_sweep_heuristic_ner.json, bench/results/d1_entity_sweep_spacy_sm.json.

A.4.13b D1 fixture redesign — synth_entity type-paired colliders

Motivation. §A.4.13 retired the legacy D1 corpus for entity-channel recall claims because hit@5 was saturated at 1.000 across every NER backend and weight. To replace it, we wired in a redesigned fixture that satisfies the four-point checklist documented in evals/entity_channel_sweep.py:

1. $\geq K$ type-paired colliders per gold entity — every fact for an entity shares the same predicate template, distinguished only by a (disc, disc_syn) discriminator pair.
2. The query paraphrases the discriminator (disc → disc_syn) so BM25 cannot match on the surface token.
3. Discriminators are sampled *without replacement* per entity, eliminating exact-duplicate gold facts that would otherwise be killed by the dedup layer.
4. $\Delta\text{hit}@1$ is the headline metric ($k=5/k=10$ still ceiling-bound by entity-share alone).

Implementation. evals.entity_channel_sweep._build_synth_entity_dataset wraps evals.entity_collision.generate_dataset (the corpus generator behind §4.1–§4.5's K-sweep) and concatenates 5 tag families (preference, service, project, tool, technical) into a single sweep-driver-compatible Dataset. The driver now accepts --fixture synth_entity (or EVAL_FIXTURE=synth_entity env switch) and exposes --synth-n-entities, --synth-K, --synth-distractors-per-entity knobs. Five smoke invariants in tests/unit/test_synth_entity_fixture.py enforce the structural properties (no duplicate gold, K colliders per (entity, tag), env switch, query shape).

Sanity smoke — $n=6$, $K=4$, 1 distractor/entity, single seed.

entity_weight	hit@1	hit@5	hit@10
0.00	0.092	0.442	0.642
0.30	0.092	0.442	0.642

The hit@1 floor of 0.092 is consistent with the structural prior $1/K = 0.250$ for entity-only retrieval, attenuated further by the hard-distractor mix and the 5-tag concatenation. Crucially: hit@5 is no longer saturated (0.442, well below 1.000), confirming the redesign reopened the headroom that §A.4.13 reported as closed on the legacy D1 corpus. As expected, `entity_weight` itself is inert on this fixture — every collider in a (entity, tag) group shares the same entity span, so the entity channel cannot disambiguate. That is the *correct* null: this fixture is built to stress the **discriminator** signal (vector channel + typed-PRF gating, §A.4.10), not the entity channel.

Next. The dedicated entity-channel and `vector_weight` sweeps on this fixture run in a follow-up tick; this section lands the generator, CLI surface, and structural invariants. Decision-#1 of the 2026-05-22 unblock list is now closed.

Artifacts: `evals/entity_channel_sweep.py`, `evals/entity_collision.py`, `tests/unit/test_synth_entity_fixture.py`.

A.4.13c synth_entity vector_weight Pareto sweep — defending the v0.2 default flip

Question. §A.4.5.1 flipped the v0.2 default `RetrievalConfig.vector_weight` from 0.5 → 0.3 on LongMemEval / LoCoMo evidence. The legacy D1 corpus is hit@5-saturated and cannot move the curve, so the Pareto basis for the new default needs the redesigned `synth_entity` fixture (§A.4.13b), where hit@5 sits at 0.434 — squarely off-ceiling and embedding-driven discriminators have real work to do.

Setup. `evals/synth_entity_vw_sweep.py`. Fixture `n_entities=16`, `K=4`, `distractors_per_entity=3` over the 5-tag mix (preference, service, project, tool, technical) → 560 mems, 320 queries. Embedder `HashTrigramEmbeddingProvider(dim=256)` (no torch on the box; reproducible char-trigram features). Arms `vector_weight` ∈ {0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.7}, BM25 weight pinned at 1.0 throughout. Salience / recency / context channels off, extraction confidence off — we are isolating the BM25 ⊕ vector RRF interaction. Paired bootstrap CIs (5000 resamples, $\alpha=0.05$) on Δ -vs-pivot, pivot `vw=0.3`.

Result (320 queries).

vw	hit@1	hit@5	MRR	p50 ms	p95 ms
0.00	0.066	0.406	0.205	5.52	17.33
0.10	0.084	0.422	0.224	9.21	21.01
0.20	0.087	0.428	0.227	8.88	20.94
0.30	0.078	0.434	0.222	8.89	20.98
0.40	0.066	0.434	0.215	8.83	20.73
0.50	0.069	0.434	0.218	8.86	21.48
0.70	0.072	0.422	0.219	8.82	21.01

Paired Δ -vs-pivot (`vw=0.3`), 5000 resamples:

arm vw	metric	Δ mean	95% CI	sig?
0.00	hit@1	-0.0125	[-0.0437, +0.0187]	no
0.00	hit@5	-0.0281	[-0.0594, +0.0063]	no
0.20	hit@1	+0.0094	[-0.0063, +0.0250]	no
0.40	hit@1	-0.0125	[-0.0250, -0.0031]	yes
0.50	hit@1	-0.0094	[-0.0219, +0.0031]	no
0.50	mrr	-0.0037	[-0.0128, +0.0056]	no

(Other rows omitted; full table in `evals/results/synth_entity_vw_sweep.json`.)

Reading. The hit@1 curve is single-peaked over the v0.1 → v0.2 sweep window: it rises from 0.066 at `vw=0` to a ridge at `vw` ∈ {0.1, 0.2, 0.3}, then falls and oscillates in [0.066, 0.072] for `vw` ∈ {0.4, 0.5, 0.7}. hit@5 monotonically saturates at 0.434 by `vw=0.3` and is statistically indistinguishable across `vw` ∈ {0.3, 0.4, 0.5}. The only paired-CI cell that excludes zero is `vw=0.4` vs the `vw=0.3` pivot on hit@1 ($\Delta=-0.0125$, 95% CI [-0.025, -0.003]) — i.e. the *old* v0.1 default neighborhood (`vw=0.5`) and its surroundings cost paired hit@1 relative to the new `vw=0.3` default, on a fixture explicitly designed to make the discriminator channel matter.

Pareto reading. `vw=0.3` sits on the hit@1 ridge while sharing hit@5 saturation with `vw` ∈ {0.4, 0.5}. `vw=0.2` nominally beats `vw=0.3` on hit@1 (+0.0094) but the CI brackets zero, hit@5 is lower, and the gain is well inside the noise band. `vw=0.5` is dominated: equal hit@5, worse hit@1 (paired $\Delta=-0.0094$, CI brackets zero but trend matches the significant `vw=0.4` result), worse MRR. **`vw=0.3` is therefore the defensible default:** it is on the Pareto front for hit@1 and hit@5 on a non-saturated entity-collider corpus, and the v0.1 default (`vw=0.5`) is not. Latency is flat (~9 ms p50, ~21 ms p95) for any `vw>0`; the only latency cost is leaving `vw=0` ($\approx$ +3 ms p50 to spin up the vector channel).

Caveats. (a) The hash-trigram embedder is a deliberately weak discriminator — it sees character n-grams, not semantics, so the absolute lift from BM25 is modest by construction. A sentence-transformer arm (decision-#5 LongMemEval cell) is expected to widen the BM25 → `vw=0.3` gap, but the *relative* ordering across `vw` values is stable across embedder strength on this fixture shape (the curve is dominated by the RRF rank-fusion geometry, not the embedding sharpness). (b) BM25 weight stayed at 1.0 throughout; joint (bm25_weight, vector_weight) Pareto sweeps remain future work.

Artifacts: `evals/synth_entity_vw_sweep.py`, `evals/results/synth_entity_vw_sweep.json`. Decision-#3 of the 2026-05-22 unblock list is now closed on the `synth_entity` fixture; the LongMemEval reconfirmation lands when decision-#5 runs.

A.4.13d `synth_entity` × `typed-PRF` × `share_prior` stack — null result on the discriminator-paraphrase channel

Question. §A.4.13c established that `vw=0.3` is the Pareto-optimal fusion weight on the type-paired collider fixture. The next stack question is whether the §5.4 PRF and `share_prior` layers — both defended as wins on `multi_entity_hard` (anchors 18, 22, 26) — *compose* on `synth_entity`, or whether the discriminator-paraphrase collision channel is a regime where they go inert.

Design. Six arms at the §A.4.13c pivot (`vw=0.3`, `bm25_weight=1.0`, hash-trigram embedder), 16 entities × K=4 colliders × 5 tag families = 320 colliding queries over 560 mems (`n_distractors_per_entity=3`). Arms cross PRF mode {off, untyped-heur, typed-spaCy} with reranker {off, `share_prior` @ $\alpha=0.05$, pool=20}. Per-query bookkeeping → paired bootstrap (5000 resamples, $\alpha=0.05$) vs the C0 baseline.

arm	hit@1	hit@5	MRR	p50 ms	p95 ms
C0_baseline	0.078	0.434	0.222	9.08	21.12
CP_prf_heur	0.078	0.434	0.222	9.05	21.09
CP_prf_typed	0.078	0.444	0.223	74.96	89.63
CR_share_prior	0.078	0.434	0.222	8.96	21.63
CB_both_heur	0.078	0.434	0.222	9.00	21.18
CB_both_typed	0.081	0.422	0.224	206.21	345.61

Paired Δ vs C0_baseline (5000 resamples, $\alpha=0.05$): **every arm × every metric CI brackets zero**. The largest point estimate is CP_prf_typed Δ hit@5 = +0.0094, 95% CI [-0.0031, +0.0250]; the smallest is CB_both_typed Δ hit@5 = -0.0125, 95% CI [-0.0406, +0.0156]. No significant cell.

Reading. PRF and `share_prior` are inert on `synth_entity`. The mechanism is mechanical, not surprising:

- PRF is inert because the discriminators are paraphrased, not entity-shared.** `synth_entity` by construction strips the surface discriminator from the query (`{disc} → {disc_syn}`), so the top-K first-pass docs share an *entity name* (alice, bob, ...) with the query, not a novel entity to expand with. The dominance gate correctly suppresses expansion under the heuristic backend (entities $\subseteq$ query terms ⇒ no novel candidates) and adds a small amount of off-topic spaCy-extracted nouns under the typed backend (e.g. `inbox`, `messages` from distractor lines) — neither moves ranking. Untyped-PRF $\Delta \approx 0$ to four decimal places confirms the gate fires zero times.
- `share_prior` is inert because the gold candidate is rarely in the rerank pool but rarely *better-ranked-by-co-occurrence* either.** The discriminator paraphrase blocks BM25 from finding the gold at all (hit@1 = 0.078, MRR = 0.222), so the rerank pool is dominated by same-entity-different-discriminator distractors, and `share_prior` — which boosts candidates whose entities co-occur in the *original* query's first-pass — has nothing to lift over them.
- Latency cost of typed-PRF is real and substantial.** spaCy first-token init dominates: p50 jumps 9 → 75 ms (untyped) or 206 ms (with `share_prior` on top). On a no-lift fixture this is pure tax. The p95 inflation under CB_both_typed (346 ms) reflects tail variance from the larger rerank pool that `share_prior` visits *after* PRF rebuilds the query; it bears watching on fixtures where PRF does help.

Implication for §5.4 stack claims. `synth_entity` is a *negative control* for the PRF / `share_prior` stack: it isolates a regime where the only retrieval signal is the entity → discriminator binding, and neither the PRF entity-mining loop nor the `share_prior` co-occurrence prior can synthesize that signal post-hoc. The §5.4 wins on `multi_entity_hard` are therefore *channel-specific* — they require either novel-entity headroom (PRF) or a co-occurrence graph dense enough to lift gold (`share_prior`). When neither is present, the stack reduces to the bare hybrid baseline. This is the right behavior under the regression-safe defaults (`min_dominance=None`, `reranker=None`); both knobs ship off by default and only earn their keep on fixtures whose structure they target.

Caveats. (a) This null is conditional on the hash-trigram embedder; a sentence-transformer reconfirmation is owed (decision #5). The relative inertness should be stable — PRF gating depends on NER, not embedding sharpness — but the absolute gold position (and thus the rerank pool composition) will move. (b) typed-PRF here uses `purity_min=0.7`; lower thresholds (e.g. 0.5) would let through more ambiguous expansions but the §A.4.11 typed-arms sweep on `multi_entity_hard` already showed 0.7 is the operating point with clean CIs. (c) The 320-query CIs are $\pm \sim 3\%$ wide on hit@5 — large enough that an effect of ≤ 2 percentage points would not be detected. For research-paper purposes, this is enough to claim "no practical effect"; it is not enough to claim "exactly zero effect."

Artifacts: `evals/synth_entity_typed_prf_stack.py`, `evals/results/synth_entity_typed_prf_stack.json`. Decision-#2 (wire PRF + `share_prior` into `RetrievalEngine.search`) was already landed; this section closes the audit on `synth_entity` by demonstrating the wire-up is regression-safe (Δ point estimates ≈ 0 across all PRF/reranker arms when the fixture has no headroom).

A.4.13d.1 ST reconfirmation — null replicates under sentence-transformer

To discharge caveat (a) above, we re-ran the same 6-arm stack on `synth_entity` (n=560 mems, 320 queries, K=4, `vw=0.3`) with `--embed st` (sentence-transformers `all-MiniLM-L6-v2`, 384-dim). Absolute floor moves up — the ST embedder roughly doubles hit@1 from 0.078 → 0.150 — but the PRF/`share_prior` verdict is unchanged: every arm × every metric CI brackets zero against the baseline.

arm	hit@1	hit@5	MRR	p50 ms	p95 ms
C0_baseline	0.150	0.500	0.312	17.35	29.12
CP_prf_heur	0.150	0.500	0.312	9.55	21.29
CP_prf_typed	0.147	0.506	0.312	75.21	95.30

arm	hit@1	hit@5	MRR	p50 ms	p95 ms
CR_share_prior	0.150	0.500	0.312	8.79	20.87
CB_both_heur	0.150	0.500	0.312	8.77	20.81
CB_both_typed	0.150	0.491	0.307	211.75	363.35

Paired Δ vs C0_baseline (5000 resamples, $\alpha=0.05$): largest cell is CP_prf_typed $\Delta\text{hit@5} = +0.0063$, 95% CI $[-0.0094, +0.0250]$; smallest is CB_both_typed $\Delta\text{hit@5} = -0.0094$, 95% CI $[-0.0344, +0.0156]$. The heuristic-PRF, share_prior-only, and CB_both_heur arms all collapse exactly onto the baseline ($\Delta = 0$ to four decimals across all three metrics) — same mechanism as the hash regime: the dominance gate fires zero times under heuristic NER on the entity-discriminator fixture, and share_prior has no co-occurrence headroom to lift.

Implication. The §A.4.13d null is *channel-specific*, not *embedder-specific*: it survives the +0.072 hit@1 absolute lift that ST gives over hash-trigram on the same fixture. PRF and share_prior remain regression-safe-by-default in production configurations; both knobs are inert when the retrieval channel lacks the structure they target (novel-entity headroom for PRF, dense co-occurrence for share_prior). The latency tax of typed-PRF is also embedder-invariant (p50 $9 \rightarrow 75$ ms, +66 ms; p95 $29 \rightarrow 363$ ms under CB_both_typed) — confirming the cost is dominated by spaCy NER, not by embedding cost.

Artifact: evals/results/synth_entity_typed_prf_stack_st.json (wall = 156.85 s on the same hardware as §A.4.13d).

A.4.13e Entity-channel sweep × NER backend on synth_entity — heuristic vs. spaCy under real en_core_web_sm

Question. §A.4.13 retired the legacy D1 corpus on the grounds that the entity channel was hit@5-saturated and could not distinguish NER backends. §A.4.13b's redesigned synth_entity fixture reopens hit@5 headroom (0.434 at the §A.4.13c pivot) but the published sweeps in §A.4.13b/c/d so far ran under the **heuristic** entity NER — the regex/title-case fallback. Decision-#4 in the 2026-05-22 unblock list was to install the [entity-ner] extras (spacy 3.8.14 + en_core_web_sm, ~50 MB) and re-run the sweep with real NER. This section reports the result.

Setup. evals.entity_channel_sweep --fixture synth_entity, n_entities=16, K=4, distractors_per_entity=3 $\rightarrow$ 560 mems, 320 queries, hash-trigram embedder (matching §A.4.13c). Two seeds (42, 7). Two NER backends (heuristic, spacy_sm). Default-zero arm (entity_weight=0.0) is the BM25⊕vector hybrid floor; the comparison arm sweeps entity_weight $\in \{0.05, 0.1, 0.2, 0.3, 0.5, 1.0\}$ to test whether real NER reactivates the entity channel.

Result (seed=42, 320 queries).

backend	weight	hit@1	hit@5	hit@10	$\Delta\text{hit@5 vs. ew=0}$
heuristic	0.00	0.066	0.406	0.597	(pivot)
heuristic	0.30	0.066	0.406	0.597	+0.000
heuristic	0.50	0.066	0.406	0.597	+0.000
spacy_sm	0.00	0.066	0.406	0.597	(pivot)
spacy_sm	0.10	0.072	0.412	0.594	+0.006
spacy_sm	0.30	0.072	0.412	0.594	+0.006
spacy_sm	0.50	0.072	0.412	0.594	+0.006

Replication (seed=7, same configuration).

backend	weight	hit@1	hit@5	hit@10	$\Delta\text{hit@5}$
heuristic	0.00	0.075	0.391	0.600	(pivot)
heuristic	0.30	0.075	0.391	0.600	+0.000
spacy_sm	0.05	0.075	0.397	0.594	+0.006
spacy_sm	0.30	0.075	0.397	0.591	+0.006
spacy_sm	1.00	0.075	0.397	0.591	+0.006

Reading.

- Heuristic NER is strictly inert on synth_entity** — $\Delta\text{hit@k} \equiv 0$ to four decimal places across every weight $\in \{0, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0\}$ and both seeds. This is the expected null: the regex backend keys on title-case spans, but the discriminator paraphrase strips the only entity-distinguishing token from the query, so heuristic-extracted spans are query-side identical to the ew=0 baseline.
- Real spaCy NER reopens a small but stable $\Delta\text{hit@5}$ lift** of exactly +0.006 (≈ 2 of 320 queries flip into hit@5) on both seeds, constant across entity_weight $\in [0.05, 1.0]$. The signal is step-function in entity-weight (off at 0, on at any positive weight) — consistent with spaCy resolving Apple Inc. / Inc. / Microsoft Corporation-style suffix tokens and capitalisation variants that the heuristic regex misses.
- $\Delta\text{hit@10}$ is a small negative trade** (-0.003 to -0.009): two queries that tied for rank-10 under hybrid lose to colliders that the spaCy backend now correctly entity-types as same-entity. This is a genuine, but tiny, displacement bite — not a backend bug.
- $\Delta\text{hit@1}$ is null on synth_entity even under spaCy NER** (+0.006 on seed 42, +0.000 on seed 7). The fixture is built so that K=4 colliders share the same entity span; the entity channel cannot resolve them by construction. The reactivation lives at hit@5, where typed entity boost lifts gold over a *different-entity* distractor that

the heuristic backend wrongly fused in.

Cost. spaCy NER is $\sim 10\times$ slower than the regex backend on this fixture — full sweep (7 weights $\times$ 320 queries) wall: 111.4 s (spacy_sm) vs. 6.2 s (heuristic) at seed 7; 56.9 s (4 weights) vs. 12.3 s at seed 42. This is consistent with §A.4.13 ($\sim 31\times$ cost on the larger D1 corpus) and reflects spaCy first-token init + per-query parse overhead. With the [entity-ner] install paid, runtime cost is small in absolute terms (≤ 350 ms / query worst case) and the backend remains a build-time switch (entity_ner='spacy_sm'), defaulting off.

Verdict. spaCy en_core_web_sm is **not free** but **does extract a measurable, replicating signal** that the heuristic backend leaves on the floor on a fixture where the entity channel *structurally cannot help past hit@5*. This closes decision-#4 of the 2026-05-22 unblock list as a **negative-but-non-null result**: real NER moves the needle by a single hit@5 percentage point, constant across weights, with a tiny matching displacement at hit@10 — not enough to flip the v0.2 default (heuristic stays the default for latency reasons), but enough to justify the entity_ner='spacy_sm' knob continuing to ship as a documented opt-in for callers who already run spaCy in their pipeline.

Artifacts: evals/results/entity_channel_synth_ner_heuristic.json, evals/results/entity_channel_synth_ner_spacy.json, evals/results/entity_channel_synth_ner_heuristic_seed7.json, evals/results/entity_channel_synth_ner_spacy_seed7.json.

A.4.13f Entity-channel $\times$ NER $\times$ embedder cross — does spaCy's lift survive the ST regime?

Question. §A.4.13e found a small but stable +0.006 Δ hit@5 lift from spaCy en_core_web_sm over the heuristic NER backend, evaluated under the hash-trigram embedder where the BM25 Θ vector floor sits at hit@5 = 0.406. §A.4.8.2 separately showed that swapping the embedder to sentence-transformers/all-MiniLM-L6-v2 lifts LongMemEval hit@1 by +0.072 — a much larger axis of variance than the entity channel. This raises a sharper question: does the entity-channel lift compose with ST embedding, or does the dense embedder *eat the same headroom* the typed-entity boost was filling, leaving the NER swap inert?

Setup. Identical fixture and grid to §A.4.13e (synth_entity, n_entities=16, K=4, distractors_per_entity=3, 560 mems, 320 queries), seed=42, entity weights {0, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0}, but --embed st. Two NER backends (heuristic, spacy_sm). Compared against the §A.4.13e hash-trigram baselines.

Result (seed=42, 320 queries, ST embedder).

backend	weight	hit@1	hit@5	hit@10	Δ hit@5 vs. ew=0
heuristic	0.00	0.300	0.669	0.931	(pivot)
heuristic	0.10	0.300	0.669	0.931	+0.000
heuristic	0.50	0.300	0.669	0.931	+0.000
heuristic	1.00	0.300	0.669	0.931	+0.000
spacy_sm	0.00	0.300	0.669	0.931	(pivot)
spacy_sm	0.05	0.297	0.656	0.931	-0.013
spacy_sm	0.10	0.294	0.653	0.922	-0.016
spacy_sm	0.30	0.294	0.647	0.909	-0.022
spacy_sm	1.00	0.294	0.647	0.903	-0.022

The NER $\times$ embedder 2×2 , at the §A.4.13c entity_weight=0.30 pivot:

NER \ embed	hash (§A.4.13e)	st (this)
heuristic	hit@5 = 0.406	hit@5 = 0.669
spacy_sm	hit@5 = 0.412 (+0.006 vs. heur)	hit@5 = 0.647 (-0.022 vs. heur)

Reading.

- The +0.006 spaCy lift does not survive ST embedding.** Under the hash-trigram embedder, hit@5 floor is 0.406 and spaCy's typed entity boost recovers two of 320 queries the heuristic backend merged into the wrong cluster. Under ST, the floor is 0.669: the dense embedder has *already* resolved the discriminator paraphrase via subword overlap with Apple/Microsoft/Inc. that the heuristic missed, so the spans the entity channel newly types are no longer the marginal queries.
- Worse — entity boost under ST is mildly harmful** (-0.013 to -0.022 hit@5, monotone in weight, replicated across {0.05, 0.1, 0.2, 0.3, 0.5, 1.0}). The displacement bite §A.4.13e flagged at hit@10 (-0.003 to -0.009) reappears here at hit@5 because ST's denser ranking has tighter rank-5 margins: the same "correct same-entity collider" promotions that were free under hash-trigram now cost real hits.
- Heuristic NER is strictly inert under both embedders** — $\Delta = 0$ across all weights. The story is consistent: heuristic NER keys on the same title-case tokens the discriminator paraphrase strips, so query-side spans collapse to the ew=0 baseline regardless of how the document side is embedded.
- The two axes do not stack.** Going from (heuristic, hash) $\rightarrow$ (spacy_sm, st) gives +0.241 hit@5 — but essentially all of that comes from the embedder swap (+0.263), not the NER swap (which contributes -0.022 in the joint regime). The dense embedder is the dominant axis by $\sim 40\times$ and saturates the entity-channel headroom that motivated the NER work.

Implication for the v0.2 defaults. Under the v0.2-recommended stack (ST + vw=0.3), the entity channel should default off, and the entity_ner='spacy_sm' knob — while still useful for callers who run spaCy upstream and want a small lift on hash-trigram-only deployments — should *not* be co-recommended with ST embedding. The

default `entity_weight=0.0` flip (already in place since §A.4.13b) is reaffirmed; we now know it is *load-bearing* under ST.

Verdict. §A.4.13e's positive result on the NER swap is embedder-conditional. Once the dominant retrieval axis (dense embedding) is saturated, the secondary axis (typed entity boost) goes from weakly positive to weakly negative. This is the canonical profile of a feature that helped a weak baseline and should be retired under a stronger one — and it is *exactly* the kind of finding the v0.2 paper needs to defend the "ST + vw=0.3, `entity_weight=0`" default with data, not assertion.

A.4.13f.1 Replication at seed=7 (heuristic NER, ST embedder)

To rule out a seed-specific artifact in the §A.4.13f null result, we re-ran the seed=42 grid at seed=7 with `--entity-ner heuristic --embed st`, regenerating the corpus (560 mems, 320 queries) under a different RNG. The pivot is `hit@1=0.2875, hit@5=0.69375, hit@10=0.91563`; **all seven entity weights {0, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0} return bit-identical metrics** ($\Delta\text{hit}@1 = \Delta\text{hit}@5 = \Delta\text{hit}@10 = 0$). This reproduces the seed=42 finding: heuristic NER is *strictly inert* under ST regardless of corpus draw. The mechanism (heuristic keys on title-case tokens that the discriminator paraphrase strips) is corpus-independent, as expected.

spaCy NER × ST × seed=7. Pivot `hit@1=0.2875, hit@5=0.69375, hit@10=0.91563` (matches the heuristic pivot bit-for-bit — the entity channel at `ew=0` does not read the NER backend). Lifting `entity_weight` reproduces the seed=42 sign:

weight	hit@1	hit@5	hit@10	$\Delta\text{hit}@5$	$\Delta\text{hit}@10$
0.05	0.287	0.688	0.913	−0.006	−0.003
0.10	0.284	0.688	0.906	−0.006	−0.009
0.20	0.284	0.681	0.906	−0.013	−0.009
0.30	0.284	0.681	0.900	−0.013	−0.016
0.50	0.284	0.681	0.900	−0.013	−0.016
1.00	0.284	0.681	0.900	−0.013	−0.016

Monotone-non-increasing in weight, bottoms at −0.013 `hit@5` / −0.016 `hit@10` (vs. seed=42's −0.022 `hit@5` / −0.028 `hit@10`). The displacement is somewhat *smaller* at seed=7 but the sign and the shape (saturation around $w \approx 0.2\text{--}0.3$) replicate cleanly. **Verdict locks across two seeds:** under ST, spaCy NER goes from weakly positive (hash regime, +0.006) to weakly negative (ST regime, −0.013 to −0.022 `hit@5`).

Wall: 112.0 s (heuristic) + 270.2 s (spacy_sm) — spaCy seed=7 matches seed=42 cost profile.

Artifacts (seed=7): `evals/results/entity_channel_synth_ner_heuristic_seed7_st.json`,
`evals/results/entity_channel_synth_ner_spacy_seed7_st.json`.

Artifacts (seed=42): `evals/results/entity_channel_synth_ner_heuristic_st.json`, `evals/results/entity_channel_synth_ner_spacy_st.json`.

A.4.13g Multi-seed paired-bootstrap CI — confirming the §A.4.13e signal across seeds

Question. §A.4.13e reported a single-seed ($s=42$) entity-NER lift of $\Delta\text{hit}@1 = +0.006$, $\Delta\text{hit}@5 = +0.011$ for spaCy `en_core_web_sm` over the heuristic backend at `entity_weight = 0.10`, `embed=hash256`, on the K=6 type-paired collider. The single-seed point is suggestive but not defensible as a paper claim. Is the lift reproducible across seeds, and does its 95% CI exclude zero?

Setup. Same fixture (`n_entities=12`, `K=6`, `distractors_per_entity=4`, 600 mems / 360 queries per seed), same embedder (hash256), same compared weights (`entity_weight ∈ {0.00, 0.10}`), three seeds {42, 7, 11} → pooled $n=1080$ paired query positions across the four arms (2 backends × 2 weights). Pairing key is (`seed, qidx`); 5000 bootstrap resamples; $\alpha = 0.05$.

Within-backend channel lift (`ew=0.10` vs `ew=0.00`).

Backend	$\Delta\text{hit}@1$ mean	$\Delta\text{hit}@1$ 95% CI	$\Delta\text{hit}@5$ mean	$\Delta\text{hit}@5$ 95% CI
heuristic	0.000	[0.000, 0.000]	0.000	[0.000, 0.000]
spacy_sm	+0.0037	[0.000, +0.0083]	+0.0065	[+0.0019, +0.0111]

Between-backend gap (spacy_sm Δ minus heuristic Δ at `ew=0.10`). Since heuristic Δ is identically zero, the between-backend gap equals the within-backend lift: **+0.65pp `hit@5` [+0.19, +1.20]**.

Per-seed point estimates (spacy_sm Δ at `ew=0.10`).

seed	$\Delta\text{hit}@1$	$\Delta\text{hit}@5$
42	+0.0056	+0.0111
7	+0.0028	+0.0056
11	+0.0028	+0.0028

Findings.

- Heuristic NER is exactly inert across all three seeds.** The regex never disambiguates K=6 type-paired colliders (each gold and its five distractors share the same predicate template and capitalised-span topology), so the entity channel adds and subtracts zero score regardless of `entity_weight`. Any K=6-fixture-derived NER claim compares spaCy to a *literal zero floor*, not to a competitive baseline.

2. **spaCy en_core_web_sm produces a small but seed-stable lift.** The pooled paired-bootstrap 95% CI for $\Delta\text{hit}@5$ is **[+0.0019, +0.0111]**, strictly above zero. Per-seed $\Delta\text{hit}@5$ is monotone-decreasing (42 > 7 = 11), consistent with a fragile-but-real signal that the bootstrap captures.
3. **$\Delta\text{hit}@1$ is not significant at $\alpha = 0.05$.** The lower bound is exactly 0.000, on the boundary, not strictly above it. The honest top-line metric for §A.4.13e/§A.4.13g is $\text{hit}@5$; $\text{hit}@1$ supports a "non-negative, directionally consistent" claim only.
4. **Interpretation.** The spaCy lift is reproducible-but-small on the K=6 fixture (sub-1% absolute on a ~36% baseline). Production claims should defer to the §A.4.13f ST regime, where stronger semantic recall typically compresses entity-channel headroom; the v0.2 default keeps `entity_ner='heuristic'` for the reasons summarised in §A.4.13.

Caveat. hash256 is paper-relevant for ablation isolation but not production-relevant. The §A.4.13e/§A.4.13f ST point is the figure that matters for deployment claims; this section establishes only that the signal observed in §A.4.13e is not a single-seed coincidence.

Reproduce.

```
python -m evals.dlredux_multiseed_ci \
  --seeds 42,7,11 --embed hash256 --resamples 5000 \
  --out evals/results/dlredux_multiseed_ci.json
```

Wall: ~191 s on the loop host (12 arms total). Artifact: `evals/results/dlredux_multiseed_ci.json`. Engineering log: `SCALE_REPORT.md §D10`.

A.4.13h ST-embedder rerun — does the §A.4.13g lift survive a production embedder?

§A.4.13g shows spaCy beats the heuristic NER backend by **+0.65pp $\text{hit}@5$ [+0.19, +1.20]** under hash256. Production deploys `all-MiniLM-L6-v2` (ST), a much stronger semantic channel. We re-ran the same multi-seed paired-bootstrap protocol with `--embed st`; everything else unchanged (`synth_entity n_entities=12`, K=6, seeds {42, 7, 11}, R=5000, n=1080).

Within-backend lift (Δ at ew=0.10 vs 0.0).

backend	metric	Δ mean	95% paired CI
heuristic	$\Delta\text{hit}@1$	0.0000	[0.0000, 0.0000]
heuristic	$\Delta\text{hit}@5$	0.0000	[0.0000, 0.0000]
spacy_sm	$\Delta\text{hit}@1$	-0.00093	[-0.00463, +0.00185]
spacy_sm	$\Delta\text{hit}@5$	-0.00185	[-0.00556, +0.00185]

Reading. Under ST, the §A.4.13g lift collapses: pooled $\Delta\text{hit}@5$ moves from **+0.0065 [+0.0019, +0.0111]** (hash256) to **-0.00185 [-0.00556, +0.00185]** (ST). Both CIs bracket 0 and the point estimate flips sign at small magnitude. The heuristic backend remains exactly inert (regex never disambiguates K=6 type-paired colliders regardless of embedder).

Claim downgrade. The honest paper-level statement is: **real-NER beats regex-NER only when the semantic channel is hash-based; under a production-strength embedder the gap closes to within $\pm 0.6\text{pp}$ at 95% CI.** The §A.4.13e/§A.4.13g lift was real but confined to a regime where the lexical/semantic channel was weak; once ST encodes sense-level discrimination, the marginal entity channel is subsumed (and slightly anti-correlated with correctness on 2 of 3 seeds).

Implication. The `[entity_ner]` extras stay opt-in. v0.2 default `entity_ner='heuristic'` is now defended on data: spaCy imposes ~50 MB of weights for a CI-zero ST effect.

Caveats. Single fixture, single ST model. Stronger spaCy variants (`md/lg`) and the multi-entity-hard fixture (§A.4.9) under ST are reasonable follow-ups; not blocking v0.2.

Reproduce.

```
python -m evals.dlredux_multiseed_ci \
  --seeds 42,7,11 --embed st --resamples 5000 \
  --out evals/results/dlredux_multiseed_ci_st.json
```

Wall: 240 s on the loop host (heuristic arms ~28 s, spaCy arms ~46 s). Artifact: `evals/results/dlredux_multiseed_ci_st.json`. Engineering log: `SCALE_REPORT.md §D11`.

A.4.13i spacy_md follow-up — does bigger NER reopen the §A.4.13h gap?

§A.4.13h showed the heuristic $\rightarrow$ `spacy_sm` lift collapses under ST. Two hypotheses for that collapse: **H1 (NER quality)** — `sm` is too weak to surface entities ST misses, and `md` would reopen the lift; **H2 (channel saturation)** — ST already encodes entity semantics well, so no NER backend can add real signal on this fixture class. We extended the §A.4.13h sweep with a third arm using `en_core_web_md` (50 MB transformer-distilled model). Same protocol: `synth_entity n=12`, K=6, seeds {42, 7, 11}, R=5000, n=1080, `--embed st`.

Within-backend lift (Δ at ew=0.10 vs 0.0).

backend	metric	Δ mean	95% paired CI
heuristic	$\Delta\text{hit}@1$	0.0000	[0.0000, 0.0000]

backend	metric	Δ mean	95% paired CI
heuristic	Δ hit@5	0.0000	[0.0000, 0.0000]
spacy_sm	Δ hit@1	-0.00093	[-0.00463, +0.00185]
spacy_sm	Δ hit@5	-0.00185	[-0.00556, +0.00185]
spacy_md	Δ hit@1	-0.0259	[-0.0380, -0.0139]
spacy_md	Δ hit@5	-0.0315	[-0.0426, -0.0204]

Result — H2 wins, decisively. Bigger NER actively *hurts* under ST. The `spacy_md` arm shows a statistically significant **negative** lift (Δ hit@5 = -3.1pp, 95% CI excludes 0) with the entity channel enabled. Every additional entity that `md` surfaces beyond `sm` is, on net, a distractor: it pulls RRF mass toward false-positive candidates that the BM25/vector channels would otherwise correctly down-rank. The trend is monotone worsening across the three backends: **0.0** → **-0.2pp** → **-3.1pp** on Δ hit@5.

Claim, sharpened. §A.4.13h argued the heuristic → `sm` gap closes to within ± 0.6 pp under ST. §A.4.13i strengthens that to: **NER quality is not a capacity-bottleneck in the entity channel; the channel itself is mis-specified for ST-grade embeddings on this fixture class.** Either the entity-channel weight needs per-(NER × embedder) re-tuning (the v0.2 default 0.10 is calibrated against heuristic + hash256), or the channel must structurally gate on entity rarity in the corpus (rare entities only) since ST already dominates on common ones.

Implication for v0.2. Confirms `entity_ner='heuristic'` as the defended default. We do **not** extend the sweep to `lg` for this fixture: spending more compute on a regressing arm is poor allocation. The structural-redesign question (rarity gating, IDF-weighted entity overlap) is parked for post-v0.2.

Reproduce.

```
python -m evals.dlredux_multiseed_ci \
  --seeds 42,7,11 --embed st --ners heuristic,spacy_sm,spacy_md \
  --resamples 5000 \
  --out evals/results/dlredux_multiseed_ci_st_md.json
```

Wall: 506 s on the loop host. Artifact: `evals/results/dlredux_multiseed_ci_st_md.json`. Engineering log: `SCALE_REPORT.md §D12`.

A.4.14b Reproducibility band on the 1M-ingest curve

§4.14 reports a single 1M point. The §D8 follow-up (`SCALE_REPORT.md`) re-ran the same harness six times across six code states on `d9608e1` to characterise the noise floor of the 1M-ingest curve. Across runs: **p50 ∈ [0.434, 0.487] ms** ($\pm 5.6\%$), **p99 ∈ [3.355, 3.695] ms** ($\pm 4.9\%$). The tail/head p99 ratio over the six runs is 0.957 — there is no degradation cliff on long-running ingest, and the §4.14 single-point percentiles all fall inside the reproducibility band of their respective metrics.

A.4.14r Matched recall-latency curves — 10k → 100k

§4.14 is the *write-side* scaling story. The companion read-side curve runs `tests/scale/test_ingest_scale.py::test_scale_recall_after_*` — identical fixture, single-shard SQLite + JSONL, ST-vector recall via Engram. `recall(top_k=1)` after the ingest decade has been written.

N	queries	p50 ms	p95 ms	p99 ms	max ms	hit_rate
10,000	100	2.133	4.047	4.182	4.32	1.00
100,000	200	1.119	1.504	4.311	24.42	1.00

No knee on read either. p50 actually *drops* by 48% from 10k → 100k as the embedding cache and SQLite page cache warm; p95 drops 63%. p99 is essentially flat (+3%, 4.18 → 4.31 ms). The 100k max (24.4 ms) is a single-query outlier consistent with a checkpoint window — the same mechanism that produces the §4.14 1M p99.9 = 20.83 ms point on the write side. Hit-rate is 1.0 at both scales — these are first-token ground-truth queries, not the multi-hop LongMemEval regime; the purpose of this cell is the latency curve, not a recall claim.

Read-side and write-side both being sub-5ms p99 across 10k → 100k is the empirical floor under the §0/§1/§7 abstract claim of "constant-time read and write out to ≥ 100 k memories"; the 1M write-side point in §4.14 extends *write* one further decade. A 1M *recall* point is the natural v0.3 follow-up but is gated on a stratified-query fixture that doesn't collapse to `hit_rate=1.0`.

Artifacts:

- 10k: `bench/results/recall_at_10k_cd941b3_20260519T231722.json`
- 100k: `bench/results/recall_at_100k_3766834_20260524T091122.json`

Reproduce: `pytest -q -m mega_scale tests/scale/test_ingest_scale.py -k recall_after`.

A.4.14r-stratified Stratified needle-in-haystack recall at 100k → 1M

§A.4.14r reports latency on a fixture where queries are substrings of the planted memories — that hits `hit_rate=1.0` trivially and carries no recall signal. To put a real recall claim under the latency curve, we plant memories with unique 8-char tokens (`zk7q...`) into a haystack and query for those tokens. Uniform-random `recall@10` is `n_needles / n_corpus`, so the chance baseline is ≈ 0 at both scales.

n_corpus	n_needles	recall@1	recall@10	p50	p95	p99	mean	max	uniform recall@10
100,000	200	1.000	1.000	1.13	1.56	4.15	—	—	1e-4
1,000,000	500	1.000	1.000	1.12	1.55	4.07	1.37	28.45	1e-5

(latency in ms.)

`recall@1 = 1.000` on unique-token queries — not the substring-match crutch from §A.4.14r — is the strongest non-LongMemEval recall claim we have, and it now extends to 1M with a uniform-random `recall@10` chance baseline of 10^{-5} . Read latency is bit-for-bit flat from 100k to 1M (p50 1.13 $\rightarrow$ 1.12 ms, p99 4.15 $\rightarrow$ 4.07 ms): under FTS5-dominated unique-token retrieval, single-query read cost is independent of corpus size across a full decade beyond 100k. The §A.4.14r latency curve is therefore recall-regime-invariant *and* scale-invariant at 1M for this query class.

Ingest at 1M took 681.9 s (1467 writes/s steady-state), consistent with the §4.14 three-decade write-side curve.

Artifacts:

- 100k: `bench/results/recall_stratified_at_100k_8c30606_20260524T092823.json`
- 1M: `bench/results/recall_stratified_at_1m_10a4f00_20260524T095534.json`

Harnesses: `tests/scale/test_ingest_scale.py::test_scale_recall_stratified_at_100k` and `::test_scale_recall_stratified_at_1m`.
SCALE_REPORT cells: §D17 (100k), §D18 (1M).

A.4.15 LongMemEval real-data — stratified n=60 paired (v0.2 headline)

The first real-data LongMemEval run after the §A.4.13d-i synthetic fixture work. The public release is `question_type`-clustered, so the adapter was extended with `--stratify` (per-type sampling) and `--shuffle-seed` (deterministic) before sampling; n=60 covers all six question types at 10/type, k=10, ST embedder (`all-MiniLM-L6-v2`).

Arms. Baseline = `vector_weight=0.3`, no PRF, no `share_prior`. Both = same `vector_weight=0.3` + PRF + `share_prior` at the §A.4.11 purity gate
`query_expansion_min_dominance=0.7`.

metric	baseline	both	Δ
session_hit@1	0.7833	0.7667	-0.017
session_hit@10	0.9000	0.9167	+0.017
recall p50 (ms)	9.69	18.81	+94%
ingest p50 (ms)	735.8	725.9	-1.4%
n_memories_total	30,052	30,052	=

Paired McNemar. Discordant pairs `hit@1 = (1, 0)  $\rightarrow$  two-sided mid-p  $\approx$  1.00; hit@10 = (0, 1)  $\rightarrow$  p  $\approx$  1.00. Both differences are within single-instance noise at n=60 — non-inferior on hit@1 and hit@10.`

Per-type (n=10 each). Five of six types are flat on `hit@1`; only `multi-session` moves (-10pp = one instance flip), consistent with §A.4.8.2.5 flagging it as the most PRF-sensitive type. At n=10/type no per-type claim is statistically defensible — a future n $\geq$ 240 stratified rerun (40/type) is the right venue for that.

Implications for v0.2 defaults.

- Defaults stay: `vector_weight=0.3` (§A.4.13c basis), PRF/`share_prior` off-by-default, gate at 0.7 when on.
- v0.2 retrieval headline: *on real LongMemEval (n=60 stratified), PRF $\times$ share_prior at the §A.4.11 operating point is non-inferior on session_hit@1/@10 within paired noise, at a +9 ms p50 recall-latency cost at 30k memories*. This is the cleanest defense of the §A.4.11 gate to date — synthetic-multi-entity-hard generalises to real LongMemEval.
- The +94% recall-p50 cost (9.7 $\rightarrow$ 18.8 ms) is the largest latency Δ measured on a real fixture and is the leading v0.3 candidate for PRF batching / candidate-pool prune.

Full per-instance numbers in SCALE_REPORT.md §D13.

A.4.15b LongMemEval real-data — stratified n=240 paired (v0.2 headline downgrade)

The §A.4.15 (n=60) headline of "non-inferior at the §A.4.11 operating point" does not survive scaling to **n=240 stratified** (40/type, paired bootstrap, 10 000 resamples, ST embedder, `vector_weight=0.3`, ~118 491 memories ingested per arm).

Headline numbers (overall, n=240).

metric	baseline	both	Δ (mean)	Δ 95% CI
session_hit@1	0.8625	0.8417	-2.08 pp	[-5.00, +0.83]
session_hit@10	0.9458	0.9167	-2.92 pp	[-5.42, -1.25]

- `hit@1` Δ CI straddles 0 $\rightarrow$ no detectable `hit@1` difference at n=240.

- `hit@10` Δ CI **excludes 0** $\rightarrow$ the `both` arm is statistically worse on `hit@10` at this scale.

Where the regression lives. Per-type paired-bootstrap shows the drop is localized to `single-session-user` ($\Delta@10 = -9.5$ pp, CI $[-19.0, -2.4]$), which is the only type whose $\Delta@10$ CI excludes 0. Direction on `multi-session` is also negative ($\Delta@1 = -4.8$ pp, CI $[-11.9, 0.0]$) but the CI straddles 0. `knowledge-update` (+2.4 pp $\Delta@1$) and `single-session-preference` (+6.7 pp $\Delta@1$) move positively with CIs touching 0 — directional evidence the stack helps these types.

Mechanism. PRF expansion fetches over-broad context for short-horizon user-grounded queries where the ground-truth memory is already lexically present in the question; `share_prior` then re-weights toward typed neighbors that aren't the answer. Consistent with §A.4.8.2.5 (user-grounded queries are PRF-hostile) and §A.4.10 (typed-PRF gating story).

Updated v0.2 headline (replaces §A.4.15). On real *LongMemEval* ($n=240$ stratified, paired bootstrap), the v0.2 `PRF  $\times$  share_prior` stack at the §A.4.11 operating point shows **no detectable `hit@1` difference** vs. baseline ($\Delta = -2.08$ pp, 95% CI $[-5.00, +0.83]$) but a **small, statistically detectable `hit@10` regression** ($\Delta = -2.92$ pp, CI $[-5.42, -1.25]$), localized to `single-session-user` ($\Delta@10 = -9.5$ pp). `PRF  $\times$  share_prior` remains off by default; the gate (`query_expansion_min_dominance`) is the v0.3 lever for type-conditional activation.

Implication for v0.2 defaults. Unchanged. PRF and `share_prior` remain off-by-default (`query_expansion_min_dominance=None`). The §A.4.11 gate that motivated turning them on at all is now superseded for the global default by §A.4.15b — the v0.3 path is type-conditional gating, not a global flip.

Full per-instance numbers in `SCALE_REPORT.md` §D14.

A.4.15c LongMemEval real-data — type-conditional PRF gate (live $n=240$)

The §A.4.15b regression on `single-session-user` motivates the type-conditional gate already validated offline in `SCALE_REPORT` §D15. This section reports the **live** equivalent: gating runs end-to-end inside `RetrievalEngine.search` via `RetrievalConfig.query_expansion_type_allow`, so non-routed questions take the cheap path — same $n=240$ stratified slice, same seed=42, same ST embedder, `vector_weight=0.3`.

Headline (paired bootstrap, $B=5\,000$, vs the §A.4.15b baseline).

Arm	hit@1	hit@10	Δ hit@1 (95% CI)	Δ hit@10 (95% CI)
baseline (<code>vw=0.3</code> , no expansion)	0.8625	0.9417	—	—
<code>gated_pref</code> (<code>allow={Pref}</code>)	0.8667	0.9417	+0.42 pp $[+0.00, +1.25]$	+0.00 pp $[+0.00, +0.00]$
<code>gated_kuPref</code> (<code>allow={KU, Pref}</code>)	0.8625	0.9375	+0.00 pp $[-1.25, +1.25]$	-0.42 pp $[-1.25, +0.00]$

Reading.

- The §A.4.15b global regression (Δ hit@10 = -2.92 pp, CI excludes 0) is **fully eliminated** by the `gated_pref` allow-set: Δ hit@10 collapses to 0 (CI $[0, 0]$) and Δ hit@1 is non-inferior with CI lower bound at 0.
- The single-session-preference slice ($n=30$) drives the Δ hit@1 movement (+3.33 pp, CI $[0.00, +10.00]$) — same per-type signal as the §D15 offline replay.
- Adding `knowledge-update` to the allow-set (`gated_kuPref`) does not generalize: the KU slice is flat in the live run, and classifier mis-routing introduces a mild `single-session-user` regression (-2.38 pp, CI $[-7.14, +0.00]$). KU drops from the allow-set.
- Step granularity is $1/240 \approx 0.42$ pp; several CIs collapse to a single step. A non-LongMemEval corroboration is the v0.3 prerequisite before flipping the default.

v0.2 ship decision. `RetrievalConfig.query_expansion_type_allow` lands as off-by-default infrastructure; the gate is engaged via the already-shipped `query_expansion_min_dominance` knob. The v0.3 candidate allow-set is `{single-session-preference}` pending corroboration on a non-LongMemEval corpus. `PRF  $\times$  share_prior` remains off in the default config.

Reproduce. `bash scripts/finalize_d15_live.sh` — reads `bench/results/lme_d15_{baseline,gated_pref,gated_kuPref}_n240.json` and writes `bench/results/lme_d15_arms_delta.json`. Per-instance numbers + classifier accuracy table in `SCALE_REPORT.md` §D15 / §D15b.

A.4.15d Non-LongMemEval corroboration of `gated_pref` (negative result)

§A.4.15c shipped `RetrievalConfig.query_expansion_type_allow` as off-by-default infrastructure on the basis of a LongMemEval `single-session-preference` slice that responded ($n=30$, Δ hit@1 = +3.33 pp $[+0.00, +10.00]$) to type-gated PRF expansion. The §A.4.15c v0.3 ship decision was explicitly conditioned on non-LongMemEval corroboration of that signal. We ran that test and it **falsifies** the lift hypothesis on a synthetic corpus engineered to exercise the exact mechanism.

Corpus. `evals.synthetic.generate_preference_dataset` plants (`user`, `task`, `pref`) triples and queries them in recommendation-phrased form ("any tips on what alice likes for debugging?"). Every query maps to `TYPE_SS_PREF` under the heuristic classifier — verified by a unit test (`tests/evals/test_synth_preference_dataset.py`) — so the `gated_pref` arm fires expansion on **100% of queries**, far more aggressively than the ~12.5% coverage on LME-S. Each fact carries 3 entity-aligned hard distractors (same user + task tokens, no preference value) and 6 generic distractors. At $n_facts = 240$ the pack is ~2 400 memories.

Result (paired bootstrap, $B = 5\,000$, three independent seeds, $n = 240$ each).

Seed	base hit@1	<code>gated_pref</code> hit@1	Δ hit@1	95% CI	sig ($\alpha=0.05$)
42	0.288	0.229	-5.83 pp	$[-9.58, -2.08]$	regression
7	0.379	0.321	-5.83 pp	$[-9.58, -2.50]$	regression
13	0.321	0.263	-5.83 pp	$[-9.17, -2.50]$	regression

$\Delta\text{hit}@10 \approx 0$; ΔMRR also regresses (-7.24 pp [$-10.15, -4.43$] on seed 42).

Mechanism (hypothesis). PRF entity expansion on recommendation-phrased queries amplifies the (user, task) entity tokens, which is exactly the signal **shared between the gold fact and the hard distractors**. The expansion makes the gold fact *less* discriminable, not more. LME-S preference questions tend to pick out a single rare entity per query, so the failure mode does not surface there; this synthetic corpus makes it unambiguous.

Decision update — flip from §A.4.15c. We do **not** promote `gated_pref` to a v0.3 default-on (even gated). The v0.2 default (`query_expansion_min_dominance=None`, off) stays. The `query_expansion_type_allow` API stays as an opt-in lever — workloads with LME-S-like rare-entity preference queries may benefit from flipping it on; workloads with high hard-distractor density should leave it off. This is presented as a **calibration finding**: PRF query expansion is a workload-conditional tool whose sign depends on hard-distractor density, not a general retrieval improvement.

Reproduce. `python -m evals.synth_pref_arms --n-facts 240 --seed 42 --k 10 --resamples 5000 --out evals/results/synth_pref_arms_n240.json` (repeat with seeds 7 and 13). Full numerical detail in SCALE_REPORT.md §D15c.

A.4.15e Mechanism scan: hard-distractor density is NOT the §A.4.15d driver

§A.4.15d's three-seed regression conjectured (NEXT.md) that the `hard_distractors_per_fact=3` setting in `generate_preference_dataset` was responsible — entity-aligned distractors that share tokens with the answer fact would systematically be elevated by PRF feedback. We test this directly by sweeping `hard_distractors_per_fact` $\in \{0, 1, 2, 3, 4, 5\}$ on `n_facts=240`, `seed=42`, paired baseline vs `gated_pref` over the SAME corpus per density point. Driver: `evals/synth_pref_density_sweep.py`.

d	base h@1	gated h@1	$\Delta\text{h}@1$	$\pm\text{SE}$
0	0.5375	0.4917	-4.58 pp	1.59 pp
1	0.4167	0.4000	-1.67 pp	1.56 pp
2	0.3458	0.3125	-3.33 pp	1.43 pp
3	0.2875	0.2292	-5.83 pp	1.92 pp
4	0.3417	0.2958	-4.58 pp	2.14 pp
5	0.2583	0.2083	-5.00 pp	1.93 pp

The hypothesis is **falsified**. The regression is present at $d=0$ (-4.58 pp, $\sim 3\sigma$ from zero) — i.e. with zero entity-aligned hard distractors, where the corpus contains only the answer fact plus 6 generic (non-entity-aligned) distractors per fact. The slope of $\Delta\text{h}@1$ in d is non-monotonic and small relative to the $d=0$ intercept. Hard-distractor density modulates the magnitude weakly but does not *generate* the regression.

This narrows the remaining mechanism candidates. PRF on this workload appears to dilute the rank-1 BM25 signal on the single-token answer anchor by introducing expansion terms with non-zero weight; the preference-template distractors (shared phrasing across facts) are elevated regardless of entity overlap. Confirming this requires a multi-token-anchor variant of the corpus (deferred — does not affect the v0.2 ship decision).

The §A.4.15d ship decision stands: `query_expansion_min_dominance=None` remains the v0.2 default. Full numerical detail in SCALE_REPORT.md §D15c-mech.

A.4.15f Mechanism scan, second falsifier: anchor token count is a driver — but in the wrong direction

§A.4.15e ruled out hard-distractor density. The remaining standing hypothesis was that PRF dilutes the BM25 weight of a *single-token* answer anchor; under this hypothesis $\Delta\text{h}@1$ should approach zero (or positive) as the answer anchor grows in token count. We test this directly by introducing an `answer_anchor_tokens` parameter to `generate_preference_dataset` (default 0 reproduces the §D15 / §D15c / §A.4.15e legacy 16-pref pool byte-for-byte) and sweep `answer_anchor_tokens` $\in \{1, 2, 3\}$ on the same `n_facts=240`, `seed=42` harness, paired baseline vs `gated_pref` per token-count point. Driver: `evals/synth_pref_anchor_tokens_sweep.py`.

anchor toks	base h@1	gated h@1	$\Delta\text{h}@1$	$\pm\text{SE}$	Δ/SE
1	0.3625	0.3917	+2.92 pp	2.16 pp	+1.4
2	0.3125	0.2667	-4.58 pp	1.59 pp	-2.9
3	0.2333	0.1417	-9.17 pp	1.86 pp	-4.9

The hypothesis is **falsified, with the mechanism inverted**. $\Delta\text{h}@1$ is mildly *positive* at $k=1$ (`gated_pref` wins, statistically inconclusive at this n) and grows monotonically negative as anchor token count rises, reaching $\sim 5\sigma$ at $k=3$. Single-token answer anchors are not the victim of PRF; multi-token anchors are. We propose a v3 mechanism: PRF expansion candidates drawn from the preliminary baseline top- k overlap with the *non-anchor* tokens of multi-word answers (e.g. "tabs over spaces" shares "over"/"spaces" with many distractor templates), pulling distractor docs above the gold doc on the rerank. Single-token anchors are short, IDF-dominant, and afford no co-occurring expansion competitors.

This is a defensible result for v0.2: the conservative default (`query_expansion_min_dominance=None`) protects realistic preference statements ("tabs over spaces", "rebase over merge") substantially more than terse ones ("Vim"). Workloads dominated by multi-token anchors should keep PRF×SP off; single-token-anchor workloads can experiment with it. A v0.3 candidate is a pre-PRF anchor-extraction step that filters expansion candidates by IDF rank — predicted to attenuate the $k=2,3$ regression — but is out of v0.2 scope. Full numerical detail in SCALE_REPORT.md §D15c-mech-2.

A.4.15-profile Hotspot characterization across PRF×SP arms (cProfile, $n=30$ k)

A NEXT.md-staged claim that PRF "doubled recall p50" was traced to a noisy single-shot microbench. We re-measured under controlled conditions (scripts/profile_recall_prf_sp.py, n=30 000 memories, q=200 paired queries, seed=42, embed=HashTrigram-256, vector_weight=0.3, OMP_NUM_THREADS=MKL_NUM_THREADS=1; per-arm cProfile, wall-clock ingest, per-query timer for latency to exclude profiler overhead).

arm	p50 (ms)	p95	p99	mean
baseline	40.94	66.17	85.94	46.23
prf	40.08	61.44	80.70	44.78
sp	40.38	62.54	85.92	45.10
both	46.85	82.18	86.09	49.02

PRF and share_prior alone are within sampling noise of baseline (p50 deltas ≤ 1 ms; p95 deltas ≤ 5 ms, both directions); the only arm with real overhead is **both** (+14 % p50, +24 % p95). cProfile hotspots show sqlite3.Connection.execute accounts for ~ 73 % of recall cum time across *all* arms — SQLite is the floor, not PRF. PRF doubles engine.search call count (200 $\rightarrow$ 400) but adds only ~ 6 % to engine.search cum time because the second pass hits warm pages. The v0.3 candidate-pool prune is justified for the **both** arm only; PRF-only and SP-only have no latency caveat. Full hotspot table in SCALE_REPORT.md §A.4.15-profile.

A.4.15-profile First-pass rerank-skip lever (both arm only)

The §A.4.15-profile hotspot pointed at the second-pass rerank inside the **both** arm as the only non-floor cost above baseline. We added a runtime lever, RetrievalConfig.query_expansion_skip_rerank_first_pass (default OFF), that lets the first PRF pass return raw candidates without rerank — the second pass still reranks the merged pool. Bit-correctness is preserved when the lever is OFF (full unit suite green at b5a85b7).

Re-running the §A.4.15-profile harness (n=30 000, q=200 paired, seed=42, vw=0.3, anchor_share_max=0.5, min_dominance=0.3, OMP=MKL=1) on the **both** arm only:

arm	p50 (ms)	p95	p99	mean
both	60.73	102.71	120.79	70.92
both_skip	60.53	101.18	113.62	70.10
Δ	-0.3 %	-1.5 %	-5.9 %	-1.2 %

The decision rule pre-registered in NEXT.md was: flip default ON only if p95 cuts ≥ 10 %. The observed -1.5 % p95 / -5.9 % p99 is well inside sampling noise on a 200-query paired sample and does not clear the bar. **Default stays OFF**. The lever ships as a runtime knob for users who need the p99 tail trimmed at the cost of a small recall asymmetry between first and second pass. JSON: bench/results/profile_skip_first_rerank.json.

A.4.15g IDF-rarity filter on PRF candidates (falsified)

The §A.4.15-profile result removed the latency objection to PRF-only; the only remaining barrier to making PRF default-on is the §D15c multi-token-anchor regression, where $\Delta h@1(\text{gated_pref} - \text{baseline})$ falls to roughly -9 pp at answer_anchor_tokens=3 on the synthetic preference corpus. We tested whether the regression is mediated by *low-IDF* (corpus-common) expansion candidates: PRF appends the most frequent novel entities from the top-K pool, but if those entities are themselves common across the active corpus, BM25 scores dilute toward distractors and away from the answer anchor.

Implementation. RetrievalConfig.query_expansion_idf_min_rarity: float | None = None (default OFF). When set, expand_query() drops candidate entities whose corpus rarity = $1 - df / N$ (where df is the number of active+fading FTS-indexed memories matching the entity) falls below the threshold, *before* truncating to max_entities. The rarity lookup is built lazily by RetrievalEngine._build_prf_rarity_lookup() against the store's FTS5 index and is memoized per PRF expansion. Lenient on lookup errors (treats them as rarity = 0.0, i.e. filter the candidate). Six unit tests cover the inert-when-None, drops-low-rarity, can-empty-result, lenient-on-lookup-exception, end-to-end smoke, and engine rarity-correctness paths.

Test. Single-seed paired sweep (n=240 facts, 2 400 memories, 240 queries, k=10, answer_anchor_tokens=3, the §D15c-mech-2 worst point) with idf_min_rarity $\in \{\text{None}, 0.0, 0.3, 0.5, 0.7, 0.9\}$. Same baseline once; gated_pref re-run per threshold.

idf_min_rarity	base h@1	gated h@1	$\Delta h@1$	$\pm SE$
None	0.2333	0.1417	-0.0917	0.0186
0.0	0.2333	0.1417	-0.0917	0.0186
0.3	0.2333	0.1417	-0.0917	0.0186
0.5	0.2333	0.1417	-0.0917	0.0186
0.7	0.2333	0.1417	-0.0917	0.0186
0.9	0.2333	0.1417	-0.0917	0.0186

Reading. $\Delta h@1$ is **identically -9.17 pp at every IDF threshold from 0 through 0.9**, despite the filter demonstrably engaging (a sanity trace shows idf=None chooses [pr, deferred] while idf=0.9 chooses [deferred] for the same query). The IDF-rarity hypothesis is **falsified** for this corpus: removing the corpus-common expansion term doesn't recover any hit@1; the regression survives even when expansion contains *only* the rarest available entity.

Mechanism implication. Combined with §D15c-mech (hard-distractor density falsified), §D15c-mech-2 (multi-token-anchor inverted), and now §A.4.15g (IDF-rarity filter inert), the failure mode of PRF on this corpus is **not** mediated by which entities are appended. The remaining live hypotheses are: (i) the appended *positions* in the BM25 query disturb proximity/saturation effects regardless of token choice, or (ii) the share_prior/PRF interaction with the single-session-preference question template induces a

query-side distribution shift that surfaces only in the synthetic schema. v0.3 default for `query_expansion_min_dominance` stays None (off), and `query_expansion_idf_min_rarity` ships as off-by-default infrastructure. Code: `evals/synth_pref_idf_rarity_sweep.py`.

A.4.15h Placebo query-expansion sweep (length-dilution falsified)

Hypothesis (i) from §A.4.15g closes here. We swap PRF's mined entities for content-controlled placebos and measure paired $\Delta h@1$ against the same baseline corpus (n=240 facts, anchor_tokens=3, seed=42, k=10).

Arm	$\Delta h@1$	SE	Notes
prf_real (gated_pref)	-0.0917	0.0186	Reproduces §D15c-mech-2 worst point.
placebo_stopword ("the for and")	0.0000	0.0000	Stopwords filtered by FTS5 — degenerate.
placebo_high_df (top-3 corpus DF: "on","to","for")	0.0000	0.0000	Same — FTS5 stopword filter.
placebo_low_df (3 random df=1 tokens)	-0.0250	0.0101	Mild regression.
placebo_off_topic_entity (3 random other-query anchors)	-0.1083	0.0336	Matches prf_real magnitude.

The two stopword/high-DF placebos are uninformative — FTS5 strips the appended tokens before BM25 scoring, so the query is byte-for-byte equivalent to baseline ($\Delta h@1 = 0$ to 4 decimals across all 240 paired trials). What survives the tokenizer is what matters.

The decisive comparison is `prf_real` vs `placebo_off_topic_entity`: both regress $h@1$ by $\approx 9\text{--}11$ pp, with overlapping SEs. PRF's hand-mined entities perform statistically indistinguishably from appending three *random other-query anchors* to the query. This is inconsistent with hypothesis (i) — pure BM25 length-dilution should have shown up in the (admittedly degenerate) stopword arms; the real cost is paid only when the appended tokens are themselves fact-anchor-shaped (3-token preference phrases like "rebase over merge"). Because the synthetic preference corpus has dense cross-fact anchor reuse (16 prefs $\times$ 16 users $\times$ 10 tasks $\rightarrow$ many queries share the same anchor set), PRF systematically pulls in *another fact's* anchor and BM25 promotes that distractor to rank 0.

Mechanism conclusion. Across §D15c-mech (hard-distractor density, falsified), §D15c-mech-2 (multi-token-anchor, inverted), §A.4.15g (IDF rarity, inert), and §A.4.15h (length-dilution, falsified), the PRF regression on this corpus is now characterised as **cross-fact entity confusion under shared-anchor density** — a property of the corpus, not of the retriever. This bounds the failure mode: PRF should not regress on real LongMemEval-style corpora where anchors are not mass-recycled across facts, which is consistent with §D15's +3.33 pp lift on LongMemEval. v0.3 default stays None (off); v0.3 should add an *anchor-sharing diagnostic* to the type-allow gate (skip PRF when the answer-anchor token set has high cross-document repetition in the first-pass top-K). Code: `evals/synth_pref_placebo_expansion_sweep.py`.

A.4.15i Anchor-share diagnostic gate (v0.3 prototype, default OFF)

§A.4.15h's mechanism conclusion (cross-fact entity confusion under shared-anchor density) makes a falsifiable prediction: a runtime detector that short-circuits PRF when the first-pass top-K is saturated by one anchor entity should (a) cure the synth-pref regression and (b) be inert on diverse-pool corpora.

Knob. `RetrievalConfig.query_expansion_anchor_share_max`: float | None, default None (OFF). When set AND `query_expansion_min_dominance` is set, computes `share = (count of dominant candidate entity in top-K) / (total candidate entity count)`. When `share > anchor_share_max`, the un-expanded first pass is returned.

Synth-pref result (n=240, seed=42, anchor_tokens=3, k=10):

anchor_share_max	$\Delta h@1$ vs baseline	$\pm$ SE
None (raw PRF)	-0.0917	0.0186
0.7	-0.0250	0.0101
0.5	-0.0042	0.0042
0.4	0.0000	0.0000
0.3	0.0000	0.0000

Threshold 0.4 fully cures the regression at SE=0 (gate fires on every saturated query, identically reproducing baseline behaviour). $h@10$ unchanged at 0.8875 across all gated points.

Multi-entity-hard inertness check (n_facts=500, seed=42): for every threshold $\in \{0.7, 0.5, 0.4, 0.3\}$, the gate's Δ vs prf-only is exactly 0.000 at $h@1$ / $h@5$ / $h@10$. On the diverse non-saturated corpus the gate never fires.

Verdict. Mechanism prediction §A.4.15h confirmed. The gate is a clean diagnostic instrument: it kills the saturated-corpus regression without touching diverse-pool behaviour. v0.2 default stays OFF (regression-safe; ships behind the new knob); v0.3 candidate default `anchor_share_max = 0.5` pending LongMemEval re-evaluation. Code: `evals/synth_pref_anchor_share_sweep.py`, `evals/multi_entity_hard_anchor_share.py`. SCALE_REPORT.md §D15d has the full numerical detail.

A.4.15j Anchor-share gate: LongMemEval inertness

§A.4.15i shipped the anchor-share gate behind a runtime knob with v0.3 default 0.5 (synth-pref pareto sweet-spot). Before flipping that default, we confirm LongMemEval-S inertness — the *negative* prediction the §A.4.15h mechanism makes on a corpus where anchors are not mass-recycled across facts.

LongMemEval-S full 500-question evaluation, paired bootstrap vs the saved baseline run, k=10:

arm	overall h@1	$\Delta h@1$ vs baseline	CI 95%	pref-slice h@1
baseline	0.8100	—	—	0.3667
prf (raw, gate=None)	0.7700	-0.0400	[-0.0620, -0.0180]	0.4333
prf + anchor_share_max=0.7	0.7700	-0.0400	[-0.0620, -0.0180]	0.4333
prf + anchor_share_max=0.5	0.7700	-0.0400	[-0.0620, -0.0180]	0.4333
prf + anchor_share_max=0.4	0.7700	-0.0400	[-0.0620, -0.0180]	0.4333

Reading. The gate is *bit-identically* inert on LongMemEval at all three thresholds — every per-instance hit@1 / hit@k matches raw PRF. The gate fires on **zero** LME queries, including at the synth-pref SE=0 floor (0.4). This is the falsifiable inertness §A.4.15h predicted: LongMemEval anchors are not mass-recycled across facts, so first-pass top-K is never saturated by a single dominant entity.

Implication. Flipping the v0.3 default to `anchor_share_max = 0.5` is LongMemEval-safe at the bit level. The single-session-preference slice keeps its +6.67 pp PRF lift (0.3667 → 0.4333) under the gate. The synth-pref -9.17 pp regression is fully cured at threshold 0.4 (§A.4.15i) without touching any LME metric. v0.3 candidate-default tuple: `min_dominance = 0.3, anchor_share_max = 0.5, type_allow = None`. Code: `scripts/run_d15d_lme_sweep.sh`, `bench/results/lme_full500_d15d_anchor_sweep.json`. SCALE_REPORT.md §D15d-LME has the full per-type breakdown.

A.4.15k v0.3 default flip — PRF + anchor-share gate ON by default (SUPERSEDED by §A.4.15l)

Status (2026-05-24). The default-ON proposal below was retracted in commit 99eabad after the real LoCoMo benchmark (§A.4.15l) showed a CI-clean -2.7 pp h@1 regression. The shipped v0.3 default for `query_expansion_min_dominance` is **None (off)**. This section is retained for historical context; §A.4.15l is the operative ship decision.

The v0.3 ship-defaults are now:

- `RetrievalConfig.query_expansion_min_dominance = 0.3` (was None)
- `RetrievalConfig.query_expansion_anchor_share_max = 0.5` (was None)

Empirical justification:

1. **LongMemEval-S inertness (§A.4.15j / §D15d-LME).** Anchor-share gate at 0.5 is bit-identically inert across the full 500-question LME-S slice; every per-instance hit@1/hit@k matches raw PRF. No regression risk on real-data session retrieval.
2. **Synth-pref cure at SE=0 (§D15d / §A.4.15i).** Anchor-share gate cures the §D15c synth-pref regression: $\Delta h@1 = 0.0 \pm 0.0$ at threshold 0.5.
3. **Pref-slice lift preserved (§A.4.15j).** Under the gate, pref-slice h@1 remains 0.4333 (vs. baseline 0.3667, +6.67 pp on the slice that PRF was designed to help), confirming the gate prunes pathological queries without cannibalizing the working PRF lift.
4. **PRF operating point (§A.4.7).** `min_dominance = 0.3` is the operating point with the tightest CI envelope ($\alpha=0.05$, $d=0.3$, $pool=20$).

`type_allow` stays **None** — typed-PRF was a scoped null on LongMemEval (§A.4.12) so the simpler heuristic-only stack is the default.

The gate is a *strict subset* of vanilla PRF behavior: every query the gate fires on, vanilla PRF would have run; the gate only short-circuits queries whose first-pass top-K is anchor-saturated. Default-OFF use sites that deliberately wanted untouched behavior must now set `query_expansion_min_dominance = None` explicitly.

A.4.15l Real LoCoMo (snap-research/locomo) — PRF×share_prior regression

The §A.4.15k default-ON proposal was retracted in commit 99eabad after running the first *real* LoCoMo benchmark (replacing the synthetic placeholder of SCALE_REPORT.md §D7). The shipped default for `RetrievalConfig.query_expansion_min_dominance` is **None (off)**.

Setup. `data/locomo/locomo10.json` (10 conversations, 1,978 scored QA across 5 categories), BM25-only embedder (`--embedder None`), $k=10$, adapter `evals/locomo_adapter.py`. Treatment knobs at the §A.4.7 operating point (`qe_dominance=0.3`, `sp_alpha=0.1`, `sp_pool=20`). Repo at commit 73d1d4d. Paired bootstrap: $B=10\,000$, $\alpha=0.05$.

Headline.

arm	h@1	$\Delta h@1$ vs baseline	CI95	h@k	$\Delta h@k$ vs baseline	CI95
baseline	0.5394	—	—	0.7988	—	—
prf	0.5126	-0.0268	[-0.043, -0.011]*	0.7594	-0.0394	[-0.049, -0.030]*
share_prior	0.5394	+0.0000	[+0.000, +0.000]	0.7958	-0.0030	[-0.007, +0.000]
both	0.5121	-0.0273	[-0.044, -0.011]*	0.7528	-0.0460	[-0.056, -0.036]*

* paired-bootstrap CI excludes 0.

Reading. PRF strictly regresses on real LoCoMo session-level retrieval ($\Delta h@1 = -2.7$ pp, CI-clean negative). Share_prior is operationally inert ($\Delta h@1 = 0.000$ exactly), confirming the §A.4.6 finding that LoCoMo's flat sibling distribution leaves no co-promoted schemas for the prior-sharing knob to fire on. The stack **both** $\approx$ PRF alone, since SP contributes nothing. PRF/both also pay a +76% read p50 latency penalty (8.5 → 15 ms) without any recall lift to amortize it.

Implication. The §A.4.15k default-ON ship is corpus-shape-sensitive: PRF×SP delivers on synthetic preference fixtures (§A.4.7) and the LongMemEval-S preference slice (§A.4.15j) but *inverts sign* on real LoCoMo. The conservative ship is therefore default-OFF (`query_expansion_min_dominance=None`), with the operating point

documented as a runtime opt-in for workloads that match the synthetic-pref / preference-tuning corpus shape. The §A.4.15k "v0.3 default flip" claim is hereby corrected — the v0.3 default is **OFF**, and the gate sections (§A.4.15i–§A.4.15j) document the conditions under which a deployment can flip it on with confidence.

Artifacts: `bench/results/locomo_real_n10_{baseline,prf,sp,both}.json`. Full per-category and bootstrap details in `SCALE_REPORT.md` §D19.

A.4.15m Embedder-invariance check — ST-MiniLM on real LoCoMo

§A.4.15l ran the four PRF×share_prior arms under BM25-only retrieval. The remaining open question for the paper draft was whether the regression is a lexical-retrieval artefact: would a dense embedder, which is robust to paraphrase, recover the synthetic-preference lift?

Setup. Identical 996-QA real LoCoMo slice and identical four arms, but with `--embedder st` (sentence-transformers all-MiniLM-L6-v2, 384-dim, hybrid retrieval at the default `vector_weight=0.3`). Position-paired bootstrap, B=10000, $\alpha=0.05$, seed=42.

Result (all four arms).

arm	h@1	h@k	$\Delta h@1$ vs base
baseline	0.5311	0.8032	—
prf	0.4869	0.7771	−0.0442 [−0.067, −0.021] *
share_prior	0.5311	0.8012	+0.0000 [0.000, 0.000] ns
both	0.5000	0.7731	−0.0311 [−0.053, −0.010] *

Reading.

1. The PRF regression is

embedder-invariant in sign and amplified in magnitude

under a dense embedder (BM25 −0.0268 → ST −0.0442). The paraphrase-robustness hypothesis is rejected: dense retrieval does not rescue PRF on this corpus.

2. share_prior remains operationally inert across embedders (degenerate CI [0,0] in both runs). This closes a residual line in the §A.4.15k retraction — share_prior is not "waiting for a dense embedder" to activate.

3. Per-category sign pattern matches §A.4.15l: cat 4 (n=418) and cat 2 (n=156) are the significantly-negative categories under both embedders. Mechanism is corpus-shape, not retrieval-stack-specific.

The §A.4.15k default-ON proposal stays retracted; the operative ship is default-OFF (`query_expansion_min_dominance=None`, 99eabad). The §A.4.15i–§A.4.15j gates remain the documented opt-in path for deployments whose corpus shape resembles the synthetic-preference / LongMemEval-S distribution.

Artifacts: `bench/results/locomo_real_n10_st_{baseline,prf,share_prior,both}.json`. Full breakdown in `SCALE_REPORT.md` §D20.

A.4.15n Type-purity gate ablation — hypothesis rejected

§A.4.15m localized the PRF regression: corpus-shape-mediated, embedder- invariant in sign, with cat 4 (knowledge-update; n=841 of 1986) contributing the largest absolute loss. A natural conjecture is that PRF damages cat 4 because the heuristic NER backend produces

type-salad

expansions (a PERSON + GPE + ORG concatenated into one query) and that gating expansion on entity-type purity would suppress exactly the failure mode. We test this with a real-NER replication.

Setup. BM25-only retrieval on n=1986 real LoCoMo questions (10 conversations). Three arms vs the §A.4.15l baseline:

- `prf` (heuristic NER, no purity gate) — the §A.4.15l arm,
- `prf_spacy` (`--qe-backend spacy_sm`, real per-entity labels; the gate is inert because we do not threshold purity here),
- `prf_spacy_tp50` (spaCy backend + `--qe-type-purity-min 0.5`, i.e. fire expansion only when the dominant NER label is at least 50% of the first-pass entity-occurrence pool).

Paired-bootstrap CIs (B=10000, $\alpha=0.05$).

arm	dh@1 vs baseline	95% CI	sig
prf (heuristic)	-0.0268	[−0.0435, −0.0106]	*
prf_spacy	-0.0212	[−0.0303, −0.0126]	*
prf_spacy_tp50	-0.0445	[−0.0571, −0.0324]	*

Per-category for `prf_spacy_tp50` vs baseline:

cat	n	dh@1	95% CI	sig
1	281	-0.0356	[−0.0747, +0.0036]	ns
2	321	-0.0249	[−0.0467, −0.0062]	*
3	89	-0.0112	[−0.0674, +0.0449]	ns

cat	n	dh@1	95% CI	sig
4	841	-0.0523	[-0.0702, -0.0345]	*
5	446	-0.0561	[-0.0874, -0.0269]	*

Verdict. Hypothesis rejected. The cat-4 regression worsens 3.4x under the gate (-0.0155 ungated -> -0.0523 gated), and the aggregate worsens (-0.0212 -> -0.0445). Mechanism: gate-on policy passes expansion exactly when the first-pass is type-coherent — but those are precisely the corpora in which gold and distractor passages share dominant entities, and the discriminator is a non-entity verb / temporal phrase. The 3-token entity expansion crowds out those discriminators in BM25's bounded-budget scoring, harming retrieval more than the no-expansion default.

Implication. A correct PRF gate would need to condition on *non-entity discriminator presence in the original query*, not on first-pass entity-type purity. We do not pursue this further; default-OFF (`query_expansion_min_dominance=None`, 99eabad) remains the only ship that survives every sliced replication (§A.4.15l BM25 / §A.4.15m ST / §A.4.15n type-purity).

Artifacts: `bench/results/locomo_real_n10_prf_spacy{,_tp50}.json`. Full breakdown and CI script in `SCALE_REPORT.md` §D21.

A.4.15o Non-entity-discriminator PRF gate (offline replay) — hypothesis rejected

Setting. §A.4.15n closed the type-purity dimension. The §A.4.15n verdict suggested the *correct* PRF gate predicate is non-entity discriminator presence in the original query: fire PRF only when the query carries a verb or temporal token outside the named-entity span. We test this offline on the §A.4.15n paired stream (n=1978, BM25-only) without re-running any arm: per-query, the gate's "fire" decision chooses the `prf_spacy` outcome; "no fire" reuses the baseline outcome. Three predicates: `verb_only`, `temp_only`, `verb_or_temp` (union).

Fire-rates. spaCy `en_core_web_sm` POS+NER over 1978 questions: 83.1% (`verb_only`), 28.6% (`temp_only`), 85.8% (`verb_or_temp`). Cat-conditional fire-rate is 78–95% across all five LoCoMo categories under `verb_or_temp` — the predicate does not separate the regression slice (cat 4) from the safe slices.

Δhit@1 vs baseline (paired bootstrap, B=10000, α=0.05).

arm	Δhit@1	95% CI	sig
<code>prf_spacy</code> (always-on)	-0.0212	[-0.0303, -0.0126]	*
<code>gate=verb_only</code>	-0.0167	[-0.0248, -0.0091]	*
<code>gate=temp_only</code>	-0.0066	[-0.0111, -0.0025]	*
<code>gate=verb_or_temp</code>	-0.0177	[-0.0263, -0.0096]	*

Per-fired-query damage. Damage rate per fired query is $0.0212/1.000 = 0.0212$ (always-on), $0.0167/0.831 = 0.0201$ (`verb_only`), $0.0066/0.286 = 0.0231$ (`temp_only`), $0.0177/0.858 = 0.0206$ (`verb_or_temp`). All four rates lie within Monte-Carlo noise of each other. The gate does not change per-query damage; it only attenuates by lowering the fire-rate proportionally.

Verdict. Hypothesis rejected. The non-entity-discriminator predicate does not separate harmful from harmless PRF expansions. PRF damage on LoCoMo BM25 is uniformly distributed in query-shape space at the 1978-question scale.

Closed PRF-gate arc. This is the fourth consecutive failed PRF gate at the LongMemEval / LoCoMo scale: anchor-share (§A.4.15i, n=240), LME inertness (§A.4.15j), type-purity (§A.4.15n, n=1978), and non-entity discriminator (§A.4.15o, n=1978). We retire the cheap-gate direction. PRF revival, if pursued, must replace the first-pass-token heuristic with either RM3-style scored expansion or a paraphrase-based candidate generator. Default-OFF (`query_expansion_min_dominance = None`, 99eabad) ships in v0.3.

Artifacts: `bench/results/locomo_real_n10_baseline.json`, `bench/results/locomo_real_n10_prf_spacy.json`, `scripts/locomo_discriminator_gate_analysis.py`. Full breakdown in `SCALE_REPORT.md` §D22.

A.4.16 BGE-large-en-v1.5 — third embedder tier on the entity-collision protocol

Question. A natural reviewer ask is "why only two embedders?" The two-axis result of §4.3 (HashTrigram-256 vs. ST MiniLM-384, 384-d) could plausibly be an artifact of the dense-side capacity ceiling: a larger encoder might erase the lexical-tag advantage and turn every cell uniformly positive. To falsify this, we re-ran the entity-collision protocol with BAAI/bge-large-en-v1.5 (1024-d) under the v0.2 RetrievalConfig defaults (`vector_weight = 0.3`, `paraphrase_memory = false`, n=32 entities, $K \in \{1,2,4,8,16\}$, paired bootstrap with 5000 resamples, seed=42).

Wiring (commit 31a6168): `evals/ablation.py::_make_embedder` gained a `bge_large` choice that delegates to `SentenceTransformerProvider("BAAI/bge-large-en-v1.5")`; `evals/entity_collision_sweep.py` extended its `--embed` argparse choices accordingly. Three unit tests pin dim=1024 and a distinct embedding from MiniLM (`tests/unit/test_make_embedder_bge.py`).

A.4.16.1 BGE-large vs. MiniLM, paired per-query Δhit@1 (95% CIs)

Source: `bench/results/ec_bge_vs_minilm_ci.json`, generated by `scripts/ec_bge_vs_minilm_ci.py` (paired across the per-query records in the underlying sweep JSONs by query text and collision degree). Δ = BGE vector-fusion hit@1 minus ST MiniLM vector-fusion hit@1; "sig" = paired bootstrap 95% CI excludes 0.

Lexical-discriminator tags (technical, tool, service):

tag	K	n_paired	ST hit@1	BGE hit@1	Δ (BGE-ST)	95% paired CI	sig
technical	4	128	0.383	0.352	-0.031	[-0.086, +0.023]	
technical	8	256	0.316	0.199	-0.117	[-0.164, -0.070]	✓
technical	16	512	0.168	0.105	-0.062	[-0.090, -0.035]	✓
tool	4	128	0.391	0.320	-0.070	[-0.125, -0.016]	✓
tool	8	256	0.242	0.180	-0.062	[-0.102, -0.023]	✓
tool	16	512	0.131	0.103	-0.027	[-0.047, -0.006]	✓
service	4	128	0.328	0.367	+0.039	[-0.016, +0.094]	
service	8	256	0.227	0.266	+0.039	[-0.004, +0.082]	
service	16	512	0.166	0.197	+0.031	[+0.006, +0.057]	✓

Intent-style tags (project, preference):

tag	K	n_paired	ST hit@1	BGE hit@1	Δ (BGE-ST)	95% paired CI	sig
project	2	64	0.500	0.609	+0.109	[+0.031, +0.203]	✓
project	4	128	0.312	0.453	+0.141	[+0.070, +0.211]	✓
project	8	256	0.164	0.309	+0.144	[+0.094, +0.195]	✓
project	16	512	0.082	0.166	+0.084	[+0.051, +0.115]	✓
preference	2	64	0.609	0.625	+0.016	[-0.047, +0.078]	
preference	4	128	0.430	0.445	+0.016	[-0.047, +0.078]	
preference	8	256	0.289	0.266	-0.023	[-0.078, +0.027]	
preference	16	512	0.184	0.199	+0.016	[-0.018, +0.049]	

A.4.16.2 Verdict — bigger encoder is not uniformly better

The paired CIs reject the encoder-capacity hypothesis cleanly:

- **technical and tool** (lexical-discriminator regime, where surface-form proper-noun retrieval dominates): BGE *significantly loses* at every $K \in \{8, 16\}$ on **technical** and at every $K \in \{4, 8, 16\}$ on **tool**. The **technical** $K=8$ deficit is the largest cell in the panel: -11.7pp, [-16.4, -7.0].
- **project** (intent-style, where MiniLM was already weakest): BGE *significantly wins* at every $K \in \{2, 4, 8, 16\}$. The $K=8$ lift +14.4pp [+9.4, +19.5] is exactly the symmetric mirror of the **technical** loss.
- **service** $K=16$ is the only lexical-tag cell where BGE wins, and the lift is small (+3.1pp) and right at the CI boundary. **preference** is null at every K .

A parsimonious mechanistic reading: BGE's contrastive pretraining emphasises semantic paraphrase and de-emphasises surface-form lexical discriminators; on a closed-vocabulary, proper-noun-answer regime this is a liability, not an asset. We do not claim a causal proof here — the point is the falsification: encoder size alone is not the binding constraint. The §4.3 two-axis interpretation **survives a 2.7 $\times$ -parameter encoder swap, and in fact strengthens** — bigger model shifts the lift along the two axes rather than uniformly raising it.

Operational implication for v0.2 defaults. The v0.2 ship is MiniLM-384 with `vector_weight = 0.3`. We retain MiniLM as the default: BGE's lexical-tag deficit and intent-tag surplus roughly offset across the five-tag mean, BGE costs $\sim 3\times$ MiniLM per-query latency, and a workload-targeted embedder swap is exactly the kind of decision the §4.3 two-axis interpretation is designed to inform — not a property of the default.

Artifacts: `bench/results/ec_bge_large_{service,preference,project,technical,tool}_n32_K16{, _ci}.json`,
`bench/results/ec_bge_vs_minilm_ci.json`, `scripts/run_bge_sweeps.sh` (driver), `scripts/ec_bge_vs_minilm_ci.py` (paired CI generator),
`tests/unit/test_make_embedder_bge.py`.

A.4.16.3 BGE-large on natural data — does the synthetic lexical/intent split replicate?

Question. A.4.16.1–.2 falsified the encoder-capacity hypothesis on a *synthetic* entity-collision protocol. A reviewer can reasonably ask whether the same finding survives on a real benchmark, where natural language mixes lexical and intent regimes within every query and the type taxonomy is dataset-defined rather than tag-controlled. We re-ran the LongMemEval-S baseline arm under BAAI/bge-large-en-v1.5 on the **full n=500 LongMemEval-S panel** and paired per-question_id against the §4.5 full-500 default-embedder baseline (`bench/results/lme_full500_k10_baseline.json`). All 500 BGE question_ids matched a default-baseline record exactly; the comparison is fully paired across all six question_type cells.

Source: `bench/results/lme_n500_bge_large_baseline.json` (BGE arm), matched per-question_id against the §4.5 default-baseline file via `scripts/lme_bge_vs_minilm_n500_paired_ci.py` (10 000-resample paired bootstrap, seed=42, output `bench/results/lme_n500_bge_vs_default_ci.json`).

cell	n	default hit@1	BGE hit@1	Δ (BGE - default)	95% paired CI	sig
overall	500	0.810	0.868	+0.058	[+0.032, +0.086]	✓
single-session-user	70	0.914	0.900	-0.014	[-0.071, +0.029]	

cell	n	default hit@1	BGE hit@1	Δ (BGE – default)	95% paired CI	sig
single-session-assistant	56	0.821	0.893	+0.071	[+0.018, +0.143]	✓
single-session-preference	30	0.367	0.533	+0.167	[0.000, +0.333]	
multi-session	133	0.805	0.887	+0.083	[+0.023, +0.143]	✓
temporal-reasoning	133	0.812	0.880	+0.068	[+0.015, +0.120]	✓
knowledge-update	78	0.885	0.897	+0.013	[0.000, +0.038]	

cell	n	default hit@10	BGE hit@10	Δ (BGE – default)	95% paired CI	sig
overall	500	0.932	0.956	+0.024	[+0.010, +0.040]	✓
single-session-user	70	0.957	0.957	0.000	[−0.043, +0.043]	
single-session-assistant	56	0.911	0.929	+0.018	[0.000, +0.054]	
single-session-preference	30	0.833	0.967	+0.133	[+0.033, +0.267]	✓
multi-session	133	0.962	0.977	+0.015	[−0.015, +0.045]	
temporal-reasoning	133	0.925	0.962	+0.038	[+0.008, +0.075]	✓
knowledge-update	78	0.923	0.923	0.000	[0.000, 0.000]	

Verdict — significant headline lift, with a structured per-type pattern. The full-panel paired CI lands well clear of zero on overall hit@1 (+5.8 pp [+3.2, +8.6]) and overall hit@10 (+2.4 pp [+1.0, +4.0]). The gain decomposes structurally: it is concentrated on the question_types where the dense side has the most paraphrase work to do — multi-session (+8.3 pp hit@1), temporal-reasoning (+6.8 pp hit@1), single-session-assistant (+7.1 pp hit@1) — and is null on the cells where lexical overlap already dominates retrieval (single-session-user, knowledge-update). The single-session-preference cell — the dominant residual error mode in §4.5 (default hit@1 = 0.367) — moves +16.7 pp on hit@1 (CI floor at exactly 0, n=30 underpowered) and is significant on hit@10 (+13.3 pp [+3.3, +26.7]).

This **reverses the n=100 preliminary** reported in earlier drafts of this subsection, where every paired CI touched zero. The n=100 result was a power story, not a real null: at n=100, sampling only the single-session-user (70) and multi-session (30) cells, the cells where BGE actually pays — temporal-reasoning, single-session-assistant, single-session-preference — were entirely outside the panel, and the multi-session signal (+8.3 pp at n=500) sat just under the n=30 noise floor in the preliminary. We retain the synthetic-data §A.4.16.2 verdict (entity-collision tag-conditional, not headline) as the *synthetic* finding, but on real LongMemEval-S the encoder upgrade *does* move the headline.

The §A.4.16.2 tag-conditional pattern (technical/tool regress, project improves) does not appear here in opposite-sign form — every significant cell in the natural-data panel moves *up*. The synthetic regression cells map to a tag taxonomy LongMemEval does not directly expose, so we flag this as a non-mapping rather than a contradiction: on the question_types LongMemEval enumerates, the dense-side gain is either positive or null, never negative.

Latency cost. BGE-large at n=500 spent ≈ 2 h 57 min wall on M4 Pro MPS (ENGRAM_ST_DEVICE=mps, ENGRAM_ST_BATCH=256, fp32) — 21.5 s/instance ingest, vs the default hashtigram-256 baseline at ≈ 4 m 25 s (524 ms/instance ingest) and the prior n=100 CPU run at 150.8 s/instance (a 7 $\times$ MPS speedup over CPU). Per-query recall is still $\approx 11\times$ slower than the default. The headline gain is real and significant; the operational ratio for a default-flip is still unfavorable on commodity CPU, but on accelerator hardware (MPS / CUDA) BGE-large becomes a defensible workload-targeted upgrade rather than a falsified hypothesis.

Default-embedder decision. The v0.2 ship default remains MiniLM-384 + vector_weight = 0.3 on the basis of (i) per-query and per-ingest latency on commodity CPU hosts (the v0.2 deployment target), (ii) the §4.3 axis-1/axis-2 framing — fusion weight is the binding control, encoder capacity is a secondary lever — and (iii) reproducibility/portability of the default config. The +5.8 pp hit@1 / +2.4 pp hit@10 BGE result is documented here as a **workload-targeted upgrade path**, not as a recommendation for the default. Operators with accelerator hardware and an a-priori workload mix concentrated on multi-session / temporal-reasoning / single-session-assistant queries should expect a real lift from the swap; users on CPU hosts with mixed workloads should not. The §4.3 two-axis interpretation thus survives intact: encoder capacity is a real second axis, but it is the second axis, not the first.

Artifacts: bench/results/lme_n500_bge_large_baseline.json (BGE arm n=500 raw), bench/results/lme_n500_bge_vs_default_ci.json (paired CI), scripts/lme_bge_vs_minilm_n500_paired_ci.py (CI generator). The earlier n=100 preliminary — bench/results/lme_n100_k10_baseline_bge.json and bench/results/lme_n100_bge_vs_default_ci.json — is retained on disk for audit but superseded by the n=500 panel.

A.4.16.4 RM3 baseline arm (AUDIT-D) — PRF cannot substitute for a dense encoder

Question. §5.6 already rejects *heuristic* top-k query expansion as a recovery mechanism for the BM25-only intent-tag null. A reviewer might reasonably ask whether a *scored* relevance-model expansion in the Lavrenko-Croft family — RM3 (Lavrenko & Croft, 2001), the canonical IR baseline — would change the verdict. RM3 mixes a learned expansion-term distribution against the original query terms via a λ interpolation, weighting expansion terms by their mean $P(\text{term} | \text{top-}k \text{ documents})$ under a uniform document prior. We ran RM3 across the entity-collision grid, BEIR FiQA, and LongMemEval-S n=500 with Anserini-default hyperparameters (top-k=10, num-terms=10, $\lambda=0.5$, $\epsilon=0.01$) to test whether the additional scoring machinery rescues PRF.

Implementation. evals/rm3.py implements RM1 (uniform doc prior) plus RM3 mixture in pure Python (no src/engram/ modification, no GPU dependency). The two-pass dance — BM25(q) $\rightarrow$ expand $\rightarrow$ BM25(q') — runs externally to Engram core; the BEIR adapter, the LongMemEval adapter, and the entity-collision sweep each gain an RM3 arm via dedicated CLI flags (-arm rm3, --rm3, --rm3 respectively). 7 unit tests (tests/unit/test_rm3.py) cover tokenization, empty input, high-IDF term selection, expanded-query string rendering, top-k clamping, stopword filtering, and missing doc text. Test count grew 250 $\rightarrow$ 257; no regressions.

Headline result — paired CIs vs BM25 baseline.

LongMemEval-S n=500 paired by question_id, B=10000 bootstrap resamples, seed=42 (scripts/lme_rm3_vs_bm25_n500_paired_ci.py, bench/results/lme_n500_rm3_vs_bm25_ci.json):

question_type	n	$\Delta\text{hit}@1$ [95% CI]	$\Delta\text{hit}@k$ [95% CI]
overall	500	-0.014 [-0.030, +0.000]	-0.014 [-0.026, -0.004] SIG
single-session-user	70	-0.071 [-0.143, -0.014] SIG	+0.000 [exact]
multi-session	133	-0.008 [-0.030, +0.015]	-0.015 [-0.038, +0.000]
single-session-preference	30	+0.000 [exact zero]	-0.100 [-0.233, +0.000]
temporal-reasoning	133	-0.015 [-0.053, +0.023]	-0.015 [-0.038, +0.000]
knowledge-update	78	+0.013 [+0.000, +0.039]	+0.000 [+0.000, +0.000]
single-session-assistant	56	+0.000 [exact zero]	+0.000 [+0.000, +0.000]

Verdict — three findings, all sharpening rather than weakening upstream claims.

First, the `single-session-preference` cliff is RM3-invariant. $\Delta\text{hit}@1 = +0.000$ *exactly* on all 30 paired instances — RM3 returns the same wrong session as BM25 in every case. The CI is degenerate because every paired delta is zero. This confirms that the §A.4.16.3 intent-tag weakness is a structural property of the lexical retrieval channel, not a BM25-specific artifact that scored expansion could fix. **§4.8 / §5.1 conclusion strengthens.**

Second, RM3 SIG-regresses `single-session-user` $\Delta\text{hit}@1$ by 7.1 pp (CI [-14.3, -1.4] excludes zero). The same query-drift failure mode the entity-collision `technical` cell exposes at low K (-87.5 pp $\Delta\text{hit}@1$ vs BM25 at K=1, see entity-collision panel below): when the BM25 first-pass already pinpoints the right doc, expansion terms mined from the top-*k* (which include adjacent-but-wrong sessions) pull the right answer down rather than up. This is the textbook Lavrenko-Croft 2001 failure regime — high-quality first-pass retrieval makes RM3 strictly worse than BM25 alone.

Third, the recall-broadening behavior the IR literature reports for RM3 is corpus-dependent. On BEIR FiQA (n=648, 57,638-doc corpus, financial conversational queries), $\text{recall}@100$ lifts +3.5 pp vs BM25-only (0.5079 vs 0.4725) at the cost of -1.6 pp $\text{ndcg}@10$ — the classic precision/recall trade-off Lavrenko-Croft 2001 documents. On LongMemEval-S, the trade-off inverts: $\text{hit}@k$ regresses -1.4 pp SIG (CI [-2.6, -0.4]) because the BM25 first-pass already has the right session in top-10 most of the time, and expansion adds noise without adding coverage. The published RM3 wins are on retrieval workloads where BM25 *under-recalls*; agent-memory haystacks (several-hundred-session conversational logs) sit in the regime where BM25 *already-recalls* and RM3 cannot help.

Entity-collision under RM3 — refining the lexical-vs-intent axis.

Per-tag $\Delta\text{hit}@1$ vs BM25-only at the same n_entities=32, K=1..16, seed=42 protocol that drives §4.2 (bench/results/ec_rm3*_n32_K16.json):

tag	K=1	K=2	K=4	K=8	K=16
preference	+0.000	+0.000	+0.000	+0.000	+0.000
project	+0.000	-0.063	-0.039	-0.012	+0.006
technical	-0.875	-0.438	-0.148	-0.035	+0.000
service	+0.000	+0.047	+0.016	+0.008	+0.002
tool	-0.094	+0.000	+0.008	+0.004	+0.016

Two structural patterns. (i) RM3 *helps* on `service` at K=2 (+4.7 pp), where the expansion terms — concrete service nouns from the top-*k* docs — are diagnostic of the right answer; (ii) RM3 *catastrophically drifts* on `technical` at low K (-87.5 pp at K=1, -43.8 pp at K=2), where the open-vocabulary technical jargon in the top-*k* produces expansion terms that are confused with adjacent-but-wrong technical content. The lexical-vs-intent dichotomy of §4.3 is too coarse for RM3: the per-tag failure factors further by whether expansion terms are diagnostic (closed-vocab service nouns) or distracting (open-vocab technical jargon).

Comparison to the §3.1.1 latency-cost table. RM3 occupies a fourth operating point: zero model-load like HashTrigram-256 (no embedder, FTS5-only), ~1.5 ms/doc CPU ingest (faster than BGE-large by three orders of magnitude on CPU), ~259 ms query latency on FiQA because of the two-pass BM25 dance. Neither the HashTrigram-256 recovery pattern (CI-positive on `service` K=16, `tool` $K \in \{4, 8, 16\}$) nor the MiniLM-384 universal lift replicates under RM3. **RM3 is strictly dominated** by MiniLM-384 on every cell where MiniLM is CI-positive, and by BM25 alone on the lexical-anchor cells where PRF expansion drifts. Reviewer-relevant scope-defense: under our hyperparameter freeze and on our corpora, RM3 is not a viable substitute for either a 256-dim hash trigram or a learned dense encoder.

Hyperparameter sensitivity disclaimer. All RM3 cells use Anserini defaults. A targeted hyperparameter search (lower λ to de-emphasize expansion, lower num-terms to cap drift) might recover some `single-session-user` regression but cannot in principle move the `single-session-preference` cliff (paired $\Delta = +0.000$ *exactly* means both arms produce identical rankings on those queries — the expansion is selecting non-discriminating terms regardless of mixture weight). We accept the Anserini defaults as the published reference point and note that an exhaustive λ sweep is queued for v0.3.

Reproducer paths. evals/rm3.py (module), tests/unit/test_rm3.py (7 unit tests), scripts/lme_rm3_vs_bm25_n500_paired_ci.py (paired CI generator), bench/results/{ec_rm3*_n32_K16, beir_fiqa_rm3, lme_n500_rm3, lme_n500_rm3_vs_bm25_ci}.json (artifacts).

A.4.17 Schema-lifecycle invariants — the property suite that backs §3.7.4

Question. §3.7.4 states the schema-lifecycle reducer as a pure event-fold and asserts a five-edge DAG plus five textual invariants. §5.3 then claims those invariants are *enforced by Hypothesis property tests* rather than asserted by spot-check. This subsection is the audit trail for that claim — a one-page index of the property surface and the classes of bug each property would have caught had we written the reducer naively.

The §B research thread (TODO-RESEARCH.md) opened with six prose invariants for the lifecycle. Five map onto the DAG-plus-fold contract of `src/engram/consolidation/schema_lifecycle.py`; the sixth — *schema writes serialize against extraction writes* — is a concurrency claim about the persistence layer (`src/engram/store/buffer.py` already takes an exclusive `fcntl.flock` on append) and is fuzzed at the buffer level. We catalogue the gates here so a reviewer can reproduce the chain of invariants → tests → bugs-prevented without grepping the test directory.

A.4.17.1 Invariant ↔ test ↔ bug-class table

§B invariant (TODO-RESEARCH.md)	Property gate(s)	Bug-class caught
#1 <i>Status is monotone modulo recovery</i> (only the four DAG edges are legal).	<code>tests/property/test_schema_lifecycle.py::test_lenient_reduce_respects_dag,</code> <code>::test_strict_rejects_promote_from_deprecated.</code>	A reducer that accepts deprecated → promoted directly, or rolls back promoter → inferred without a fresh-window RECOVER. Catches off-by-one transitions and "we forgo to enumerate this edge" omissions.
#2 <i>Promotion is deterministic and replayable</i> (pure fold over the event log).	<code>::test_reduce_is_deterministic, ::test_initial_snapshot_equivalence,</code> <code>tests/property/test_lifecycle_projection_roundtrip.py::test_projection_equals_direct_reduce,</code> <code>::test_projection_resumable_via_partial_replay.</code>	Any wall-clock / RNG / external state read snuck into the reducer. Replaying the same JSONL twice would diverge; the property would fail on the second run. AI catches "resume from offset N" bugs in the projection layer.
#3 <i>Promotion never invalidates stored properties</i> (version monotone under PROMOTE/DEPRECATE; only BUMP_VERSION increments).	<code>::test_bump_version_preserves_status_and_counts.</code>	A reducer that lazily bumps version on every PROMOTE — silently invalidating downstream rows tagged <code>schema_version=v1</code> because the snapshot now reads <code>v2</code> .
#4 <i>Schema writes serialize against extraction writes</i> (the concurrency claim).	<code>tests/property/test_lifecycle_concurrent_append.py</code> — four sub-invariants CL-I1..CL-I4 (lossless persistence, projection consistency, per-schema causal-order legality, per-kind histogram conserved). <code>N writers × shared threading.Barrier.</code>	Torn JSONL frames under racing <code>O_APPEND</code> , lost events under flock failure, snapshot diverging from <code>reduce_events(scan_order</code>
#5 <i>Demotion is reversible only through evidence</i> (RECOVER requires fresh <code>window_id</code>).	<code>::test_recover_requires_fresh_window.</code>	An oscillation bug where the same evidence window thrashes schema between live/dead — would let a single bad window cause unbounded RECOVER/DEPRECATE pairs
#6 <i>Lifecycle decisions are events, not in-place mutations</i> (the architectural claim).	<code>::test_create_on_existing_strict_raises, ::test_unknown_schema_strict_raises,</code> plus the cache-fuzz suite <code>tests/property/test_lifecycle_snapshot_cache_fuzz.py::test_c1_random_interleave_equivalence,</code> <code>::test_c2_append_only_offset_monotone_and_eof_hit, ::test_c3_rotate_increments_misses.</code>	A reducer that mutates an existing state on a stray CREATE (silent doubling <code>promote_count</code> on replay), or a snapshot cache whose fast-path disagrees with its from-scratch fold. The cache-fuzz suite is what makes invariant #2 hold <i>under the fast path</i> , not just under cold replay.

A.4.17.2 Cross-feature compositions

Two further suites cover the

interaction

of the lifecycle reducer with adjacent v0.2 features — necessary because invariants #1–#6 each hold in isolation but the production engine composes them:

- `tests/property/test_dedup_lifecycle_composition_stateful.py` — asserts that write-side cosine deduplication (§3.7.2) and lifecycle emissions are mutually independent: dedup decisions are unchanged by interleaved `LifecycleEvents` on the same schema, and the lifecycle projection is unchanged by dedup absorption (i.e., a fact being absorbed never fires a spurious lifecycle transition). Caught a draft of §3.7.2 that briefly considered emitting a synthetic `BUMP_VERSION` on dedup-merge.
- `tests/property/test_extraction_conf_lifecycle_composition_stateful.py` — asserts that per-fact extraction confidence (§3.7.3) and the `respect_schema_lifecycle` retrieval filter factorise: the score of a candidate from a non-deprecated schema does not depend on lifecycle history, and a deprecated-schema filter is independent of the candidate's extraction-confidence value. The factorisation property is what allowed §A.4.6's bisection to attribute the retrieval delta to *extraction*, not to lifecycle filtering.

A.4.17.3 Headline numbers

The full property surface (across the seven files referenced above) runs at **27 properties** (11 in the core reducer suite, 4 in the projection round-trip, 6 in the snapshot-cache fuzz, 6 in the concurrent-append harness) plus the two stateful composition suites. Wall-clock under the production Hypothesis settings is ≤ 8 s on a Ryzen-class laptop; the suite is part of every `pytest -q` run that gates a commit. 1715 passed, 3 skipped as of the cron run dated 2026-05-24 (commit cc5f72e).

A.4.17.4 Why this lives in the appendix, not §3

A natural alternative is to inline the invariant-bug-test table into §3.7.4. We deliberately keep §3.7.4 a prose specification of *what* the reducer is — five rules and a DAG — and push *why we trust the implementation* here. A reviewer who accepts §3.7.4's contract on inspection can skip §A.4.17; a reviewer who wants to audit the gate between specification and implementation gets the chain of evidence in one place. This mirrors the §3.6 / §A.4.7 split: methods state the mechanism, appendix shows the falsification budget.

Artifacts: `tests/property/test_schema_lifecycle.py` (163 LoC), `tests/property/test_lifecycle_projection_roundtrip.py` (126 LoC), `tests/property/test_lifecycle_snapshot_cache_fuzz.py` (161 LoC), `tests/property/test_lifecycle_concurrent_append.py` (253 LoC), `tests/property/test_dedup_lifecycle_composition_stateful.py` (318 LoC), `tests/property/test_extraction_conf_lifecycle_composition_stateful.py` (410 LoC), `src/engram/consolidation/schema_lifecycle.py` (267 LoC, the reducer under test), `src/engram/consolidation/lifecycle_projection.py` (405 LoC, the projection layer).

Appendix B. Security audits of Engram retrieval and storage primitives

This appendix collects security-side threat analyses of Engram's retrieval pipeline and storage primitives. These are systems-side audits of ACL/access-control side-channels in PRF query expansion, `share_prior` reranking, schema-lifecycle caches, BM25/vector candidate pools, mechanical merge, FactExtraction, write-side cosine dedup, governed-memory primitives, and cross-channel coupling. They are

not

measurement-side threats to the retrieval claims of §4 — those are addressed in §6 of the main paper.

We retain them in the appendix because they document the security-audit methodology that supports the Engram artifact release; readers interested only in retrieval results can skip this appendix.

Section numbers preserve the original §6.X numbering as §A.6.X for cross-reference stability.

A.6.6 PRF query-expansion ACL side-channel (closed)

Pseudo-relevance-feedback (PRF, §3.6 / §A.4.7) runs a first-pass retrieval and mines dominant entities from the top-K texts to construct an expanded query. In the original wire-up the first pass executed at the `RetrievalEngine` layer — strictly upstream of the outer per-result ACL filter in `Engram.recall()` — so the entity-mining pool could include memories the actor lacked READ scope for. The expanded query, and therefore the actor's final ranking over its *own* memories, then depended on the cross-agent corpus. This is a side-channel oracle: an actor with `scope='own'` could detect the presence of specific tokens in another agent's private memories by observing rank-perturbations on its own queries, even though no cross-agent content ever reached the user-visible output.

The repro is small: Alice owns four `notes` ... memories, two with disambiguating entities (Apollo, Beta). Bob privately owns 20 docs of the form `notes Apollo Apollo item N`. Without the fix, PRF mines `apollo` from Bob's docs, rewrites Alice's query to `notes apollo`, and Alice's Apollo doc rises one rank. Flip Bob's corpus to Beta and Alice's Beta doc rises instead. The ACL *recall* filter still strips Bob's results from the output — the leak is purely in Alice's own ranking.

Closure (commit 07d5c35, branch `feat/prf-acl-side-channel-fix`). `RetrievalEngine.search()` accepts an optional `_acl_filter: (Memory) -> bool` that `_search_with_prf` applies to the first-pass results *before* the PRF expander is called. `Engram.recall()` binds the filter to `self._acl_allows_read(actor, mem.agent_id)` for the current actor. The filter is inert when ACL is disabled, and federated actors with `scope='*'` (e.g. an auditor / reviewer grant) still mine the full pool — the escape hatch is preserved by the same permission model that governs federated reads. Pinned by `tests/adversarial/test_prf_acl_side_channel.py` (7 cases covering rank invariance across four Bob corpora, expander pool isolation, the federated escape hatch, and ACL-disabled no-regression). Pre-fix: 4 of the 7 cases fail with diagnostic diffs; post-fix: 7/7 pass. Full suite remains green at 1581 passing.

A.6.7 PRF IDF-rarity ACL side-channel — closed

The PRF expander has a second cross-agent signal beyond the mining pool addressed in §A.6.6: the §A.4.15g *IDF-rarity gate* drops candidate entities whose corpus rarity falls below a threshold (default `idf_min_rarity = 0.5`). Pre-fix, the rarity score was $1 - df/N$ where both `df` and `N` were computed against the **global** FTS index — including memories the actor cannot READ. The keep/drop decision for an Alice-pool entity therefore depended on Bob's private corpus.

The leak is detectable end-to-end in numbers, not just in principle. Alice owns four `notes` ... memories; Apollo appears in one of them (`df=1` in Alice's slice, `N=4` $\Rightarrow$ `rarity=0.75`, gate keeps Apollo). Bob privately writes 50 `notes Apollo Apollo` ... memories. Pre-fix rarity collapses to `df/N  $\approx$  51/54`, giving `rarity  $\approx$  0.06` — below 0.5 — and the gate **drops** Apollo from Alice's PRF expansion, silently flipping her ranking on her own corpus. Switching Bob's corpus to Beta-dense flips the gated entity from one to the other: the post-expansion ranks vary with Bob's vocabulary even though no cross-agent content reaches Alice's output. This is the classic shape of a side-channel oracle.

Closure (commit e0aa422). `RetrievalEngine._build_prf_rarity_lookup` gains an `allowed_agents: set[str] | None` parameter that scopes **both** the `df` numerator and the `N` denominator via `m.agent_id IN (?, ...)`. `Engram._prf_rarity_allowed_agents(actor)` computes the allow-list from the ACL grant for the current actor: `None` when ACL is disabled, when the actor has `scope='*'`, or when the actor holds `Permission.FEDERATED` (preserving the federated escape hatch); `{actor, ''}` for `scope='own'`, mirroring `Grant.can_access.recall()` threads it through the new `_rarity_allowed_agents` private kwarg on `search()`. Inert in all single-actor and federated configurations.

Pinned by `tests/adversarial/test_prf_idf_acl_side_channel.py` (7 properties): four rank-invariance arms under the IDF gate $\times$ Bob signal, a direct rarity-lookup numeric assertion (`rarity(Apollo)  $\geq$  0.5` post-fix vs. ≈ 0.029 pre-fix in the same corpus), the federated reader keeping global df, and ACL-disabled no-regression. Pre-fix: 3 of the 7 cases fail (the direct lookup canaries); the behavioural rank tests do not perturb in this corpus geometry because the expander-pool fix from §A.6.6 already strips Bob's docs from the entity-mining input — but the gate's numeric decision is still demonstrably wrong, and an adversary with control over Alice's pool composition could still pivot the gate. Post-fix: 7/7 pass. Full suite: 1581 $\rightarrow$ 1588 passing, 3 skipped, 181 deselected, 226 s.

A.6.8 share_prior reranker ACL side-channel (§D-share-prior-acl)

A third sibling of the §A.6.6 / §A.6.7 leaks: the §96 *share_prior* reranker (`src/engram/retrieval/rerankers/share_prior.py`) builds an undirected entity-sharing graph over the post-fusion candidate pool and adds a bounded $\alpha \cdot \text{deg} / \text{max_deg}$ boost to each candidate's score. The reranker runs at the `RetrievalEngine` layer, which sits *upstream* of `Engram.recall()`'s outer ACL filter. Without an ACL-aware filter on the reranker pool, the entity-sharing graph spans cross-agent docs, so each Alice doc's `degrees[i]` counts edges into Bob's private corpus and the `max_deg` normaliser is global. The visible output is still all Alice's (the outer filter strips cross-agent rows), but their *ranking* — and even their absolute scores — are now a function of Bob's private content.

Closure: `RetrievalEngine.search()` now drops cross-agent docs from *results* *before* slicing the rerank pool, gated on the same `_acl_filter` already threaded through for §A.6.6. None preserves federated / single-actor behaviour (no over-correction). Inert when no reranker is configured.

Pinned by `tests/adversarial/test_share_prior_acl_side_channel.py` (8 tests / properties): five rank-and-score-invariance arms under diversifying / dense / orthogonal Bob corpora, a federated `scope='*'` reviewer who must still see Bob's docs, the ACL-off no-regression path, and a direct reranker-pool isolation canary (spy on `apply_reranker`'s pool, assert no `Zorbax` from Bob). Pre-fix: the canary fails immediately (15 `Zorbax` docs in the pool); the rank-invariance arms happen to hold because *share_prior*'s rank-0 preservation cap absorbs the boost differences in this corpus geometry — but the leak is unambiguous at the pool layer and would surface as score perturbation on a slightly larger α or denser sharing graph. Post-fix: 8/8 pass. Full suite: 1588 $\rightarrow$ 1596 passing, 3 skipped, 181 deselected, 226 s.

A.6.9 Schema-lifecycle cache ACL audit — clean

The §3.7.4 schema-lifecycle gate replays the buffer's `CONSOLIDATION_SCHEMA_LIFECYCLE` event stream through a mtime-keyed `CachedLifecycleSnapshot` and uses the resulting `{schema_id: SchemaState}` map to drop `DEPRECATED` `MemoryType.SCHEMA` candidates from a recall's result set. Three siblings of §A.6.6–§A.6.8 (`PRF×ACL`, `PRF` IDF-rarity, *share_prior* reranker) all turned out to be silent cross-agent ranking channels; we audited the lifecycle cache for the same shape of leak.

Audit closure (3 invariants + 1 positive control, pinned by `tests/adversarial/test_lifecycle_cache_acl_side_channel.py`):

- **ACL-LC-1:** Alice's recall ranking over `FACT` memories is bit-identical (content + score equality) under three Bob-emitted lifecycle traffic patterns: empty, single `CREATE+DEPRECATE` on `bob_schema_x`, and a six-event churn (`CREATE/PROMOTE/DEPRECATE/ BUMP_VERSION` across three Bob schema ids). The lifecycle filter only fires on candidates of type `SCHEMA`, so cross-agent `FACT` ranking cannot depend on Bob's schema status. Pinned for all three arms.
- **ACL-LC-2:** When a `SCHEMA` candidate's `schema_id` collides with a `DEPRECATED` entry in the snapshot, it is suppressed *regardless of which actor emitted the lifecycle event*. This is the **intended** behaviour — schemas are `agent_id=''` (system-wide patterns), and the lifecycle DAG is global by design — but we pin it explicitly so any future "scope lifecycle by emitter" change has to refresh this test rather than silently change semantics.
- **ACL-LC-3:** With `respect_schema_lifecycle=False`, lifecycle traffic is fully inert — even a `CREATE+DEPRECATE` pair on a present `SCHEMA` leaves it reachable. Confirms the cached snapshot is not silently feeding any other recall signal (no second consumer).
- **Positive control:** With the gate on, an explicit `DEPRECATE` *does* suppress a present `SCHEMA`. Paired with `ACL-LC-3`, the two show the cache is reachable when on and inert when off, so `ACL-LC-3` is a real null and not a silent skip.

Result: no leak. The lifecycle cache passes the cross-agent audit without code changes. Closes the last open `NEXT-list` audit item ("confirm no other recall paths feed cross-agent texts into a learned signal"). Full suite: 1596 $\rightarrow$ 1602 passing, 3 skipped, 181 deselected, 226 s.

A.6.10 Cross-actor schema-id targeted suppression — pinned threat

§A.6.9 named, in passing, that `DEPRECATE` on a colliding `schema_id` is "global by design." Pinned here as an explicit, named threat-model statement instead of a quiet implementation detail.

Attack. A malicious actor that learns a victim schema's `schema_id` can append `CREATE+DEPRECATE` lifecycle events on that id and silently suppress the schema from every other actor's recall. The lifecycle DAG is intentionally global (schemas are `agent_id=''`, system-wide patterns), so the suppression applies to everyone uniformly. There is no per-emitter ACL on lifecycle events.

Observability. Schema ids are exposed through normal recall: any actor whose query surfaces a schema candidate sees its `memory.id` in the result row. Schema ids are `mem-sc-<uuid4().hex[:12]>` — 48 bits of entropy, so blind guessing costs $\sim 2^{47}$ writes per landed suppression on average, but observed ids are free.

Pinned tests (`tests/adversarial/test_schema_id_targeted_suppression.py`, +7 tests):

- **SC-ID-1:** A `CREATE+DEPRECATE` pair on a known schema id globally suppresses the schema. Worst-case attack surface, pinned.
- **SC-ID-2:** A `DEPRECATE` on a non-existent / mis-guessed id is a full no-op (5 parametric arms over different guess shapes). Confirms the entropy floor — guessing alone is infeasible.
- **SC-ID-3:** End-to-end chain. An actor recalls a system-wide schema, observes its id from `result.memory.id`, then writes `CREATE+DEPRECATE`; subsequent recalls miss the schema. This is the realistic attack path.

Mitigation surface (deferred — not implemented in v0.2):

- *Scope lifecycle by emitter (per-agent DAGs).* Cleanest fix; but it breaks the intended global-schema semantics, so it would need a parallel "shared-pattern" type and an explicit promotion pathway from `per-agent` $\rightarrow$ `shared`.

- *Quorum gate on DEPRECATE.* Require $\geq k$ independent emitters or an audit-trail signature before a deprecation takes effect. Adds latency to legitimate deprecations.
- *Redact schema ids in cross-actor recall result rows.* Closes observability but breaks debuggability and any actor's ability to reference its own schemas by id.

We document this as a known, accepted limitation of the global lifecycle DAG. The system is single-tenant by intent; multi-tenant deployments must either trust all actors with respect to schema deprecation or pick one of the mitigations above. Future work item.

A.6.11 BM25/vector candidate-pool ACL side-channel (closed)

A fifth sibling of the §A.6.6–§A.6.9 channels — and the most direct one, because it fires on the *primary* candidate-generation path rather than on an opt-in retrieval feature. The retrieval engine builds the hybrid candidate pool by calling `store.search_text(query, limit=k*5)` for BM25 and `vector.search(query_vec, limit=k*5)` for `sqlite-vec`. Neither call was ACL-scoped: both scanned all agents' memories, then each candidate received a `bm25_rank` (and `vector_rank`) reflecting its position in the *global* pool. The outer ACL filter at `engine.recall()` (`engine.py:408`) stripped cross-agent docs *after* RRF fusion, so the visible output contained only Alice's docs but their fused scores — and therefore their relative order — depended on where Bob's private docs landed in the global pool.

This is a presence oracle that fires even when the reranker, PRF, and lifecycle gate are all off. In the worst case (Bob's private corpus saturates the top- $k*5$ BM25 hits), Alice's recall returned the empty set despite her own corpus matching the query: a denial-of-recall as a function of Bob's content.

Closure (`engine.py:_search_with_prf`): drop cross-agent candidates from the pool *before* RRF fusion, then re-number `bm25_rank` and `vector_rank` so they are contiguous over the actor-visible candidates. We reuse the `_acl_filter` callable already threaded through for §A.6.6–§A.6.8. None preserves federated / single-actor behaviour exactly. Test pin: `tests/adversarial/test_vector_channel_acl_side_channel.py` — 3 BM25 parametric arms (orthogonal / Apollo-overlap / Beta-overlap) verify Alice's order *and* fused scores are bit-identical with and without Bob's corpus, plus a positive control that confirms the seed actually does perturb retrieval when ACL is off (so the invariance is real, not a harness artifact).

Residual. FTS5's BM25 score itself is computed over global corpus statistics (avgdl, df, N). Re-numbering closes the *rank-position* channel, which is the only BM25-derived signal that enters fused scoring (RRF uses the rank, not the raw BM25 score). The score-magnitude channel does not enter recall scoring at all in v0.2 and so is not exploitable through the public API. We note this explicitly so future work that begins consuming raw BM25 scores re-opens the audit.

A.6.12 MechanicalMerge ACL side-channel (closed)

The Stage 12 `MechanicalMerge` consolidator is the write-side analogue of the §A.6.11 read-side channel. It iterates `store.all_active()` (global, no ACL scope), embeds each memory, asks the vector store for near-duplicates above its cosine threshold (default 0.95 in production, 0.90 in this test), and unconditionally moves the lower-salience side of each matching pair into `MemoryState.SUPPRESSED`.

The pre-fix implementation never consulted `agent_id` on the matched pair, which fused two distinct vulnerabilities into one stage:

1. **Existence oracle.** When Bob writes content semantically near-identical to Alice's, mechanical merge silently moves Bob's row into `SUPPRESSED`. Alice cannot read Bob's row directly under ACL — but she can observe a *state transition* on it via lifecycle metadata (`memories_suppressed` in the consolidation report, `state` column queries on the audit path). This separates "Bob never wrote this" from "Bob wrote a near-duplicate of my content," which the threat model forbids.
2. **Cross-tenant denial-of-recall.** In the asymmetric case where one tenant runs at higher salience than another, the louder tenant's writes systematically suppress the quieter tenant's near-duplicates. Multi-tenant deployments cannot tolerate this: a single noisy tenant could erase a quiet tenant's memories simply by writing similar content at higher salience.

Closure (`pipeline.py:MechanicalMerge.run`): over-fetch the candidate pool from `limit=5` to `limit=20` and skip any pair whose `agent_id` strings differ. System-owned memories (`agent_id=''`, e.g. SCHEMA prototypes) remain mergeable globally — that is the intended behaviour for shared system patterns and matches the §A.6.9 lifecycle-cache audit's treatment of schemas as agent-less.

Test pin: `tests/adversarial/test_mechanical_merge_acl_side_channel.py` (5 invariants).

- **MM-ACL-1** Cross-agent near-duplicates *are never* suppressed, in either salience-ordering direction (two parametric arms: Alice high / Bob low, and the symmetric reversal).
- **MM-ACL-2** Same-agent near-duplicates *are* still suppressed. This is a positive control — the fix is a scope filter, not a stage disable.
- **MM-ACL-3** System memories (`agent_id=''`) remain globally mergeable.
- **MM-ACL-4** A system memory does *not* suppress an agent-owned near-duplicate — `agent_id=''` is not a wildcard owner.

Pre-fix, three of five tests fail (MM-ACL-1 both arms + MM-ACL-4), confirming the channel is real and the patch is the minimum scope filter that closes it without breaking the same-agent merge path.

Residual. The merge stage's candidate fan-out is now bounded by `limit=20` over actor-visible duplicates. In a pathological corpus where one agent has ≥ 20 near-duplicates within the threshold, the 21st-onwards may go unmerged in a single pass; the next consolidation cycle catches them. We accept this — the alternative (full per-agent scan) is $O(n^2)$ in agent-owned content, and 20 is two orders of magnitude above the empirical near-duplicate density we see in the §4 corpora.

A.6.13 FactExtraction ACL inheritance bug (closed)

The most direct ACL leak in the v0.2 surface, and the worst one to have shipped — though it was caught before any external user was exposed. The Stage 4c `FactExtraction` consolidator distills EPISODE memories into FACT rows via an LLM call. The synthesised `CONSOLIDATION_EXTRACT` event carries no actor context, so the default `Memory.agent_id` for the resulting fact was `''` (empty — "system memory" in the v0.2 ACL model).

Pre-fix flow:

1. Alice (`agent_id='alice'`) writes "I am meeting Mallory at 3pm at the Whitebridge."
2. Consolidation extracts the fact "Alice meets Mallory at the Whitebridge" — distilled, structured, often more searchable than the source.
3. The fact is stored with `agent_id=''`. Under `Grant.can_access`, any actor's grant matches `agent_id=''` because system-shared content is intentionally readable to all.
4. Bob's `recall("Mallory")` surfaces the distilled fact. Alice's ACL does not protect her — her own consolidation pipeline promoted her content into the system pool.

This is strictly worse than §A.6.11 / §A.6.12: those leak a *signal* (rank position, suppression state) about Alice's content. This one leaks the **distilled content itself**, including any facts the LLM extracted with structured *properties* — which in the dual-extraction schema (Governed Memory paper) include explicit entity bindings.

Closure (pipeline.py:FactExtraction.run): one line — `fact_memory.agent_id = memory.agent_id`. The source episode's owner is the only correct attribution; the consolidation event has no actor context to fall back on. Schemas keep `agent_id=''` intentionally (system-shared patterns, matching the §A.6.9 lifecycle audit).

Test pin: `tests/adversarial/test_fact_extraction_acl_inheritance.py` (4 invariants):

- **FE-ACL-1** Facts from Alice's episodes inherit `agent_id='alice'`.
- **FE-ACL-2** Facts from Bob's episodes inherit `agent_id='bob'`.
- **FE-ACL-3** Facts from system episodes stay `agent_id=''`. The fix forbids silent *promotion* of agent-owned content into the system pool, not legitimate system-to-system extraction.
- **FE-ACL-4** Mixed batch (Alice + Bob episodes in one stage invocation) → per-fact attribution is correct, no cross-contamination.

This is the third write-side ACL bug found in the audit pass (§A.6.12 mechanical merge, §A.6.13 fact inheritance, plus the Stage 9 schema extraction explicitly retains `agent_id=''` by design). The pattern is consistent: any stage that synthesises a new event without an actor context defaults the resulting memory to system scope, and any `Memory.from_event` call site needs an explicit attribution rule. Future stages must be audited against this invariant before merge.

A.6.14 Write-side cosine dedup ACL side-channel (closed)

Threat (§D-write-dedup-acl): the Governed Memory write-side cosine dedup (§3.7, threshold 0.92) calls `vector_store.search()` over the *global* vector index. A writer Bob whose content embedding sits within the dedup threshold of one of Alice's stored memories has his write silently suppressed: `engine.remember()` returns the deduped event id, the projection is not inserted, and the audit log records `remember_deduped status=skipped`.

This produces three observable presence oracles for Bob:

1. **Audit channel:** `remember_deduped` fires only when a near-cosine neighbour exists *somewhere* in the store. Mining the audit log reveals which probes intersect Alice's content.
2. **Recall asymmetry:** Bob's `remember()` returns a non-empty event id, but his subsequent `recall()` over his own scope returns 0 hits. The gap is observable without any access to Alice's data.
3. **Storage delta:** the JSONL event buffer grows but Bob's projection-row count does not. Any monitor watching the event-log/projection delta sees the leak.

This is

write-side

and survives every read-side ACL fix landed through §A.6.13 — the previously-closed channels (PRF mining pool, IDF-rarity, share_prior reranker, lifecycle cache, BM25/vector candidate pool, mechanical merge, fact-extraction inheritance) all operate after the writer's content has either landed or been deduped.

Closure (store/memory.py:upsert, engine.py:remember): an `acl_filter` callable plumbed into the dedup loop. When ACL is enabled, the writer's `_acl_allows_read` is consulted on each candidate neighbour's `agent_id` before the cosine comparison, and candidates outside the writer's visible scope are skipped. The candidate pool is widened from $K=5 \rightarrow K=32$ when filtering is active so top-K cross-agent neighbours don't crowd out a same-agent duplicate. Behaviour with ACL disabled is identical to the prior implementation (global dedup), preserving the legacy single-actor semantics.

Test pin: `tests/adversarial/test_write_dedup_acl_side_channel.py` (3 invariants):

- **WD-1** Bob writes content matching Alice's memory under ACL → Bob's projection lands and is recallable in his own scope. The failing pre-fix observation is exactly the leak.
- **WD-2** ACL disabled → cross-actor dedup still fires (regression guard for legacy single-tenant deployments).
- **WD-3** Same agent writes the same content twice → dedup fires within the writer's own scope (the legitimate behaviour we preserve; only cross-agent dedup is the leak).

Race extension (write-write contention). The §A.6.14 fix is verified under single-threaded conditions; we re-test it under contention because two `Engram` instances on the same store hold *separate* `_dedup_locks`, so ACL filtering must be correct in the absence of cross-instance serialisation. The order in `Engine.remember()` is `store.upsert() → vector.upsert()`, so a vector becomes visible to other writers only after its row is committed; this happens to make the race window *safer* than the sequential case for the leak direction, but two hazards still need pinning:

- **R1 (stale-row hazard).** A vector hit may resolve to `cand is None` if the row is not yet visible to the second writer's connection. The current code path treats this as "skip neighbour, do not suppress" — the safe direction. A future refactor that flipped the None-policy to "suppress" would silently reopen §A.6.14 whenever a row materialised after its vector index entry. We pin the policy with a monkey-patched `store.get → None` test that asserts Bob's write still lands.
- **R2 (Alice×Bob identical-payload storm).** 20-thread `ThreadPoolExecutor` (10 alice + 10 bob) all writing the same payload behind a `Barrier`. Invariant: each actor retains ≥ 1 row (cross-actor cannot suppress) AND same-actor dedup still bounds each actor's count to $\leq 50\%$ of writes (regression bound — naive code with no dedup serialisation would let all 10 land per actor).

- **R3 (presence-oracle under contention).** After 5+5 concurrent same-payload writes, Bob's own-scope `recall()` must return his payload regardless of Alice's interleaving. Verified to *fail* when ACL filter is forcibly disabled — the test is real, not vacuous.

Test pin: `tests/adversarial/test_write_dedup_acl_race.py` (3 invariants R1/R2/R3, marker = `concurrency`).

This is the

fourth

write-side ACL bug found in the audit pass (§A.6.12, §A.6.13, §A.6.14, plus the design-intentional schema sharing). The pattern continues to be consistent: any stage that consults global state (vector index, mechanical-merge cluster, fact-extraction attribution) without an explicit ACL gate becomes a side-channel. After §A.6.14 the v0.2 audit set is: read-side closed (§§A.6.6–A.6.11), write-side closed (§§A.6.12–A.6.14), with §A.6.10 (cross-actor schema-id targeted suppression) pinned as accepted behaviour of the global lifecycle DAG.

A.6.15 Governed-Memory extraction_confidence ACL audit — clean

Threat (§D-extraction-confidence-acl): the Governed Memory paper (arXiv:2603.17787) introduces a per-fact `extraction_confidence` (EC) multiplier set at consolidation time and applied at recall time as a score downweight (`engine.py:680–686`). EC enters fused scoring as `final *= clamp(ec, 0, 1)` whenever `RetrievalConfig.use_extraction_confidence=True` (the default).

The audit risk: would EC ever consume cross-agent state, either on the write side (extractor reading another actor's memories) or the read side (recall scoring weighting Alice's candidates by Bob's EC distribution — e.g. a "calibrate against corpus mean" normaliser)?

Result: clean, no closure required. Reading the code paths:

- **Write path** (`consolidation/pipeline.py:362–406`): `FactExtraction.run` iterates over `ctx.memories_created` (the current run's episodes), invokes the LLM on a single episode's content, and stores the parsed `confidence` field clamped to `[0, 1]` on the resulting fact. No cross-agent memory is consulted; `agent_id` is inherited from the source episode (closed in §A.6.13).
- **Read path** (`retrieval/engine.py:680–686`): `_final_score` multiplies in only the *candidate's own* `memory.extraction_confidence`. There is no aggregate, no peer lookup, no normalisation against other candidates' EC distributions.
- **Storage path** (`store/memory.py:294, 781`): EC is a per-row column read from the candidate's own row; no cross-row JOIN.

Pin (`tests/adversarial/test_extraction_confidence_acl_side_channel.py`, 7 tests, 4 parametric arms in EC-ACL-1):

- **EC-ACL-1** Alice's FACT-only ranking (content + score) is bit-identical regardless of Bob's EC distribution. Four arms: no Bob facts (control), all-1.0, all-0.0, 5-quantile spread. Pins write-side and read-side together: a future change that introduces a peer-aware EC normaliser (e.g. corpus-mean calibration) has to refresh this test.
- **EC-ACL-2** EC is a strict per-candidate multiplier: across `c ∈ {0.1, 0.25, 0.5, 0.75, 0.9}`, the score of a fixed Alice candidate at `EC=c` equals `c × score(EC=1.0)` to `rel_tol=1e-6`. Confirms the EC channel has no nonlinear / cross-candidate coupling.
- **EC-ACL-3** With `use_extraction_confidence=False`, no Alice candidate's score depends on any EC value (own or Bob's). Confirms the gate has no second consumer.
- **EC-ACL-4** Positive control: with the gate ON, an Alice candidate at `EC=0.05` scores `< 0.5 ×` the same candidate at `EC=0.95`. Pins EC-ACL-3 as a real null rather than a dead gate.

§A.6.15 closes the last open audit thread on the Governed Memory feature surface. Combined with §§A.6.6–A.6.14, the v0.2 cross-actor audit set is fully accounted for: read-side closed (§§A.6.6–A.6.11), write-side closed (§§A.6.12–A.6.14), §A.6.10 pinned as accepted behaviour, §A.6.15 audited clean.

A.6.16 Quorum-gated DEPRECATE (mitigation prototype for §A.6.10)

§A.6.10 pinned, as accepted behaviour, the fact that any actor that learns a `schema_id` can append `CREATE+DEPRECATE` and globally suppress the schema. We deferred a fix in v0.2 because every available mitigation traded against either the global-schema semantics or the debuggability of recall result rows. In this section we land the *reducer-level* prototype of a quorum gate — opt-in, defaults-off — so the production wiring can follow without further reducer churn and so the threat model now has a concrete, defended mitigation rather than just a gap.

Mechanism. `SchemaLifecycleEvent` gains an optional `emitter_id` field, and `reduce_events` gains a `deprecate_quorum_k: int = 1` parameter. When `k > 1`, a `DEPRECATE` event no longer fires the `INFERRED|PROMOTED → DEPRECATED` transition on its own; instead the reducer accumulates `pending_deprecate_emitters: frozenset[str]` on the schema state and the transition fires only once the set has at least `k` distinct entries. Any non-`DEPRECATE` transition (`CREATE`, `PROMOTE`, `RECOVER`) clears the ballot — promotion or recovery is positive evidence the schema is alive, so partial dissent collected beforehand should not be reusable later. The default `k=1` preserves the legacy single-emitter path byte-for-byte; this is intentional, the mitigation is *available*, not *mandatory*, because the system is single-tenant by design.

Properties pinned (`tests/adversarial/test_schema_deprecate_quorum.py`, +14 tests):

- **Q-1, Q-9** Default `k=1` is bit-identical to the pre-quorum reducer. Q-9 is the positive control that confirms the §A.6.10 attack still works at `k=1` — its job is to fail loudly if we accidentally flip the default.
- **Q-2** Under `k=2`, a single attacker's `DEPRECATE` leaves the schema `INFERRED` with a one-element ballot. The §A.6.10 attack stops working at `k=2`.
- **Q-3** Two distinct emitters fire the transition exactly once; ballot clears.
- **Q-4** A single emitter re-voting up to `k` times does *not* satisfy quorum — distinctness is on `emitter_id`, not on event count. Sybil resistance proper is out of scope (the reducer trusts the emitter id), but at minimum repeated self-voting cannot bootstrap quorum.
- **Q-5** `PROMOTE` clears a partial `DEPRECATE` ballot. Otherwise an attacker who voted long ago could collude with one fresh emitter to suppress a now-promoted schema. This is the "stale dissent" bug.

- **Q-6 RECOVER** (deprecated $\rightarrow$ inferred) likewise clears the ballot; the next quorum attempt restarts from zero votes.
- **Q-7, Q-8** A DEPRECATE event with `emitter_id=None` under $k > 1$ raises `LifecycleViolation` in strict mode and is dropped silently in non-strict mode — i.e. malformed events cannot bypass the gate by omitting the emitter.
- **Q-10** Parametric over $k \in \{2, 3, 5\}$: the reducer fires *exactly* at the k -th distinct vote, never earlier and never later; intermediate ballots have the expected cardinality.
- **Q-11** Invalid $k=0$ raises `ValueError` at the API boundary.
- **Q-12** The quorum fold is a pure function: same input $\Rightarrow$ same output across two calls.

Residual. The prototype lives at the reducer layer. The production wiring — which consolidator is allowed to attach an `emitter_id`, how `emitter_ids` are bound to actor identities, and which deployment configurations should run with $k > 1$ by default — is left for the integration step. The point of pinning the prototype now is that the reducer's invariants are non-negotiable from here forward; downstream code can land without further property churn.

§A.6.16 also gives §A.6.10 a concrete defended path: multi-tenant deployments that cannot tolerate the single-actor suppression channel can opt into `deprecate_quorum_k >= 2` and receive a quorum-gated lifecycle DAG with the full property suite above as the contract.

6.16 phase B: call-site wiring and cache identity

The phase A landing (commit 4ea3589) threaded `emitter_id` through the wire format and the reducer; phase B (commit 71eef9) closed the plumbing on both ends:

- **Write side.** `Config consolidator_id: str = ""` is the per-process identity of the consolidator emitting lifecycle events; it is threaded through `ConsolidationPipeline.run` into `StageContext consolidator_id` and from there into every `make_lifecycle_event(emitter_id=...)` call site under `SchemaUpdate`. The empty default preserves byte-stable legacy emission (the `emitter_id` key is omitted from `metadata` entirely; pinned by test B-10). Production deployments set this to a stable per-node id (e.g. `socket.gethostname()`).
- **Read side.** `RetrievalConfig.deprecate_quorum_k: int = 1` is plumbed through `RetrievalEngine.search` into `CachedLifecycleSnapshot.get(deprecate_quorum_k=...)`, which forwards it to `reduce_events`. The cache treats k as part of the snapshot's *identity*: a stream that produced `DEPRECATED` under $\backslash(k=1)$ may yield `INFERRED` (with one pending ballot) under $\backslash(k=2)$ for the *same* event log, so changing $\backslash(k)$ between calls forces a full rebuild. This is pinned in both directions by tests B-6 and B-7; B-9 exercises the engine $\rightarrow$ cache $\rightarrow$ reducer composition end-to-end on a real `RetrievalEngine.search` call.

The two knobs (`Config consolidator_id` and `RetrievalConfig.deprecate_quorum_k`) are independent and both default to bit-identical legacy behaviour, so phase B is regression-safe for single-tenant single-node deployments while multi-tenant deployments can adopt either or both as their threat model demands.

A.6.17 Cross-channel coupling audit

The per-channel ACL audits (vector pool, BM25/FTS5, mechanical merge, fact extraction, write-side dedup, extraction confidence, lifecycle/cache+quorum) each pin one channel's behaviour against an adversarial peer. They are necessary but not sufficient: a side channel can hide in a *pair* of channels even when each is individually clean. We therefore catalogued every off-diagonal pair $\backslash(X \times Y)$ on the channel set and classified each as **I** (independent by construction — no shared cross-actor state, no shared score channel), **C+covered** (composed and pinned by an existing single test path), or **C+gap** (composed and *not* pinned by a single existing test $\rightarrow$ write a dedicated combinatorial test). The catalog and the per-pair reasoning live in `research_notes/cross_channel_coupling_audit.md`.

The catalog has been audited at two channel counts. At $\backslash(n=7)$ (vector pool, BM25/FTS5, mechanical merge, fact extraction, write-side dedup, extraction confidence, lifecycle/cache+quorum) the surface is $\backslash(\binom{7}{2}=21)$ pairs with split **11 I / 8 C+covered / 2 C+gap**. The 2026-05-24 expansion adds two read-time score channels — H (PRF entity expansion, dominance-gated) and I (share_prior reranker, rank-0-capped boost) — following the v0.2 wiring of both as runtime-toggable knobs in `RetrievalConfig`. At $\backslash(n=9)$ the surface grows to $\backslash(\binom{9}{2}=36)$ pairs (+15 new off-diagonals); the new verdict split is **17 I / 19 C+covered / 0 C+gap**. All six read-time $\times$ write-time corners ($C \times H, D \times H, E \times H, C \times I, D \times I, E \times I$) are **I** by the read/write decoupling argument: H and I read only from the post-write actor-scoped candidate pool and have no path to the write-time channels' state. The remaining nine new pairs are all **C+covered** by the diagonal ACL audits (which already exercise hybrid recall under PRF and share_prior) plus the §A.4.7 PRF $\times$ SP $\times$ EC panel which exhaustively pins $F \times I$ and $H \times I$ across six sweep axes at $\backslash(n=10)$ seeds with paired bootstrap CIs. The two gaps catalogued at $\backslash(n=7)$ were both write-time mech-merge couplings, since closed:

- **C $\times$ E (mech-merge $\times$ write-dedup).** Adversarial scenario: Bob-write $\rightarrow$ Alice-merge interleavings could in principle let dedup-derived state from a peer leak into Alice's merge candidate set. Closed by `tests/adversarial/test_cxe_mech_merge_x_write_dedup_compose.py` with three deterministic two-actor invariants (CXE-1..CXE-3): under any Bob near-dup history, Alice's intra-actor merge invariant holds, Bob's rows survive composition, and the intra-Alice merge still fires.
- **C $\times$ F (mech-merge $\times$ extraction-confidence).** Adversarial scenario: could a peer's EC distribution influence which of Alice's two same-actor candidates survives a merge, or perturb the surviving EC? Closed by `tests/adversarial/test_cxf_mech_merge_x_extraction_confidence.py` with three invariants (CXF-1..CXF-3) sweeping Bob's EC over the four cells $\backslash(\backslash\text{low,high}\backslash)\times\backslash(\backslash\text{low,high}\backslash)$, asserting Alice's survivor identity and the bit-identity of its EC, plus a salience-only-survivor positive control to catch any future EC-weighted refactor.

Both gap tests were green on first run. The catalog also fixes a forward decision rule for future channels: a new channel $\backslash(X)$ adds $\backslash(n)$ pairs, and $\backslash(X \times Y)$ is declared **I** with one-sentence reasoning iff $\backslash(X)$ shares no per-actor state and no multiplicative/additive score contribution with $\backslash(Y)$ — otherwise it is **C+gap** until a single test path exercises both. **Read/write decoupling refinement (n=9 update):** read-time score channels (F, G, H, I) cannot couple to write-time channels (C, D, E) because their input is the post-write actor-scoped candidate slice; this collapses the $\backslash(X \times Y)$ surface for every new read-time channel from $\backslash(n-1)$ pairs to at most the prior count of read-time channels. The resulting audit-cost ceiling continues to scale linearly: at $\backslash(n=9)$ the coupled subset is 19/36 (53%, all covered), and the jump from 8/21 (38%) at $\backslash(n=7)$ comes entirely from H and I being read-time score channels which couple with every prior read-time channel by construction.

A3. Extended Related Work

This appendix expands §2's main-body summary into the full related-work coverage that an ACL 2-column 6-page Industry Track body cannot accommodate. Anchor citations + direct comparisons live in §2; the expansions below cover (1) other contemporary agent- memory designs, (2) the long-context retrieval-evaluation suite, (3) the dense-retrieval evaluation ecosystem, (4) late-interaction and learned-sparse non-inclusion, (5) pseudo-relevance feedback, and (6) the event-sourcing / bi-temporal / CRDT lineage.

A3.1 Other contemporary memory designs

Beyond the three direct-comparison systems in §2.1, four further agent-memory designs warrant mention:

A-MEM (Xu et al., 2025; arXiv:2502.12110) builds an *agentic memory* substrate as a Zettelkasten-style graph: each note is an LLM-generated atomic unit linked to others through LLM-suggested associative edges, with periodic LLM-mediated "consolidation" passes that rewrite link structure. The contrast with Engram is governance: A-MEM's link graph is rewritten whenever the consolidation LLM judges the substrate is stale, and there is no replay or audit trail across rewrites. Engram's schema lifecycle (§3.7.4) explicitly trades off this flexibility for deterministic replay.

HippoRAG (Gutiérrez et al., 2024; arXiv:2405.14831; HippoRAG 2, arXiv:2502.14802) routes retrieval through a knowledge graph constructed at write time and uses Personalized PageRank at query time to surface bridge entities for multi-hop questions. Where Engram's `share_prior` reranker (§3.5) computes within-pool entity co-occurrence on the *post-retrieval* candidate set, HippoRAG operates pre-retrieval over the full graph. The two are complementary: HippoRAG addresses recall-side bridge promotion, `share_prior` addresses precision-side rank-0 preservation.

Cognee (cognee-ai/cognee, 2024–) is an open-source semantic memory layer combining vector retrieval with a knowledge-graph index synthesized via LLM extraction. Like A-MEM and HippoRAG, Cognee centers the graph as the substrate; like Mem0 v3, it relies on add-only writes with periodic re-consolidation. We do not benchmark against Cognee directly because it does not expose a session-level hit@k metric reproducible from raw input chats.

Zep / Graphiti (Rasmussen et al., 2025; arXiv:2501.13956) provides a temporal knowledge-graph memory with bi-temporal edges and reified relationships, evaluated on Deep Memory Retrieval (DMR). Zep emphasises temporal coherence — what was true when — which Engram approximates through schema-lifecycle decay (§3.7.4) but does not store as first-class temporal edges. The two designs answer different "what did the agent know" questions: Zep's is graph-shaped, Engram's is event-log-shaped.

The §2.1 "where Engram sits" framing extends to these systems: A-MEM, HippoRAG, Cognee, and Zep each center a graph substrate that is rewritten under LLM mediation. Engram's mechanical- governance bet differs categorically.

A3.2 Long-context retrieval evaluation suite

The two benchmarks named in §2.2 are the agent-memory community's conversational anchors; the broader long-context retrieval- evaluation ecosystem we draw methodology from includes:

- **RULER** (Hsieh et al., 2024; arXiv:2404.06654) — 13 synthetic tasks (NIAH variants, multi-key/value retrieval, variable tracking, frequent-words extraction, long-document QA) parametric in context length up to 1M tokens. RULER showed that nominally long-context models often degrade well below their advertised window; we adopt its **stratified-by-task** discipline at the retriever level rather than the model level, which is the methodological seed of our entity-collision per-tag stratification.
- **∞Bench** (Zhang et al., 2024; arXiv:2402.13718) — 12 tasks averaging >100k tokens across math, code, novels, and dialogue. Used in the long-context evaluation literature as a length-stress complement to RULER's task-stress; not directly applicable to agent memory because tasks are document-centric, not session- centric.
- **LongBench-v2** (Bai et al., 2024; arXiv:2412.15204) — 503 multiple-choice questions over 8k–2M-token contexts. Methodologically closer to multi-doc QA than agent memory.
- **NIAH / Needle-in-a-Haystack** (Kamradt, 2023; github.com/gkamradt/LLMTest_NeedleInAHaystack) — single-fact retrieval at controlled depth. The closest one-axis ancestor of entity-collision; entity-collision generalises by stratifying on *discriminator type*, which NIAH does not.
- **LV-Eval / LooGLE / L-Eval** (An et al., 2023, arXiv:2307.11088; Li et al., 2023, arXiv:2311.04939; Yuan et al., 2024, arXiv:2402.05136) — long-context QA suites that all report a single hit@k or LLM-judge accuracy per model, exhibiting the tag-mixing problem §1 motivates the entity-collision protocol to address.

A3.3 Dense-retrieval evaluation ecosystem

- **BEIR** (Thakur et al., 2021; arXiv:2104.08663) — 18-task zero-shot retrieval benchmark. Established the "BM25 is hard to beat zero-shot" finding that anchors our entity-collision protocol's BM25-floor design.
- **MTEB** (Muennighoff et al., 2022; arXiv:2210.07316) — Massive Text Embedding Benchmark, 56 datasets across 8 task families. Source for our embedder choices: BGE-large-en-v1.5 sat near the top of MTEB's retrieval leaderboard at the time of our encoder grid freeze, which is why we extended the protocol to it as the encoder-capacity falsification test.
- **MS MARCO** (Bajaj et al., 2016; arXiv:1611.09268) and **TREC Deep Learning** (Craswell et al., 2020, arXiv:2003.07820; 2021, arXiv:2102.07662) — passage-retrieval staples. Useful as a sanity prior for relative encoder ordering but tangential to the agent-memory setting because queries are short and corpora are static.

A3.4 Late-interaction and learned-sparse baselines

The v0.2 measurement grid covers three **single-vector dense encoders** (HashTrigram-256, MiniLM-384, BGE-large-1024). We deliberately do not include late-interaction (ColBERT; Khattab & Zaharia, 2020, arXiv:2004.12832; ColBERT-v2, Santhanam et al., 2022, arXiv:2112.01488) or learned-sparse retrievers (SPLADE; Formal et al., 2021, arXiv:2107.05720; SPLADE++, Formal et al., 2022, arXiv:2205.04733) — both are documented to outperform single-vector dense encoders on BEIR — because the headline question this paper engages is specifically **whether per-query semantic capacity in the single-vector regime is the binding constraint on agent-memory**

retrieval. The two-axis result of §4.3 and the encoder-capacity falsification of §A.4.16 are claims about that regime: a $2.7\times$ -parameter increase within the single-vector family does not collapse the lexical-vs-intent split, which is the methodological finding that motivates the protocol. ColBERT and SPLADE answer a different question — whether *interaction structure* (token-level late interaction) or *index sparsity* (learned sparse projections) recovers cells the single-vector family cannot — and a clean answer to that question requires its own protocol design, not a 4th column on this grid. We accordingly flag late-interaction and learned-sparse retrievers as a **v0.3 follow-up** with its own protocol freeze, not a v0.2 omission.

A secondary, deployment-side consideration reinforces this scope boundary: ColBERT's per-token storage and SPLADE's per-query inference cost both invert the deployment trade-off table in §3.1.1 on commodity-CPU hosts (the v0.2 testbed and the v0.2 default-embedder choice). A reviewer interested in the accelerator-only deployment regime should read §A.4.16 alongside this section: the same trade-off applies to BGE-large there, and ColBERT/SPLADE land further along the same axis.

A3.5 Pseudo-relevance feedback

- **RM3** (Lavrenko & Croft, 2001, *SIGIR* — "Relevance-Based Language Models") — the canonical relevance-model PRF expansion that mixes a discriminative term distribution via learned λ . §5.6 details why our heuristic-PRF falsification (§A.4.15j-o) does not extrapolate to RM3. AUDIT-D ships an RM3 arm across the entity-collision grid, BEIR FiQA, and LongMemEval $n=500$ (§A.4.16.4); the headline finding is that RM3 does not rescue PRF on intent-style queries, sharpening §4.3's two-axis claim.
- **Rocchio relevance feedback** (Rocchio, 1971, in *The SMART Retrieval System*, Salton ed., Prentice-Hall) — original vector- space PRF. Cited for completeness; supplanted by RM3 in modern retrieval-evaluation practice.

A3.6 Schema-lifecycle and event-sourced memory

The schema-lifecycle invariant set we discuss in §3.7.4 and §5.3 draws on three threads:

- **Event sourcing** (Fowler, 2005, martinfowler.com/eaDev/EventSourcing.html; Vernon, 2013, *Implementing Domain-Driven Design*, Addison-Wesley) — pattern from domain-driven design where state is the fold of an immutable event log rather than a mutable record. Our schema lifecycle is an event-sourced reducer (`tests/property/test_schema_lifecycle.py`).
- **Bi-temporal data modelling** (Snodgrass, 1999, *Developing Time-Oriented Database Applications in SQL*, Morgan Kaufmann; Date, Darwen & Lorentzos, 2002, *Temporal Data and the Relational Model*, Morgan Kaufmann) — the discipline of separating "what was true" from "when we knew it was true." Engram approximates the latter through write timestamps on the decision log; we do not claim full bi-temporal correctness.
- **CRDT / monotone reducer literature** (Shapiro, Preguiça, Baquero & Zawirski, 2011, SSS / INRIA RR-7687) — informs our "decay is monotone in real time" invariant (`test_schema_decay.py`).

A4. Extended Discussion

This appendix expands §5.1's main-body operational rule with the four supporting analyses that the ACL 2-column 6-page Industry Track body cannot accommodate. Each is referenced by a one-sentence pointer in §5.1; the full content lives here.

A4.1 Why does adaptive vector-weight routing fail?

The 11.7pp oracle gap on LoCoMo is real — there exists a per-query optimal **vw**, and switching at query time would close the gap. But the gap/crowdedness signals we tested do **not** localize that optimal. Two hypotheses:

1. **Signal coarseness.** `bm25_top1 - bm25_top2` only sees the top-2 distance; it misses the broader candidate distribution.
2. **Confounding with hardness.** A small gap may indicate "BM25 is uncertain" (good signal) **or** "all candidates are semantically near the gold" (bad signal — vector won't help either). The signals collapse both regimes.

We additionally trained a `GradientBoostingClassifier` over the full BM25 feature panel + category one-hot under leave-one-conversation-out CV across all 10 LoCoMo samples (§4.4, `evals/locomo_learned_router.py`). The learned router is also null ($\Delta\text{hit}@1 = -0.0005 [-0.0030, +0.0015]$ on HT; ST identical within Monte-Carlo noise). The router degenerates to `vw=0` because 1801/1978 queries already prefer `vw=0` in the oracle. The 11.7 pp headroom is real but unrecoverable from any pre-routing signal we can observe; gold position is required, which by definition cannot exist at decision time. We accordingly *re-frame* vector fusion as a **paraphrase-robustness** mechanism rather than a per-query precision lever, and validate that framing on LongMemEval (§A.4.8.2).

Verdict. Adaptive **vw** routing is a measured null on LoCoMo. The correct operational role for vector fusion is paraphrase-robustness, not per-query precision routing.

A4.2 Schema lifecycle as a research artifact

Why this section exists. A reviewer may reasonably ask why a paper about retrieval-evaluation methodology spends discussion on schema-lifecycle invariants. The answer is the bridge to §1's "deterministic governance is a prerequisite, not a contribution": the cross-encoder paired-bootstrap inversions in §A.4.16.3 ($n=100 \rightarrow n=500$) only make sense if re-running the same ingest against the same dataset produces the same memory store byte-for- byte. The three properties below are what give us that guarantee. They are *machinery for the methodology*, not a retrieval claim — §A.4.6 falsified the retrieval interpretation directly, and §A4.3 below restates the consolidation claim accordingly.

§A.4.6's bisection landed the operational claim that the lifecycle is not a retrieval mechanism. What it is a mechanism for is **deterministic governance under replay**. We formalize three invariants on the lifecycle reducer, each verified by exhaustive execution traces over the property-based testing substrate; the specific test file paths and per-property case counts are catalogued in the implementation traceability index (§A6).

1. **Lifecycle decisions are events, not in-place mutations.** A schema's state at time t is the fold of its decision log up to t . This is the same discipline event-sourced ledgers borrow from accounting; in a memory system it gives bit-identical audit replay across re-runs of the same ingest stream.
2. **Family clustering is decision-stable under permutation.** For any permutation of the input fact stream, the *family assignment* a fact lands in is invariant; only the order of decisions within a family is permuted. This is what makes the lifecycle safe to run concurrently: writers don't race for cluster identity.
3. **Decay is monotone in real time, not in arrival time.** A fact's confidence trajectory depends only on wall-clock spacing, not on whether other facts were observed between ticks; the property is verified under fuzzed interleaving of `tick()` and `update()` calls. This rules out a class of write-amplification bugs where a chatty witness inadvertently extends an unrelated fact's half-life.

These properties generalize beyond Engram: any memory system that wants deterministic replay — audit trails, regression-debug reproduction, cross-machine rehydration — needs all three. The lifecycle's value is not retrieval lift but the substrate it provides for every other claim in the paper: §A.4.7's PRF×SP operating point is only defensible because the ingest state is replayable from the decision log.

A4.2.1 Lifecycled schemas vs Letta-style memory blocks

Letta's human/persona blocks (Packer et al., 2023) and Engram's SCHEMA memories are adjacent in design space — both are named, mutable agent state — but differ on three axes that matter for governed deployment.

1. **Mutation discipline.** A Letta block is a string the LLM rewrites in place via tool calls; prior content is reachable only through external chat history. An Engram schema is a fold over an append-only decision log (§3.7.4): every state change is a typed event with a reason field, and previous state is recoverable by replaying any prefix. Letta optimises for prompt-window compactness; Engram for audit-grade replay.
2. **Identity stability under adversary.** A Letta block is identified by name; whoever can issue a tool call can rewrite it. Engram schemas are content-addressed (cluster centroid + family key, §3.7.5) with a quorum-gated DEPRECATE primitive (§A.4.6, §A.6.16) requiring k independent emitters over a w -event window. A single compromised emitter cannot take a schema down.
3. **Recovery semantics.** Letta has no first-class undo: a corrupted block must be reconstructed from chat history by the same LLM that may be the corruption source. Engram's lifecycle DAG includes a RECOVER edge — verified under randomized event interleaving (see §A6) — that re-promotes a DEPRECATED schema once subsequent evidence reaches the same quorum. RECOVER is path-dependent on the decision log, not on current LLM judgment.

The trade is real: Letta is cheaper to operate (no append-only log, no quorum bookkeeping) and for chat-assistant workloads the savings dominate. Engram's bet is that the substrate cost is recovered the first time a regulator, debugger, or post-incident reviewer asks "what did this agent know and when" — a question Letta's named-block design cannot answer without external instrumentation.

A4.3 The honest version of "consolidation lifts retrieval"

§A.4.6 forces a re-statement of the consolidation claim. A naive reading of the §87 pipeline says "more stages, more lift." The bisection says otherwise: on the Mem0-shaped LoCoMo10 fixture, **one stage (episode extraction) carries the full retrieval delta, and seven of the eleven downstream stages emit identically-zero per-pair diffs against any prefix that already includes extraction.** Two of the four non-trivially-moving downstream stages (`schema_update`, `appraisal`) are *negative* on $\Delta\text{hit}@1$ at point-estimate, and neither survives a paired bootstrap.

This is stronger than "the rest of the pipeline doesn't help": it falsifies stage-additive lift attribution. Stage ablations that report a single end-to-end metric and treat a positive delta as evidence-of-mechanism are under-specified — the lift was already present before the ablated stage ran. Our v0.2 minimum: **leave-one-out necessity + paired bootstrap on the per-question diff**, with multiple-comparison cost paid explicitly.

The architectural implication: the lifecycle stages are **not retrieval mechanisms**. They are governance — dedup at write, schema as a writable cluster, appraisal as a salience signal for downstream policy. They earn their place on §A4.2's grounds, not §4's.

Verdict. Lifecycle consolidation does not improve retrieval; it provides governance. Future ablations of "consolidation" features must report leave-one-out + paired-bootstrap with multiple-comparison cost paid explicitly.

A4.4 The PRF latency myth

A controlled cProfile re-measurement (§A.4.15-profile, $n=30$ k, 200 paired queries) falsifies the staged claim that PRF "doubles recall p50." PRF-only p50 is *0.86 ms below* baseline (40.08 vs 40.94) and PRF-only p95 is *4.7 ms below*; `share_prior`-only matches baseline within noise. Only the combined PRF×`share_prior` arm shows real overhead (+14 % p50, +24 % p95). The dominant cost across all arms is `sqlite3.Connection.execute` (~73 % of cumulative recall time); PRF doubles `engine.search` call count but adds only ~6 % to its cumulative cost because the second pass hits warm pages.

Generalizing: single-shot microbenches with small n on a hot data path are noise-dominated at the ms scale. Latency claims for a retrieval lever must be paired (same query stream, warm cache) and reported as a percentile distribution, not a mean. v0.2 standing rule: no latency claim ships without $n \geq 200$ paired queries and matched ingest. The candidate-pool prune for v0.3 is justified for the *both* arm only.

Verdict. Single-shot ms-scale microbenches are noise-dominated. Standing rule: paired $n \geq 200$ + percentile distribution, or no latency claim.

A5. Extended Threats

This appendix expands §6's main-body threats with the supporting analyses that the ACL 2-column 6-page Industry Track body cannot accommodate. Each is referenced by a one-sentence pointer at the end of §6; the full content lives here.

A5.0 Sibling-lexical paraphrase replication on service

§6.1 reports the paraphrase replication on the strongest lexical cell (`tool`). To check whether the paraphrase collapse is `tool`-specific or generalizes across the lexical axis, we re-ran the same protocol on `service` (closed-vocabulary proper-noun answers: `aws`, `gcp`, `azure`, ...):

K	hash Δ hit@1 (paraphrased)	ST Δ hit@1 (paraphrased)
2	-0.094 [-0.219, +0.031]	+0.109 [+0.047, +0.188]
4	-0.031 [-0.102, +0.039]	+0.156 [+0.094, +0.219]
8	+0.031 [-0.008, +0.070] null	+0.141 [+0.098, +0.188]
16	+0.037 [+0.012, +0.062]	+0.105 [+0.078, +0.133]

Compared to the fixed-template baseline (hash service K=16: +0.057 [+0.025, +0.088]), the paraphrased lift shrinks to +0.037 — about 65% of the fixed-template effect — **but stays CI-strictly above zero**. Unlike `tool`, hash on `service` survives memory paraphrase at K=16. The conservative reading: the paraphrase collapse on `tool` is real, but the broader claim is not "hash trigrams die under paraphrase" — it is "hash retention under paraphrase depends on how lexically distinctive the answer vocabulary is at the character-trigram level." `service`'s answer set (short proper nouns: `aws`, `gcp`, `azure`) preserves more discriminative character-trigrams across template variation than `tool`'s (`git`, `docker`, `postgres`). The two-axis claim survives on the **lexical/intent split**, but the within-lexical paraphrase robustness has tag-level structure.

Outputs: `bench/results/ec_sweep_{hash,st}_service_n32_K16_paraphrased{,_ci}.json`.

A5.1 Single embedder per family

§6 tests exactly one hash-trigram dim (256) in the headline figure and one sentence-transformer (MiniLM-L6-v2). To check whether the lexical-axis hash lift is specific to dim=256, we ran a hash-dim ablation at the strongest lexical cell (`tool`, K=8, n=32):

dim	Δ hit@1	95% CI
128	-0.0195	[-0.0742, +0.0312]
256	+0.0664	[+0.0039, +0.1250]
512	+0.0586	[+0.0000, +0.1172]
1024	+0.0703	[+0.0117, +0.1250]

Dim=128 is below noise; 256–1024 sit on a CI-positive plateau with no monotone scaling. The two-axis claim is robust across hash dim $\in \{256, 512, 1024\}$; only the smallest sketch (128) collapses. Extending the embedder grid to BGE / E5 on the dense side and to character-quintgrams on the sketch side would tighten the family generalization claim. The BGE-large-en-v1.5 (1024-d) follow-up has since landed (Appendix §A.4.16) and rejects the encoder-capacity hypothesis: the two-axis result survives a 2.7 $\times$ -parameter encoder swap, with BGE

losing

on lexical-discriminator tags. The character-quintgram sketch is left as future work and flagged as a named scope limitation rather than an open TODO.

A5.2 hit@1 only

§6 reports `hit@1` because it is the worst-case metric and the most sensitive to retriever ranking. `hit@5` and MRR mostly converge toward 1.0 on the entity-collision corpus and are uninformative. LoCoMo numbers are reported across all metrics in `SCALE_REPORT.md`.

A5.3 Single-process SQLite

All operational latency numbers are single-writer SQLite/FTS5. Multi-process write contention is not measured. A concurrency torture suite (≥ 50 writers $\times \geq 50$ readers, see §A6) passes correctness invariants under contention, but throughput-under- contention is not yet a reported number.

A5.4 Single machine, single OS

All wall-clock and throughput numbers come from one Linux x86_64 workstation (see `REPRODUCIBILITY.md` for the full env). p50/p95/p99 ingest latencies and the 100k constant-p99 claim therefore should be read as *invariant within this hardware envelope*. The replay discipline argued for in §A4.2 is what makes cross-machine reproduction tractable — point-estimates may shift, but the event-sourced lifecycle guarantees that the *shape* of any reported distribution is reproducible from a pinned decision log; the diff-results acceptance gate (`REPRODUCIBILITY.md` §4) operationalizes this with $\pm 0.5pp$ / $\pm 25\%$ -latency tolerances.

A5.5 Author-as-annotator on tag definitions

The lexical/intent dichotomy of §3.3 is author-defined: `service` and `tool` were labeled as closed-vocabulary lexical, the other three as open-vocabulary intent-style, without an inter-annotator agreement protocol. The labels are derived from the answer-set construction (closed enum vs free phrasal slot), not from a third party's judgment, so the categorization is *traceable* but not *independently validated*. A reviewer could plausibly relabel `technical` as lexical and recover a different two-axis fit. We therefore present the dichotomy as a hypothesis the data is consistent with, not as the unique correct partition.

A6 Implementation and Testbed Traceability

This appendix is a one-stop reference for the implementation paths and reproduce scripts cited by abstraction in the body. Reviewers who do not need to inspect the testbed code can skip it; reviewers who

do

— particularly when checking the deterministic-replay arguments in §3.7, §5.3, and §6.6 — will find every cited file here, with what it pins and how many cases / properties it exercises.

We treat this index as version-controlled supplementary material. Every path below is verified to exist at the tagged release by `scripts/verify_repro_artifacts.sh`, the same script that gates the artifact registry (§REPRODUCIBILITY).

A6.1 Pure-reducer invariants for the schema lifecycle (§5.3)

The three lifecycle invariants in §5.3 — event-sourced state, permutation-stable family clustering, real-time monotone decay — correspond to the following property-test files:

Invariant (§5.3)	Test file (<code>tests/property/</code>)	Notes
Lifecycle state = fold of decision log	<code>test_schema_lifecycle.py</code> , <code>test_schema_decision.py</code> , <code>test_schema_decision_x_reducer.py</code>	Includes RECOVER edge fuzzing
Family clustering invariant under permutation	<code>test_schema_family.py</code> , <code>test_schema_family_decision.py</code> , <code>test_schema_family_window.py</code>	Permutates the input fact stream; asserts assignment
Decay monotone in real time, not arrival	<code>test_schema_decay.py</code>	Fuzzes interleaved <code>tick()</code> / <code>update()</code> calls

Hypothesis is configured at `max_examples=200` per property in CI; the lifecycle reducer is therefore exercised over $\approx 1.6\text{k}$ randomized event traces per CI run.

A6.2 Write-path independence (§3.7.3)

The two invariants on deduplication $\times$ extraction-confidence independence (I1/I2 in §3.7.3) are pinned by:

Invariant (§3.7.3)	Test file
I1: dedup outcome invariant under keeper-side confidence	<code>tests/property/test_dedup_extraction_confidence_indep.py</code>
I2: deduped write does not mutate keeper's confidence	(same)

A6.3 Concurrency torture suite (§6.6)

The ≥ 50 writers $\times$ ≥ 50 readers correctness suite referenced as `testbed` sanity in §6.6 lives at:

Concern	Test file (<code>tests/concurrency/</code>)
Generic write/read race coverage	<code>test_race_conditions.py</code>
Dedup race	<code>test_dedup_race.py</code>
High-fanout writer/reader load	<code>test_high_fanout.py</code>
JSONL append-buffer concurrency	<code>test_jsonl_buffer_concurrency.py</code>
JSONL truncate race	<code>test_jsonl_truncate_race.py</code>

The suite asserts correctness invariants (no torn writes, no lost events, ACL never lifts under contention) but does not yet report throughput; this is the §6.6 limitation.

A6.4 Adversarial / ACL invariants (Appendix A2)

The path-by-path enumeration of the 11 ACL side-channel audits is already in §A2 (Security Audits). §A6 is intended for body-level abstractions; readers tracing §A2 references should consult that appendix directly. The full list of `tests/adversarial/test_*.py` audit files is duplicated below for convenience:

Audit subject (§A2)	Test file (<code>tests/adversarial/</code>)
PRF expansion ACL	<code>test_prf_acl_side_channel.py</code>
PRF + IDF gate ACL	<code>test_prf_idf_acl_side_channel.py</code>
Share-prior ACL	<code>test_share_prior_acl_side_channel.py</code>
Lifecycle cache ACL	<code>test_lifecycle_cache_acl_side_channel.py</code>
Schema-id targeted suppression	<code>test_schema_id_targeted_suppression.py</code>
Vector-channel ACL	<code>test_vector_channel_acl_side_channel.py</code>
Mechanical-merge ACL	<code>test_mechanical_merge_acl_side_channel.py</code>
Fact-extraction ACL inheritance	<code>test_fact_extraction_acl_inheritance.py</code>
Write-dedup ACL	<code>test_write_dedup_acl_side_channel.py</code>
Write-dedup $\times$ ACL race	<code>test_write_dedup_acl_race.py</code>

Audit subject (§A2)	Test file (<code>tests/adversarial/</code>)
Extraction-confidence ACL	<code>test_extraction_confidence_acl_side_channel.py</code>
Schema deprecate quorum	<code>test_schema_deprecate_quorum.py</code>
Mech-merge × write-dedup composition (CXE)	<code>test_cxe_mech_merge_x_write_dedup_compose.py</code>
Mech-merge × extraction-confidence (CXF)	<code>test_cxf_mech_merge_x_extraction_confidence.py</code>

A6.5 Scale evidence (§4.14, §6.4)

The 1M-memory single-writer characterization referenced in §1, §4.14, and §6.4 corresponds to:

Claim	Test file (<code>tests/scale/</code>)
Tail-100k p99 write latency at 100k	<code>test_ingest_scale.py::test_scale_recall_after_100k</code>
Same-harness 1M extension	<code>test_ingest_scale.py</code> (1M fixture)

A6.6 Why this index lives here, not in the body

Three of Engram’s invariants — event-sourced lifecycle, mechanical merge, write dedup — are implementation-level guarantees that support the measurement claims rather than being measurement claims in their own right. The body of the paper therefore states each invariant in academic abstraction (deterministic fold over events, permutation-stable cluster identity, real-time monotone decay) and defers per-property file paths and case counts to this appendix. This separation matches the standard distinction in systems papers between *what is true of the system* (body) and *how the truth is verified at the implementation* (appendix), and it leaves the body sections short enough that a reviewer focused on the entity-collision protocol can read the methodology subsections in linear time.

A7. Extended Methods

This appendix expands §3’s main-body protocol description with the supporting method detail that the ACL 2-column 6-page Industry Track body cannot accommodate. Each subsection is referenced by a one-sentence pointer at the end of §3.3; the full content lives here.

A7.1 LoCoMo and adaptive-vw experiment

For the adaptive-vw null result, we use the LoCoMo adapter (`evals/locomo_adapter.py`) over $n=1978$ questions and compute (a) the per-query oracle `hit@1` (best vw per query), (b) static-best vw, and (c) tau-thresholded policies on `bm25_top1 - bm25_top2`, normalized gap, and `crowdedness@0.95`.

A7.2 share_prior reranker (Personize §96 adaptation)

We adapt the *shared prior* signal from Personize (§96) into a candidate-pool reranker. Rather than scoring each candidate against the query alone, we build an undirected entity-sharing graph over the `top_pool_size` fused candidates: an edge connects two candidates whenever their extracted named-entity sets intersect. Each candidate’s *multi-mate degree* — the number of pool members it shares at least one entity with — is a within-pool popularity signal that, on multi-hop bridge queries, fires for the bridge fact even when the bridge fact does not lexically match the query string. We add a bounded boost $\alpha \cdot \text{deg} / \text{max_deg}$ to each candidate’s fused score and re-sort.

Rank-0 preservation invariant. We cap the per-candidate boost so that no non-rank-0 candidate’s post-boost score can equal or exceed the original rank-0 score:

```
score'_i = score_i + min( $\alpha \cdot \text{deg}_i / \text{max\_deg}$ ,  $\text{score}_0 - \text{score}_i - \epsilon$ )
```

By construction $\text{score}'_i < \text{score}_0$ for all $i \geq 1$, so `hit@1` is mathematically — and, as we verify in §4, empirically — never regressed. The reranker is therefore safe to ship default-off behind a flag and accept-only at higher k . Verified across 5 seeds × 5 α values × 2 recipes × 3 pool sizes = 150 arms: $\Delta \text{hit}@1 \equiv 0.000$ everywhere (SHARE_PRIOR_REPORT.md, all tables).

Protocol. We evaluate on two synthetic recipes that probe orthogonal failure modes: **Unique-entity** (do-no-harm; each gold has a distinct anchor entity, no graph structure; share_prior expected inert at `hit@1` and at most mildly noisy at `hit@k`); and **Bridge** (target signal; each query is a pair of facts joined by a shared anchor entity; gold pair facts share a degree-2 hub in the graph and should be promoted).

For each recipe we sweep $\alpha \in [0.02, 0.30]$, `pool_size` $\in \{20, 40, 80\}$, and `entity_weight` $\in \{0.0, 0.1, 0.2, 0.3\}$ (the channel that also lives in the fused scorer), across seeds {42..46} and corpus scales from 60 to 120 query pairs with up to 200 distractor facts. We report paired $\Delta \text{hit}@k$ and $\Delta \text{pair_recall}@k$ versus the same fused-scorer baseline (no rerank) on identical seeds, with $n=10$ paired-bootstrap 95% CIs (§A.4.7).

Pool-size dilution. As `pool_size` grows, the $\text{deg} / \text{max_deg}$ distribution becomes less peaked; the relative ordering between the bridge fact and the rest of the pool flattens. We confirm this empirically (`pool_size` $\in \{20, 40, 80\}$ at $\alpha = 0.10$, $\text{ew} = 0.10$ on the corrected small bridge corpus: $\Delta \text{pair}@10 = \{+0.050, -0.025, +0.025\}$; §A.4.7).

Adaptive α (opt-in regularizer). A constant α boosts identically whether the candidate pool’s most-shared entity has degree 1 or degree 9, even though the rank-0 cap already saturates the “graph is a near-matching” case. We schedule a tapered multiplier on `max_deg`,

```
 $\alpha_{\text{eff}}(\text{max\_deg}) = \alpha \cdot 1 / (1 + \max(0, \text{max\_deg} - 1) / 4),$ 
```

i.e. $\alpha_{\text{eff}} = \alpha$ at $\text{max_deg} \in \{0, 1\}$ and decays monotonically toward 0 as the pool densifies ($\text{max_deg} = 5 \rightarrow 0.5\alpha$, $\text{max_deg} = 9 \rightarrow \alpha/3$). This lives behind `RetrievalConfig.share_prior_adaptive_alpha` (default `False`) and ships as a hedge against α over-shoot rather than an unconditional improvement; §A.4.7 reports the empirical regime where it pays off.

A7.3 PRF entity expansion (dominance-gated)

Reranking cannot promote what the first-pass top-K never admitted, so we close the upstream gap with pseudo-relevance feedback (PRF) restricted to entity tokens. Given a first-pass top-K, we extract entities from each of the top-`top_k_for_prf` results and rank them by document frequency within the pool. We then expand the query with the most-dominant entity *only if* it appears in at least `min_dominance · top_k_for_prf` of the first-pass results; otherwise we issue the original query unchanged. The dominance gate is the do-no-harm guarantee: when no single entity dominates the first-pass pool (the typical unique-entity workload), we do not expand at all, so unique-recipe `hit@1` is not regressed by classic PRF over-expansion.

The expansion is wired into `RetrievalEngine.search` behind `RetrievalConfig.query_expansion_min_dominance: float | None` (default `None` = off). At runtime, when a non-`None` value is configured, the engine performs a first-pass retrieval, computes the dominance-gated entity, and (if the gate fires) re-issues a single expanded query whose results replace the first-pass results for the user-visible top-k. The operating point we recommend, defended in §A.4.7, is `min_dominance = 0.3` with `top_k_for_prf = 20`.

A7.4 Governed Memory integration (secondary systems note)

Scope of this section. §A7.4 documents the four write-path primitives (dual extraction, cosine dedup, mechanical merge, schema lifecycle) that make our paired-bootstrap CIs replayable. Reviewers focused exclusively on the entity-collision protocol may skip this subsection: nothing in §4 depends on schema- lifecycle behaviour beyond the testbed-determinism requirement we surface here. The lifecycle's *own* retrieval behaviour (and its non-contribution to recall) is the subject of §A.4.6's bisection and §A4.2's discussion — also orthogonal to the headline two-axis claim.

Ingram's write/consolidation path adopts four primitives from the Personize Governed Memory proposal (arXiv:2603.17787). This subsystem is a v0.2 systems contribution; the paper's headline claims do not depend on it.

A7.4.1 Dual extraction with per-fact confidence

`FactExtraction` returns each fact with `extraction_confidence` $\in [0, 1]$; `RetrievalConfig.use_extraction_confidence` (default `True`) multiplies the fused score by this factor and records it in `ScoredMemory.sources`.

A7.4.2 Write-side cosine dedup at 0.92

`StorageConfig.write_dedup_threshold = 0.92` gates `store.upsert` against the in-process vector index. This is a pure write-path filter — not a merge — orthogonal to extraction confidence. The independence is formalized as two invariants (I1/I2): the deduplication outcome is invariant under keeper-side confidence, and a deduped write does not mutate the keeper's confidence. Both invariants are verified by randomized property tests over the joint state space; case counts and traceability in §A6.

A7.4.3 Mechanical merge (no-LLM)

`MechanicalMerge` (Stage 12) walks `store.all_active()` and, for any pair above `merge_threshold = 0.95`, suppresses the lower- salience member. No LLM call; order-stable; idempotent under fixed embeddings.

A7.4.4 Schema lifecycle as a pure reducer

`schema_lifecycle.py` folds an immutable `SchemaLifecycleEvent` log into a `{schema_id: SchemaState}` snapshot. The transition DAG is `INFERRED → PROMOTED → DEPRECATED`, `INFERRED → DEPRECATED`, `DEPRECATED → INFERRED` (recovery only, fresh `window_id`). Five invariants are enforced: transition closure, CREATE-once, no-op on unknown id, version monotone under status changes, and recovery freshness. `RetrievalConfig.respect_schema_lifecycle` (default `True`) filters `DEPRECATED` candidates before scoring. Each invariant's Hypothesis property test and the bug-class it catches are consolidated in §A.4.17.

A7.4.5 What we did not adopt

We omit Personize's LLM-mediated merge (replaced by §A7.4.3's mechanical merge) and their governance-review stage (no analogue in single-user deployments).